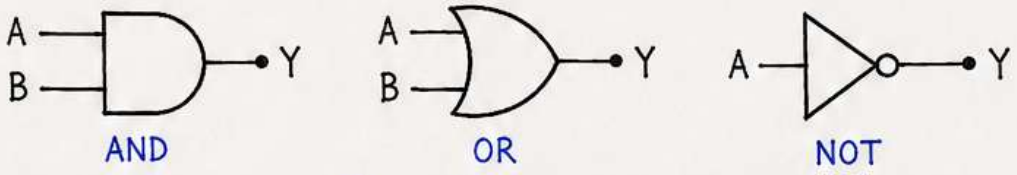
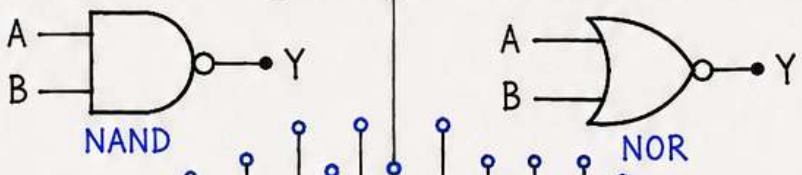
0 1 0 1 1 0 1 0 1 1 0 1 0 1

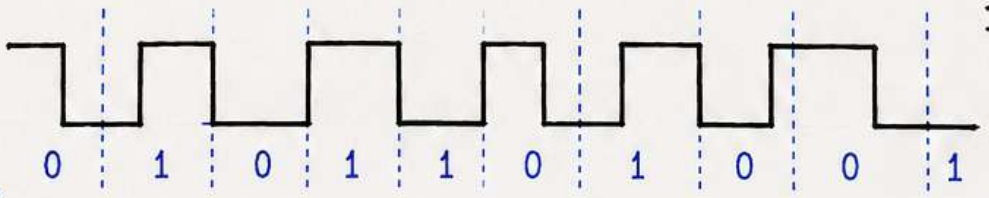
DIGITAL ELECTRONICS





by

Konda Tharun Vivekananda



INDEX

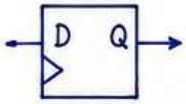
S.No.	Main Heading / Concept	Page
1	Introduction to Digital Electronics	1
2	Advantages & Disadvantages of Digital Systems ; Number System Introduction & Classification	2
3	Number Systems — Different Ranges ; Base Conversion Steps	3
4	Binary to Hexa Conversion Examples ; Binary to Hexa Conversion — Floating Point Examples	4
5	Binary to Decimal Conversion ; Octal to Binary Conversion	5
6	Octal to Decimal Conversion ; Octal to Hexadecimal Conversion	6
7	Hexa Decimal to Binary Conversion ; Hexa Decimal to Decimal Conversion	7
8	Hexa Decimal to Octal Conversion ; Numerical Complements	8
9	Numerical Complements — 1's & 2's ; 9's & 10's ; 15's & 16's Complements	9
10	Unsigned & Signed Representation ; Binary Subtraction using 1's & 2's Complement	10
11	Subtraction using 9's & 10's Complement	11
12	Fixed Point Representation ; Floating Point Representation	12
13	Boolean Algebra	13
14	Logic Gates & Implementation of Gates using Universal Gates NAND and NOR	14
15	Boolean Minimization — Introduction ; Boolean Properties	15
16	Boolean Minimization using K-Maps — Steps ; All Types of K-Map Examples	16
17	Introduction to Implicants and Types ; All Types of K-Map Examples	17

INDEX

S.No.	Main Heading / Concept	Page No.
1	Combinational Circuits – Introduction & Design Procedures; Advantages, Disadvantages & Applications; Combinational vs Sequential Circuits	18
2	Arithmetic Circuit Design – Adders : Half-Adder	19
3	Arithmetic Circuit Design – Adders : Full-Adder; Arithmetic Circuit Design using Building Blocks	20
4	Arithmetic Circuit Design – Subtractors : Half-Subtractor & Full-Subtractor	21
5	Multiplexer – 2×1 Design using Logic Gates; 2×1 MUX using Logic Gates	22
6	Multiplexer – 4×1 Design using Logic Gates; 4×1 MUX using Logic Gates	23
7	Multiplexer – 8×1 and 16×1 Design using Logic Gates	24
8	Multiplexer (MUX) – Introduction, Applications, Advantages, Disadvantages	25
9	MUX Tree (Multiplexer Tree)	26
10	MUX Tree – Examples of 16×1 MUX using 4×1 MUX; 64×1 MUX using 8×1 MUX	27
11	Demultiplexer (De MUX) – Introduction	28
12	Demultiplexer (De MUX) – 1×4 and 1×8	29
13	Decoder – Introduction & 2:4 Decoder	30
14	Decoder – 3:8 and 4:16	31
15	Encoder	32
16	Encoder – 8:3 and Priority Encoder – 4:2	33
17	Comparator – General; Comparator – 1 Bit	34
18	Comparator – 2 Bit and 4 Bit	35
19	8 Bit Comparator using 4 Bit Comparator; Magnitude Comparator – 4 Bit	36
20	Code Converters – BCD to Decimal; Gray to Binary; BCD to Excess-3 (XS-3) Code	37

1010
0101

INDEX



S.No.	Main Heading / Concept	Page No.
1	Sequential Circuits — Introduction; Combinational vs Sequential Circuits; Types of Sequential Circuits	39
2	Flip-Flops — Definition, Types, Comparison; Edge Triggered & Level Triggered Flip-Flops; Basic Flip-Flop Circuits	40
3	Flip-Flops — JK Flip-Flop; SR Flip-Flop; D Flip-Flop; T Flip-Flop	41
4	Latches — SR Latch; Gated SR Latch; D Latch; Gated D Latch	42
5	Latches — SR Latch; Gated SR Latch; D Latch; T Latch	43
6	All Flip-Flop Conversions — JK to SR; JK to T; JK to D	44
7	All Flip-Flop Conversions — SR to JK; SR to T; SR to D	45
8	All Flip-Flop Conversions — D to JK; D to T; D to SR	46
9	All Flip-Flop Conversions — T to JK; T to D; T to SR	47
10	Counters — Introduction; Classification of Counters; Basic Counter; Synchronous Counter; Truth Table; Waveform; Applications	48
11	Counters — Synchronous Counters	49
12	Counters — Asynchronous (Ripple) Counters	50
13	Shift Registers — General	51
14	Registers — General	52
15	Shift Registers — Types: SISO, SIPO	53
16	Shift Registers — Types: PISO, PIPO	54
17	Shift Registers / Counters — Types: Ring Counter, Johnson (Twisted Ring) Counter	55
18	Frequency Divider and Multiplier	56
19	Edge Detector Circuit — Rising Edge and Falling Edge Detection	57
20	State Machine	58
21	Finite State Machine (FSM) — Introduction, Types, Need, Components, Classification, Design Steps	59
22	Finite State Machine Examples — Sequence Detector and Traffic Light Controller	60

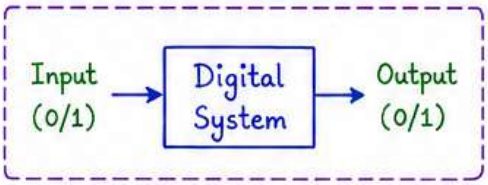
1010
0101

Digital Electronics

1. Introduction to Digital Electronics

- Digital Electronics deals with electronic circuits that process discrete (binary) signals.
- It works with two logic states : 0 and 1.
- Information is represented in binary form.
- Digital circuits perform operations like logic, arithmetic, storage and control.
- Widely used in computers, microprocessors, communication, embedded systems, etc.

Basic Digital System

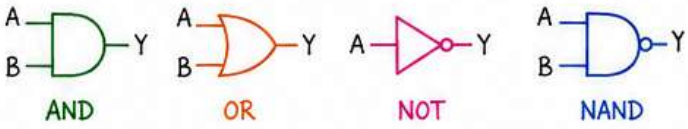


Logic Levels

Logic 1 (HIGH) → +V (e.g., 5V)

Logic 0 (LOW) → 0V (GND)

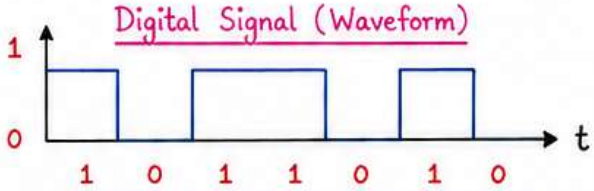
Common Digital Gates



2. Characteristics of Digital Systems

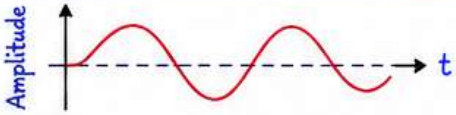
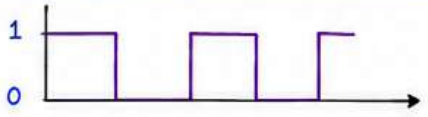
- ★ Operate with discrete values (0 and 1).
- ★ High noise immunity.
- ★ Easy storage and reproduction of data.
- ★ Reliable and accurate operation.
- ★ Supports complex operations using simple logic circuits.
- ★ Programmable and flexible.
- ★ Low power consumption with modern technologies.

Digital Signal (Waveform)



Digital signals have sudden transitions between two discrete levels.

3. Analog vs Digital Systems — Comparison

Parameter	Analog System	Digital System
Signal type	Continuous in time and amplitude	Discrete in time and amplitude
Signal levels	Infinite number of levels	Two levels (0 and 1)
Waveform		
Noise immunity	Low (Easily affected by noise)	High (Less affected by noise)
Accuracy	Less accurate	Highly accurate
Storage	Difficult	Easy
Processing	Complex circuits required	Simple logic circuits
Reproduction	Deteriorates with copying	Exact copy possible
Power consumption	Generally higher	Generally lower (in modern ICs)
Examples	 Microphone, AM radio, Thermometer	 Computer, Digital watch, Calculator

Digital Electronics

1. Advantages of Digital Systems

① **High Noise Immunity** : Digital signals use discrete levels (0 and 1), so they are less affected by noise compared to analog signals.

② **Reliability** : Digital systems give consistent and accurate results.

③ **Easy Storage** : Digital data can be stored for long time without degradation.

④ **Easy Processing** : Complex operations can be performed using simple logic circuits.

⑤ **Programmable** : Digital systems can be easily programmed and updated.

⑥ **Reproducibility** : Digital data can be copied and transmitted without loss of quality.

⑦ **Integration** : Highly suitable for VLSI implementation and large scale integration.

⑧ **Low Power (Modern ICs)** : Advanced digital ICs consume less power.



Digital systems are the backbone of modern computers, communication, control systems and embedded applications.

2. Dis- Advantages of Digital Systems

① **Requirement of A/D and D/A Conversion** : Analog signals need converters to interface with digital systems.

② **Quantization Error** : Occurs during A/D conversion.

③ **Limited Resolution** : Depends on the number of bits used.

④ **Higher Complexity for some operations** : Eg: Multiplication, Division need more hardware.

⑤ **Clock Dependency** : Most digital systems require a clock signal.

⑥ **Design Effort** : More design and verification time compared to simple analog circuits.



No system is perfect. Digital systems have limitations but their advantages far outweigh the disadvantages in modern applications.

Quick Example (Base-2)

$N = 10110_2$

1's complement = 01001_2 (invert bits)

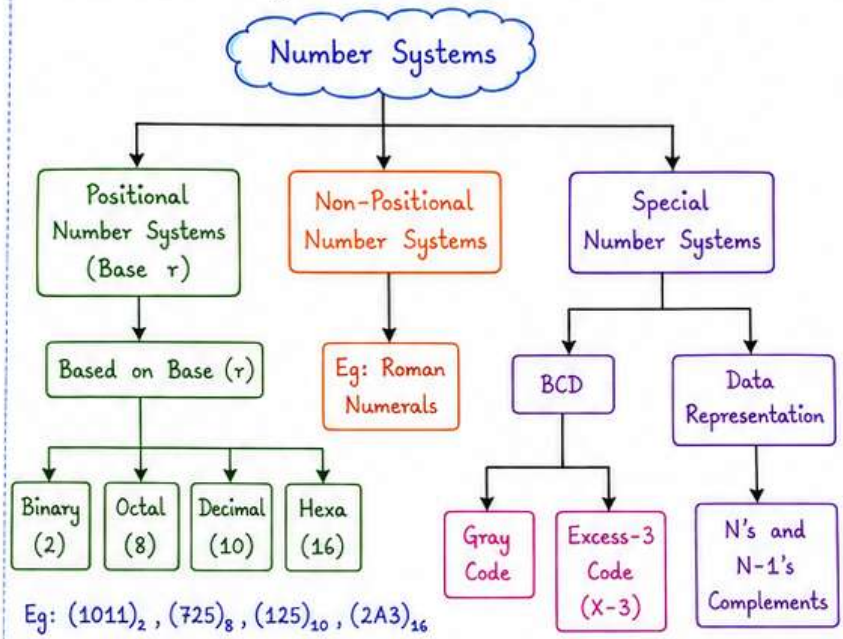
2's complement = $01001_2 + 1 = 01010_2$

Key Point

Choice of number system and representation depends on application requirements.

3. Number System Intro & Classification Tree

- A number system is a method of representing numbers using a set of symbols called digits.
- It is defined by its base (radix).



Eg: $(1011)_2$, $(725)_8$, $(125)_{10}$, $(2A3)_{16}$

(a) BCD Codes

Decimal	8421 BCD	Gray Code	Excess-3 Code (X-3)
0	0000	0000	0011
1	0001	0001	0100
2	0010	0011	0101
3	0011	0010	0110
4	0100	0110	0111
5	0101	0111	1000
6	0110	0101	1001
7	0111	0100	1010
8	1000	1100	1011
9	1001	1101	1100

(b) Data Representation (Complements)

N's Complement (r's Complement)	N-1's Complement (Diminished r's Complement)
• Subtract each digit from (r - 1). • r = Base • eg: In base-2 → 1's complement eg: In base-10 → 9's complement	• Subtract each digit from (r - 1) then add 1 to the result. • eg: In base-2 → 2's complement eg: In base-10 → 10's complement

Interview Point

☆ Binary system is most widely used in digital electronics. Other systems are used for convenience, error reduction and special applications.

Author: Konda Tharun Vivekananda

1. Number System of Different Ranges

➔ A number with base (r) has r digits : 0 to (r-1)

Number System	Base (r)	Digits Used	Example Numbers	Range for n-digits (Min to Max)
Binary	2	0, 1	$(1011)_2$	0 to $(2^n - 1)$
Octal	8	0-7	$(347)_8$	0 to $(8^n - 1)$
Decimal	10	0-9	$(259)_{10}$	0 to $(10^n - 1)$
Hexadecimal	16	0-9, A-F	$(2AF)_{16}$	0 to $(16^n - 1)$

For n-digits in base r:

Minimum = 0 , Maximum = $r^n - 1$

Common Ranges (n = 4 digits)

Base	Min Value	Max Value	Total Numbers
Binary	2	15 $(1111)_2$	16
Octal	8	511 $(777)_8$	512
Decimal	10	9999	10000
Hex	10	$FFFF_{16}$	65536

➔ As base increases, the range for same number of digits increases.

Signed Numbers (n-bit)

Range for n-bit Signed Binary (Using 2's Complement)

$= -(2^{n-1})$ to $(2^{n-1} - 1)$

Example : For n = 4 → Range = −8 to +7

Important Notes

- ✓ Number of digits in a system decides its range.
- ✓ Larger the base, larger the range for same digits.
- ✓ Binary uses only 0 and 1, others use more symbols.
- ✓ Hexadecimal is very compact representation.

Interview Point

Number systems are the foundation of digital computing. Efficient conversion between bases is essential in design, verification and debugging.

2. Base Conversion Steps (General Method)

To convert a number from base r_1 to base r_2

General Procedure ($r_1 \rightarrow r_2$)

1. Convert the given number from base r_1 to decimal (base 10).
2. Convert the decimal number to base r_2.
3. Write the digits in proper order.

Step-1 : Convert from any base (r_1) to Decimal

- Multiply each digit by its positional weight $(r_1)^i$.
- Add all the products.
- That sum is the decimal equivalent.

Positional Weight

	...	$(r_1)^3$	$(r_1)^2$	$(r_1)^1$	$(r_1)^0$	$(r_1)^{-1}$	$(r_1)^{-2}$ ...
For base r_1 :	...						
Place value :	...	8's	4's	2's	1's	$\frac{1}{2}$'s	$\frac{1}{4}$'s ...

Step-2 : Convert Decimal to any base (r_2)

(a) Integer part (Division Method)

- Divide decimal number by r_2.
- Note the remainder.
- Repeat with quotient.
- Stop when quotient = 0.
- Read remainders in reverse order.

(b) Fractional part (Multiplication Method)

- Multiply fractional part by r_2.
- Note the integer part.
- Repeat with remaining fraction.
- Stop when fraction = 0 or desired precision reached.
- Read integers in order.

Step-3 : Combine Results

- Write integer part result followed by '.' and fractional part result.
- This gives the number in base r_2.

Quick Example : (Convert $(1011.01)_2$ to Decimal)

$$(1011.01)_2 = 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 0\times2^{-1} + 1\times2^{-2}$$
$$= 8 + 0 + 2 + 1 + 0 + 0.25$$
$$= 11.25_{10}$$

Short Tricks / Tips

- ★ To binary → Decimal : use positional weights.
- ★ Decimal → Binary : use division for integer, multiplication for fraction.
- ★ Decimal → Octal : group binary in 3 bits.
- ★ Decimal → Hex : group binary in 4 bits.
- ★ Always check the base and write the subscript.

1. Binary to Octal Conversion Examples

☆ Rule : Group the binary digits in sets of 3 starting from the binary point (.). If required, add leading 0's on left and trailing 0's on right to make groups of 3.

(A) Normal (Integer) Examples

Ex 1 : $(101101.11)_2 \rightarrow (?)_8$

Soln : 101 | 101 . 110 → 5 5 6

$\therefore (101101.11)_2 = (55.6)_8$

Ex 2 : $(1110101.001)_2 \rightarrow (?)_8$

Soln : 001 | 110 | 101 . 001 → 1 6 5 1

$\therefore (1110101.001)_2 = (165.1)_8$

Ex 3 : $(100111010.101)_2 \rightarrow (?)_8$

Soln : 100 | 111 | 010 . 101 → 4 7 2 5

$\therefore (100111010.101)_2 = (472.5)_8$

(B) Floating (Fractional) Examples

Ex 1 : $(0.110101)_2 \rightarrow (?)_8$

Soln : 0 . 110 | 101 → 6 5

$\therefore (0.110101)_2 = (0.65)_8$

Ex 2 : $(0.101011)_2 \rightarrow (?)_8$

Soln : 0 . 101 | 011 → 5 3

$\therefore (0.101011)_2 = (0.53)_8$

Ex 3 : $(1.011101)_2 \rightarrow (?)_8$

Soln : 1 . 011 | 101 → 1 3 5

$\therefore (1.011101)_2 = (1.35)_8$

Key Point (Octal)

★ Each octal digit represents exactly 3 binary bits.
★ Octal digits range from 0 to 7.

2. Binary to Hexa Conversion Examples

☆ Rule : Group the binary digits in sets of 4 starting from the binary point (.). If required, add leading 0's on left and trailing 0's on right to make groups of 4.

(A) Normal (Integer) Examples

Ex 1 : $(101101111.11)_2 \rightarrow (?)_{16}$

Soln : 1011 | 0111 | 1100 → B 7 C

$\therefore (101101111.11)_2 = (B7.C)_{16}$

Ex 2 : $(110101.0111)_2 \rightarrow (?)_{16}$

Soln : 0011 | 0101 . 0111 → 3 5 7

$\therefore (110101.0111)_2 = (35.7)_{16}$

Ex 3 : $(100110101110.1011)_2 \rightarrow (?)_{16}$

Soln : 0100 | 1101 | 0110 . 1011 → 4 D 6 B

$\therefore (100110101110.1011)_2 = (4D6.B)_{16}$

(B) Floating (Fractional) Examples

Ex 1 : $(0.11010101)_2 \rightarrow (?)_{16}$

Soln : 0 . 1101 | 0101 → D 5

$\therefore (0.11010101)_2 = (0.D5)_{16}$

Ex 2 : $(0.10100111)_2 \rightarrow (?)_{16}$

Soln : 0 . 1010 | 0111 → A 7

$\therefore (0.10100111)_2 = (0.A7)_{16}$

Ex 3 : $(1.011101001)_2 \rightarrow (?)_{16}$

Soln : 1 . 0111 | 0100 . 1000 → 1 7 4 8

$\therefore (1.011101001)_2 = (1.748)_{16}$

Key Point (Hexa)

★ Each hexadecimal digit represents exactly 4 binary bits.
★ Hex digits range from 0 to 9 and A to F.

Legend (Binary Grouping)

Group of 3 bits (Octal) — 101 → 5 (Octal)
Group of 4 bits (Hexa) — 1011 → B (Hexa)

Digital Electronics

1. Binary to Decimal Conversion

- Each bit position in a binary number has a weight of 2^n, where $n = 0,1,2,3,\ldots$
- Multiply each bit with its weight and add.
- For floating point, fractional bits have negative powers of 2.

A) Normal Binary Numbers

① $(1011)_2 = ?$

8 4 2 1

$= 1\times8 + 0\times4 + 1\times2 + 1\times1$

$= 8 + 0 + 2 + 1 = 11_{10}$

② $(11010)_2 = ?$

16 8 4 2 1

$= 1\times16 + 1\times8 + 0\times4 + 1\times2 + 0\times1$

$= 16 + 8 + 0 + 2 + 0 = 26_{10}$

③ $(100101)_2 = ?$

32 16 8 4 2 1

$= 1\times32 + 0\times16 + 0\times8 + 1\times4 + 0\times2 + 1\times1$

$= 32 + 0 + 0 + 4 + 0 + 1 = 37_{10}$

B) Floating Point Binary Numbers

① $(101.11)_2 = ?$

4 2 1 . 1/2 1/4

$= 1\times4 + 0\times2 + 1\times1 + 1\times(1/2) + 1\times(1/4)$

$= 4 + 0 + 1 + 0.5 + 0.25 = 5.75_{10}$

② $(110.01)_2 = ?$

4 2 1 . 1/2 1/4

$= 1\times4 + 1\times2 + 0\times1 + 0\times(1/2) + 1\times(1/4)$

$= 4 + 2 + 0 + 0 + 0.25 = 6.25_{10}$

③ $(1001.101)_2 = ?$

8 4 2 1 . 1/2 1/4 1/8

$= 1\times8 + 0\times4 + 0\times2 + 1\times1 + 1\times(1/2) + 0\times(1/4) + 1\times(1/8)$

$= 8 + 0 + 0 + 1 + 0.5 + 0 + 0.125 = 9.625_{10}$

Key Point:
- Integer part uses positive powers of 2.
- Fractional part uses negative powers of 2.

2. Octal to Binary Conversion

- Each octal digit is equivalent to 3 binary bits.
- Convert each octal digit to its 3-bit binary equivalent.
- For floating point, convert both integer and fractional parts separately.

A) Normal Octal Numbers

① $(57)_8 = ?$ $\quad 5 \to 101,\ 7 \to 111$

So, $(57)_8 = (101\ 111)_2 = \underline{(101111)_2}$

② $(123)_8 = ?$ $\quad 1 \to 001,\ 2 \to 010,\ 3 \to 011$

So, $(123)_8 = (001\ 010\ 011)_2 = \underline{(001010011)_2}$

③ $(640)_8 = ?$ $\quad 6 \to 110,\ 4 \to 100,\ 0 \to 000$

So, $(640)_8 = (110\ 100\ 000)_2 = \underline{(110100000)_2}$

B) Floating Point Octal Numbers

① $(7.3)_8 = ?$ $\quad 7 \to 111,\ 3 \to 011$

So, $(7.3)_8 = (111 . 011)_2 = \underline{(111.011)_2}$

② $(12.5)_8 = ?$ $\quad 1 \to 001,\ 2 \to 010,\ 5 \to 101$

So, $(12.5)_8 = (001\ 010 . 101)_2 = \underline{(001010.101)_2}$

③ $(40.6)_8 = ?$ $\quad 4 \to 100,\ 0 \to 000,\ 6 \to 110$

So, $(40.6)_8 = (100\ 000 . 110)_2 = \underline{(100000.110)_2}$

Tip:
* For octal → replace each digit with 3-bit binary.
* Remove leading zeros in integer part (optional).
* Keep fractional part as is.

Quick Reference

Binary place values → ... 32 16 8 4 2 1 . 1/2 1/4 1/8 ...

Octal to Binary Map → 0 → 000, 1 → 001, 2 → 010, 3 → 011, 4 → 100, 5 → 101, 6 → 110, 7 → 111

Binary to Decimal → Sum of (bit × weight)

Octal to Binary → Replace each octal digit with its 3-bit binary equivalent.

Common Mistakes

- × Forgetting to include fractional place weights.
- × Using wrong power of 2.
- × Skipping leading zeros in octal to binary conversion.
- × Mixing up integer and fractional parts.

Digital Electronics

1. Octal to Decimal Conversion

- Each octal digit has a weight of 8^n, where $n = 0,1,2,\ldots$
- Multiply each digit with its weight and add.
- For floating point, fractional part uses negative powers of 8.

A) Normal Octal Numbers

(1) $(57)_8 = ?$

$8^1 \quad 8^0$

$= 5 \times 8 + 7 \times 1$

$= 40 + 7 = 47_{10}$

(2) $(123)_8 = ?$

$8^2 \quad 8^1 \quad 8^0$

$= 1 \times 8^2 + 2 \times 8^1 + 3 \times 8^0$

$= 64 + 16 + 3 = 83_{10}$

(3) $(640)_8 = ?$

$8^2 \quad 8^1 \quad 8^0$

$= 6 \times 8^2 + 4 \times 8^1 + 0 \times 8^0$

$= 384 + 32 + 0 = 416_{10}$

B) Floating Point Octal Numbers

(1) $(7.3)_8 = ?$

$8^0 \quad 8^{-1}$

$= 7 \times 8^0 + 3 \times 8^{-1}$

$= 7 + \frac{3}{8} = 7 + 0.375 = 7.375_{10}$

(2) $(12.5)_8 = ?$

$8^1 \quad 8^0 \quad 8^{-1}$

$= 1 \times 8^1 + 2 \times 8^0 + 5 \times 8^{-1}$

$= 8 + 2 + \frac{5}{8} = 10 + 0.625 = 10.625_{10}$

(3) $(40.6)_8 = ?$

$8^1 \quad 8^0 \quad 8^{-1}$

$= 4 \times 8^1 + 0 \times 8^0 + 6 \times 8^{-1}$

$= 32 + 0 + \frac{6}{8} = 32 + 0.75 = 32.75_{10}$

Tip :
- Write place values for each digit.
- Multiply digit with its corresponding power of 8.
- Add all the products to get decimal value.

Quick Reference

Octal Places → ... 64 8 1 . 1/8 1/64 1/512 ...

Decimal Conversion → Sum of (digit × weight)

Example → $(23.4)_8 = 2 \times 8 + 3 \times 1 + 4 \times 1/8$

$= 16 + 3 + 0.5 = 19.5_{10}$

2. Octal to Hexadecimal Conversion

- Convert octal number to binary (3 bits per digit).
- Group binary in 4 bits from right.
- Convert each 4-bit group to hexadecimal.
- Same steps for fractional part.

A) Normal Octal Numbers

(1) $(57)_8 = ?$ 5 → 101 , 7 → 111

Octal → Binary : $(57)_8$ = 101 111

Group in 4 bits : 0010 1111

Hex : 0010 → 2 , 1111 → F

∴ $(57)_8 = (2F)_{16}$

(2) $(123)_8 = ?$ 1 → 001 , 2 → 010 , 3 → 011

Octal → Binary : $(123)_8$ = 001 010 011

Group in 4 bits : 0001 0100 11 → 0001 0100 0011

Hex : 0001 → 1 , 0100 → 4 , 0011 → 3

∴ $(123)_8 = (143)_{16}$

(3) $(640)_8 = ?$ 6 → 110 , 4 → 100 , 0 → 000

Octal → Binary : $(640)_8$ = 110 100 000

Group in 4 bits : 0001 1010 0000

Hex : 0001 → 1 , 1010 → A , 0000 → 0

∴ $(640)_8 = (1A0)_{16}$

B) Floating Point Octal Numbers

(1) $(7.3)_8 = ?$ 7 → 111 , 3 → 011

Octal → Binary : $(7.3)_8$ = 111 . 011

Group 4 bits : 0000 0111 . 0110

Hex : 0000 → 0 , 0111 → 7 , 0110 → 6

∴ $(7.3)_8 = (0.76)_{16}$

(2) $(12.5)_8 = ?$ 1 → 001 , 2 → 010 , 5 → 101

Octal → Binary : $(12.5)_8$ = 001 010 . 101

Group 4 bits : 0001 0010 . 1010

Hex : 0001 → 1 , 0010 → 2 , 1010 → A

∴ $(12.5)_8 = (12.A)_{16}$

(3) $(40.6)_8 = ?$ 4 → 100 , 0 → 000 , 6 → 110

Octal → Binary : $(40.6)_8$ = 100 000 . 110

Group 4 bits : 0001 0000 . 1100

Hex : 0001 → 1 , 0000 → 0 , 1100 → C

∴ $(40.6)_8 = (10.C)_{16}$

Key Point :
- 1 Octal digit = 3 Binary bits
- 1 Hex digit = 4 Binary bits

Common Mistakes
- × Wrong binary grouping (must from right).
- × Forgetting leading zeros.
- × Mixing integer & fractional parts.

Summary

- Octal → Decimal : use powers of 8.
- Octal → Hex : Octal → Binary → Hex.

Remember ☆

Practice more examples to increase speed & accuracy.

Interview Point ☆

Understanding base conversions is fundamental for digital system design!

Digital Electronics

1. HexaDecimal to Binary Conversion

- Convert each hex digit to its 4-bit binary equivalent.
- Replace the hex digit by 4-bit binary.
- For fractional part, convert each hex digit after point to 4-bit binary.

A) Normal HexaDecimal Numbers

① $(2F)_{16}$ = ? 2 → 0010 , F → 1111

So, $(2F)_{16}$ = $(0010\ 1111)_2$

② $(A3C)_{16}$ = ? A → 1010 , 3 → 0011 , C → 1100

So, $(A3C)_{16}$ = $(1010\ 0011\ 1100)_2$

③ $(7B8E)_{16}$ = ? 7 → 0111 , B → 1011 , 8 → 1000 , E → 1110

So, $(7B8E)_{16}$ = $(0111\ 1011\ 1000\ 1110)_2$

B) Floating Point HexaDecimal Numbers

① $(3.A)_{16}$ = ? 3 → 0011 , A → 1010

So, $(3.A)_{16}$ = $(0011\ .\ 1010)_2$

② $(F.4)_{16}$ = ? F → 1111 , 4 → 0100

So, $(F.4)_{16}$ = $(1111\ .\ 0100)_2$

③ $(1C.B)_{16}$ = ? 1 → 0001 , C → 1100 , B → 1011

So, $(1C.B)_{16}$ = $(0001\ 1100\ .\ 1011)_2$

Hex to Binary Mapping (4 bits per digit)

Hex:	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Bin:	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

Tip:

* Integer part → convert each hex digit to 4-bit binary.
* Fractional part → convert each hex digit after point to 4-bit binary.
* No calculation required, just mapping.

2. HexaDecimal to Decimal Conversion

- Multiply each hex digit by 16^{position} (left to right for integer part, negative powers for fraction part).
- Add all the products to get decimal value.

A) Normal HexaDecimal Numbers

① $(2F)_{16}$ = ? (2 → 16^1, F → 16^0)

$= 2 \times 16^1 + 15 \times 16^0$

$= 32 + 15 = 47_{10}$

② $(A3C)_{16}$ = ? (A → 16^2, 3 → 16^1, C → 16^0)

$= 10 \times 16^2 + 3 \times 16^1 + 12 \times 16^0$

$= 2560 + 48 + 12 = 2620_{10}$

③ $(7B8E)_{16}$ = ? (7 → 16^3, B → 16^2, 8 → 16^1, E → 16^0)

$= 7 \times 16^3 + 11 \times 16^2 + 8 \times 16^1 + 14 \times 16^0$

$= 28672 + 2816 + 128 + 14 = 31630_{10}$

B) Floating Point HexaDecimal Numbers

① $(3.A)_{16}$ = ? (3 → 16^0, . , A → 16^{-1})

$= 3 \times 16^0 + 10 \times 16^{-1}$

$= 3 + \frac{10}{16} = 3 + 0.625 = 3.625_{10}$

② $(F.4)_{16}$ = ? (F → 16^0, . , 4 → 16^{-1})

$= 15 \times 16^0 + 4 \times 16^{-1}$

$= 15 + \frac{4}{16} = 15 + 0.25 = 15.25_{10}$

③ $(1C.B)_{16}$ = ? (1 → 16^1, C → 16^0, . , B → 16^{-1})

$= 1 \times 16^1 + 12 \times 16^0 + 11 \times 16^{-1}$

$= 16 + 12 + \frac{11}{16} = 28 + 0.6875 = 28.6875_{10}$

Rules to Remember

* For integer part → use positive powers of 16 (16^0, 16^1,...).
* For fractional part → use negative powers (16^{-1}, 16^{-2},...).
* A = 10, B = 11, C = 12, D = 13, E = 14, F = 15.

Quick Reference

Hexadecimal → Base 16 → Digits : 0 to 9 and A to F

Binary → Base 2 → Digits : 0 and 1

Each Hex digit = 4 Binary bits ($16 = 2^4$)

Key Point

Hex to Binary → mapping (easy, no calculation)

Hex to Decimal → weighted sum (powers of 16)

Interview Point ☆

Why use Hexadecimal?

→ More compact than binary.

→ Easy to convert to binary (4 bits).

→ Widely used in Computer Systems, Memory addresses, Machine Code.

Digital Electronics

1. HexaDecimal to Octal Conversion

- Convert hexadecimal number to binary (4 bits per digit).
- Group binary in 3 bits from right to left.
- Replace each 3-bit group by its octal equivalent.

A) Normal HexaDecimal Numbers

① $(2F)_{16}$ = ? 2 → 0010 , F → 1111

Hex → Binary : 0010 1111

Group in 3 bits : 010 111 → 2 7

∴ $(2F)_{16}$ = $(27)_8$

② $(A3C)_{16}$ = ? A → 1010 , 3 → 0011 , C → 1100

Hex → Binary : 1010 0011 1100

Group in 3 bits : 001 010 011 100 → 1 2 3 4

∴ $(A3C)_{16}$ = $(1234)_8$

③ $(7B8E)_{16}$ = ? 7 → 0111 , B → 1011 , 8 → 1000 , E → 1110

Hex → Binary : 0111 1011 1000 1110

Group in 3 bits : 011 110 111 000 111 010 → 3 6 7 0 7 2

∴ $(7B8E)_{16}$ = $(367072)_8$

B) Floating Point HexaDecimal Numbers

① $(3.A)_{16}$ = ? 3 → 0011 , A → 1010

Hex → Binary : 0011 . 1010

Group (int) : 001 1 → 1 3

Group (frac) : 101 000 → 5 0

∴ $(3.A)_{16}$ = $(13.50)_8$

② $(F.4)_{16}$ = ? F → 1111 , 4 → 0100

Hex → Binary : 1111 . 0100

Group (int) : 001 111 → 1 7

Group (frac) : 010 000 → 2 0

∴ $(F.B)_{16}$ = $(17.20)_8$

③ $(1C.B)_{16}$ = ? 1 → 0001 , C → 1100 , B → 1011

Hex → Binary : 0001 1100 . 1011

Group (int) : 000 111 00 → 0 7 4

Group (frac) : 101 100 → 5 4

∴ $(1C.B)_{16}$ = $(074.54)_8$ (leading 0 optional)

Quick Reference

- 1 Hex digit = 4 Binary bits
- 3 Binary bits = 1 Octal digit
- Add leading 0's (left) if required for grouping.
- For fractional part, add trailing 0's (right).

2. Numerical Complements

The complement of a number (N) in base R is the number which, when added to N, gives R^n, where n is the number of digits used.

There are two important complements used in digital systems → R's complement and (R-1)'s complement.

A) R's Complement

- To obtain R's complement → Subtract each digit from (R-1).
- R's complement of N = $R^n - N$

In Binary R = 2

Example

Decimal N	Binary (4 bits)	1's Complement	2's Complement
5	0101	1010	1011
9	1001	0110	0111
13	1101	0010	0011
3	0011	1100	1101

B) (R-1)'s Complement

- To obtain (R-1)'s complement → Subtract each digit from (R-2).
- (R-1)'s complement of N = $R^n - N - 1$

In Binary R = 2

Example

Decimal N	Binary (4 bits)	(R-1)'s Complement
5	0101	1010
9	1001	0110
13	1101	0010
3	0011	1100

Relationship in Binary (R = 2)

2's complement = 1's complement + 1 = $2^n - N$

1's complement = $2^n - 1 - N$

(R-1)'s complement = $2^n - 2 - N$

Notes / Important Points

- For n digits in base R → range of numbers = 0 to $R^n - 1$
- Complements are useful for subtraction.
- In binary, subtraction is performed by adding 2's complement.
- (R-1)'s complement is used in end-around carry addition.

Summary / Key Points

- Hex to Octal → via Binary (group 3 bits).
- Fractional part → group similarly after point.
- R's complement → digit wise (R-1) subtraction.
- (R-1)'s complement → digit wise (R-2) subtraction.
- Forgetting +1 ≠ 2's complement + 1.
- Complements simplify subtraction in computers.

Common Mistakes

- ✗ Wrong grouping direction.
- ✗ Missing leading / trailing 0's.
- ✗ Not converting fractional part.
- ✗ Confusing 1's & 2's complement.
- ✗ Forgetting +1 in 2's complement.

Interview Point ☆

- → Complements help in performing subtraction using only addition circuits.
- → 2's complement representation is most widely used in modern digital systems.
- → Essential for ALU design, CPU operations and signal processing.

Digital Electronics

1. Numerical Complements – 1's and 2's

- Base $R = 2$ (Binary system)
- 1's complement → invert all bits ($0 \leftrightarrow 1$).
- 2's complement → 1's complement + 1.
- n bit number → Range : 0 to $(2^n - 1)$
- 2's complement of N = $2^n - N$

In Binary $R = 2$

A) 1's Complement – Examples

Decimal N	Binary (4 bits)	1's Complement
5	0101	1010
9	1001	0110
13	1101	0010

B) 2's Complement – Examples

Decimal N	Binary (4 bits)	1's Complement	+1	2's Complement
5	0101	1010	+1	1011
9	1001	0110	+1	0111
13	1101	0010	+1	0011

Quick Rule (Binary)
- To get 1's comp → invert all bits.
- To get 2's comp → invert all bits and add 1.
- Two numbers are complements if their sum = 2^n.

2. Numerical Complements – 9's and 10's

- Base $R = 10$ (Decimal system)
- 9's complement → subtract each digit from 9.
- 10's complement → 9's complement + 1.
- n digit number → Range : 0 to $(10^n - 1)$
- 10's complement of N = $10^n - N$

In Decimal $R = 10$

A) 9's Complement – Examples

Decimal N	Decimal (4 digits)	9's Complement
5237	5237	4762
9086	9086	0913
1359	1359	8640

B) 10's Complement – Examples

Decimal N	9's Complement	+1	10's Complement
5237	4762	+1	4763
9086	0913	+1	0914
1359	8640	+1	8641

Quick Rule (Decimal)
- 9's comp → subtract each digit from 9.
- 10's comp → 9's comp + 1.
- Two numbers are complements if their sum = 10^n.

3. Numerical Complements – 15's and 16's

- Base $R = 16$ (Hexadecimal system)
- 15's complement → subtract each hex digit from F.
- 16's complement → 15's complement + 1.
- n hex digits → Range : 0 to $(16^n - 1)$
- 16's complement of N = $16^n - N$

In Hexa $R = 16$

A) 15's Complement – Examples

Hex N	Hex (4 digits)	15's Complement
2F3A	2F3A	D0C5
A1B7	A1B7	5E48
7C08	7C08	83F7

B) 16's Complement – Examples

Hex N	15's Complement	+1	16's Complement
2F3A	D0C5	+1	D0C6
A1B7	5E48	+1	5E49
7C08	83F7	+1	83F8

Quick Rule (Hexadecimal)
- 15's comp → subtract each digit from F.
- 16's comp → 15's comp + 1.
- Two numbers are complements if their sum = 16^n.

General Properties of Complements

1) R's complement of N = $R^n - N$
2) $(R-1)$'s complement of N = $R^n - N - 1$
3) $(R-1)$'s complement of N = (R's complement) – 1
4) R's complement + N = R^n
5) $(R-1)$'s complement + N = $R^n - 1$
6) Complement of zero = Maximum number $(R^n - 1)$
7) Complement of maximum number = Zero

Note : Here, R is the Base and n is the number of digits used.

Why Complements are Useful ?

- Used in subtraction → A – B = A + (complement of B).
- Simplifies hardware in digital computers.
- Helps in error detection and end-around carry in communication.

Example : Subtraction using 2's Complement

Find : 9 – 5 (Using 4 bit binary)

9 = 1001
5 = 0101

} 2's complement of 5 = 1011

```
  1001
+ 1011
------
1 0100   → Discard carry → 0100 = 4
```

So, 9 – 5 = 4

Summary / Key Points

- 1's comp → invert all bits (Binary).
- 2's comp → invert bits and add 1.
- 9's comp → subtract each decimal digit from 9.
- 10's comp → 9's comp + 1.
- 15's comp → subtract each hex digit from F.
- 15's comp → 15's comp + 1.

Important Tip

For any base R,

R's comp + N = R^n

$(R-1)$'s comp + N = $R^n - 1$

(where n = number of digits)

Common Mistakes

- ✗ Forgetting to add +1 for 2's, 10's, 16's complement.
- ✗ Using wrong base while finding complements.
- ✗ Ignoring number of digits (n) while calculating.
- ✗ Dropping carry blindly in subtraction.

Digital Electronics

1. Unsigned Representation & Signed Representation

→ Unsigned Representation : All bits represent magnitude only. Range : 0 to $(2^n - 1)$.

n bits	Range	Example (n = 4)
3 bits	0 to 7	000 to 111
4 bits	0 to 15	0000 to 1111
5 bits	0 to 31	00000 to 11111

→ Signed Representation (Sign-Magnitude) : MSB indicates sign. 0 → Positive , 1 → Negative.
Range : $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$ and one 0.

Sign (MSB)	Meaning	Example (4 bits)	Decimal
0	Positive	0 101	+5
1	Negative	1 101	−5
0	Zero	0 000	0
1	Negative Zero	1 000 (not used)	0

* Most widely used methods for signed numbers are 1's Complement and 2's Complement representations.

Range Comparison (n bits)

Representation	Range
Unsigned	0 to $(2^n - 1)$
Sign-Magnitude	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$ and two zeros
1's Complement	$-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$
2's Complement	$-(2^{n-1})$ to $+(2^{n-1} - 1)$

3. Binary Addition using 1's and 2's Complement

A) Using 1's Complement

① (+5) + (−3) (4 bits)

+5 → 0101
−3 → 1100 (1's comp)

```
  0101
+ 1100
------
1 0001  (sum)
```

End around carry

0001 = +1 = 1

② (+6) + (−9) (4 bits)

+6 → 0110
−9 → 0110 (1's comp)

```
  0110
+ 0111
------
  1100  (sum)
```

No around carry

1100 → Negative
Magnitude → 1's comp of 1100 = 0011 = 3
∴ Result = −3

B) Using 2's Complement

① (+5) + (−3) (4 bits)

+5 → 0101
−3 → 1101 (2's comp)

```
  0101
+ 1101
------
1 0010  (sum)
```

Discard carry

0010 = 2

② (+6) + (−9) (4 bits)

+6 → 0110
−9 → 0111 (2's comp)

```
  0110
+ 0111
------
  1101  (sum)
```

No carry

1101 → Negative
Magnitude → 2's comp of 1101 = 0011 = 3
∴ Result = −3

2. Binary Subtraction using 1's and 2's Complement

• Rule : Subtract B from A → A − B = A + (Complement of B) (of same number of bits)

A) Using 1's Complement

① 13 − 5 (using 4 bits)

A = 13 → 1101
B = 5 → 0101
1's comp of B → 1010

```
  1101  (A)
+ 1010  (1's comp of B)
------
1 0111  (sum)
```

End around carry = 1

```
  0111
+    1
------
  1000  = 8  (Answer)
```

② 6 − 9 (using 4 bits)

A = 6 → 0110
B = 9 → 1001
1's comp of B → 0110

```
  0110
+ 0110
------
  1100  (sum)
```

No end around carry

1100 → Negative
Magnitude → 1's comp of 1100 = 0011 = 3
∴ Result = −3

B) Using 2's Complement

① 13 − 5 (using 4 bits)

A = 13 → 1101
B = 5 → 0101
2's comp of B → 1011

```
  1101
+ 1011
------
1 0101  (sum)
```

End around carry = 1

```
  0101
+    1
------
  1000  = 8  (Answer)
```

② 6 − 9 (using 4 bits)

A = 6 → 0110
B = 9 → 1001
2's comp of B → 0111

```
  0110
+ 0111
------
  1101  (sum)
```

No end around carry

1101 → Negative
Magnitude → 2's comp of 1101 = 0011 = 3
∴ Result = −3

Note :
* In 1's complement method → add end around carry if present.
* In 2's complement method → discard end around carry.
* If carry does not exist → result is negative.
* To get magnitude → take 1's or 2's complement of result.

Comparison in Addition

Method	Positive Result	Negative Result
1's Complement	End around carry exists	No end around carry
2's Complement	Discard final carry (if any)	No carry (MSB = 1)

Important Points
* Addition and subtraction only 2's complement use same hardware (adder).
* For negative result → MSB (Sign bit) = 1.

Summary / Key Points
* Unsigned → magnitude only.
* Signed → need sign bit.
* 1's comp → invert bits.
* 2's comp → invert bits and add 1.
* Subtraction → Add complement.
* Addition → follow carry rules.

Common Mistakes
× Forgetting end around carry in 1's comp.
× Not adding +1 while taking 2's comp.
× Keeping final carry in 2's comp addition.
× Interpreting MSB = 1 as large positive.
× Confusing 1's and 2's complement rules.

Interview Point ☆
Why 2's complement is preferred ?
→ Unique zero representation.
→ Easier hardware implementation.
→ Same adder for addition & subtraction.
→ More negative numbers than positives.
→ Widely used in digital computers and ICs.

Digital Electronics

1. Subtraction using 9's and 10's Complement

★ Subtraction can be performed using complements.
★ If A − B is to be performed, it can be done by adding the complement of B to A.
★ For decimal numbers → 9's or 10's complement.
★ For binary numbers → 1's or 2's complement.

9's Complement (decimal)

★ Obtain by subtracting each digit from 9.
★ End-around carry is added to the result.

Steps :
1. Take 9's complement of subtrahend (B).
2. Add it to minuend (A).
3. If carry is generated, add it to the LSB.
4. Discard the carry. The remaining digits are the result.

End-around carry is important in 9's & 10's complement subtraction.

10's Complement (decimal)

★ Obtain by adding 1 to the 9's complement.

Steps :
1. Take 10's complement of B.
2. Add it to A.
3. If carry is generated, discard it. Result = remaining digits.
4. If no carry → result is negative. Take 10's complement of the sum and prefix '−' sign.

Examples (decimal)

(i) Using 9's Complement : 842 − 357

B = 357
9's comp. of B

$$\begin{array}{r} {\scriptstyle 1}\ 842 \\ +\ 642 \\ \hline 1484 \end{array}$$ ← end-around carry

Add carry to LSB → 484 + 1 = 485

∴ 842 − 357 = 485

(ii) Using 10's Complement : 1000 − 289

B = 289
10's comp. of B = 711

$$\begin{array}{r} 1000 \\ +\ 711 \\ \hline 1711 \end{array}$$ ← carry

Discard carry → 711

∴ 1000 − 289 = 711

Note :
★ In 9's & 10's complement subtraction, if carry is generated → result is positive.
★ If no carry → result is negative. (Take complement of sum and put − sign)



Negative Result Example (Decimal)

(i) 285 − 683

B = 683
10's comp. of B = 317

$$\begin{array}{r} 285 \\ +\ 317 \\ \hline 602 \end{array}$$ No carry → Negative result

10's comp. of 602 = 398

∴ 285 − 683 = −398

Negative Result Example (Octal)

(ii) $145_8 - 276_8$

B = 276_8
8's comp. of B = 502_8

$$\begin{array}{r} 145_8 \\ +\ 502_8 \\ \hline 647_8 \end{array}$$ No carry → Negative result

8's comp. of 647_8 = 121_8

∴ $145_8 - 276_8 = -121_8$

2. Subtraction using 7's and 8's Complement

★ Similar to 9's & 10's complement but base is 8.
★ Used in octal number system.

7's Complement (octal)

★ Obtain by subtracting each digit from 7.
★ End-around carry is added to the result.

Steps :
1. Take 7's complement of subtrahend (B).
2. Add it to minuend (A).
3. If carry is generated, add it to LSB.
4. Discard carry. Remaining digits are result.

8's Complement (octal)

★ Obtain by adding 1 to the 7's complement.

Steps :
1. Take 8's complement of B.
2. Add it to A.
3. If carry is generated, discard it. Result = remaining digits.
4. If no carry → result is negative. Take 8's complement of sum and prefix '−' sign.

Examples (octal)

(i) Using 7's Complement : $526_8 - 247_8$

B = 247_8
7's comp. of B = 530_8

$$\begin{array}{r} {\scriptstyle 1}\ 526_8 \\ +\ 530_8 \\ \hline 1056_8 \end{array}$$ ← end-around carry

Add carry to LSB → $056_8 + 1 = 057_8$

∴ $526_8 - 247_8 = 057_8$

(ii) Using 8's Complement : $1000_8 - 345_8$

B = 345_8
8's comp. of B = 433_8

$$\begin{array}{r} 1000_8 \\ +\ 433_8 \\ \hline 1433_8 \end{array}$$ ← carry

Discard carry → 433_8

∴ $1000_8 - 345_8 = 433_8$

Comparison Summary

Complement	Base	How to obtain	Carry	If no carry
9's Comp.	10 (Decimal)	Subtract each digit from 9	Add end-around carry	Take 9's comp. of sum and put − sign
10's Comp.	10 (Decimal)	Add 1 to 9's complement	Discard carry	Take 10's comp. of sum and put − sign
7's Comp.	8 (Octal)	Subtract each digit from 7	Add end-around carry	Take 7's comp. of sum and put − sign
8's Comp.	8 (Octal)	Add 1 to 7's complement	Discard carry	Take 8's comp. of sum and put − sign

Key Points

★ Complement method converts subtraction into addition.
★ Carry indicates positive result.
★ No carry indicates negative result.
★ End-around carry is used in 9's & 7's complement.
★ Carry is discarded in 10's & 8's complement.

Summary / Key Points

→ Subtraction can be performed using complements.
→ For decimal numbers → use 9's and 10's complement.
→ For octal numbers → use 7's and 8's complement.
→ Carry rules :
  • 9's & 7's → Add end-around carry.
  • 10's & 8's → Discard the carry.
→ If no carry → result is negative (take complement of sum).
→ This method simplifies hardware implementation.

Interview Point

Subtraction using complements is useful in digital systems because it eliminates the need for a separate subtractor. Addition circuits (adders) can be used for subtraction, reducing hardware complexity and cost.

Number Representation in Digital Electronics

1. Fixed Point Representation

→ Introduction :

- In fixed point representation, the position of the binary point (radix point) is fixed and known.
- Both integer and fractional parts are represented with a fixed number of bits.
- Used in many digital systems like DSP, microcontrollers and embedded systems.

→ Format :

b_{n-1}	b_{n-2}	...	b_0	b_{-1}	b_{-2}	...	b_{-m}

← Integer part (n bits) → | ← Fractional part (m bits) →

→ Value :

$$V = \sum_{i=-m}^{n-1} b_i \, 2^i \qquad \text{where, } b_i \in \{0,1\}$$

→ Range (Unsigned) : $0 \le V \le 2^n - 2^{-m}$

→ Resolution (LSB weight) : $\Delta = 2^{-m}$

→ Example 1 : unsigned fixed point

Given : 8-bit number with 4 integer bits and 4 fractional bits (Q4.4 format).

Number : 1011.0110_2

Calculation :

$= 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 0\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} + 0\times2^{-4}$

$= 8 + 0 + 2 + 1 + 0 + 0.25 + 0.125 + 0$

$= \boxed{11.375}$

→ Example 2 : signed fixed point (two's complement)

Q3.4 format (1 sign bit, 3 integer bits, 4 fractional bits)

Number : 1101.1000_2

MSB = 1 → negative number.

First find magnitude : Invert + 1 → 0010.1000

$= 2 + 0.5 = 2.5$

∴ Value = $\boxed{-2.5}$

Key Points (Fixed Point)

★ Binary point position is fixed.
★ Fractional part has constant number of bits.
★ Simple hardware and low complexity.
★ Suitable when dynamic range is limited.
★ Widely used in DSP, controllers and real-time systems.

2. Floating Point Representation

→ Introduction :

- In floating point representation, the binary point is not fixed.
- The number is represented in normalized form using a significand (mantissa) and an exponent.
- It can represent a very large range of numbers (both very small and very large).
- Widely used in scientific computing, processors (IEEE 754), and high-precision applications.

→ General Form :

$$N = (-1)^S \times M \times 2^E$$

S – sign bit (0 : +ve, 1 : -ve)
M – mantissa / significand
E – exponent (can be +ve or -ve)

→ Normalized Form :

$M = 1.f$ (for normalized numbers)

i.e., $1 \le M < 2$
Binary point is just after the MSB.

→ IEEE 754 Single Precision (32-bit) :

1 bit	8 bits	23 bits
S	Exponent (E)	Mantissa / Fraction (F)

→ Value : $N = (-1)^S \times (1 + F) \times 2^{(E-127)}$

Bias = 127 (for normalized numbers)

→ Example :

Represent 25.5 in IEEE 754 single precision.

Step 1 : Convert to binary
$25.5 = 11001.1_2 = 1.10011_2 \times 2^4$

Step 2 : Sign bit — S = 0 (positive)

Step 3 : Exponent — $E = 4 + 127 = 131 = 10000011_2$

Step 4 : Mantissa — F = 10011 followed by zeros
F = 10011000000000000000000 (23 bits)

Representation :

0	10000011	10011000000000000000000

Comparison : Fixed Point vs Floating Point

Parameter	Fixed Point	Floating Point
Binary point	Fixed position	Not fixed (variable)
Representation	Integer + fractional bits	Sign, exponent, mantissa
Dynamic range	Limited	Very large
Precision	Constant (fixed)	Variable (higher)
Hardware	Simple, low cost	Complex
Applications	DSP, embedded systems	Scientific computing, processors

Summary / Key Points

★ Fixed point → fixed binary point, limited range, simple hardware.
★ Floating point → variable binary point, large dynamic range.
★ Fixed point value : $V = \sum_{i=-m}^{n-1} b_i \, 2^i$, Resolution = 2^{-m}.
★ Floating point value : $N = (-1)^S \times M \times 2^E$ (IEEE 754 : $N = (-1)^S \times (1+F) \times 2^{(E-127)}$)
★ Floating point is preferred when wide dynamic range and high precision are required.
★ Fixed point is efficient for real-time and hardware-limited systems.

Interview / Viva Point

★ Why floating point is needed ?
→ To represent very small and very large numbers with high precision.
★ What is the advantage of normalized form ?
→ Ensures a unique representation and maximum precision.

Boolean Algebra

1. Introduction

Boolean algebra is a mathematical system of logic in which variables can have only **two values : 0 or 1**.

It is the foundation of digital logic design and switching circuit analysis.

2. Basic Operations

Operation	Symbol	Name	Description
+		OR	$A + B = 1$ if either $A=1$ or $B=1$ (or both)
$\cdot$		AND	$A \cdot B = 1$ only if $A=1$ and $B=1$
$\overline{A}$		NOT (Complement)	$\overline{A} = 1$ if $A = 0$ $\overline{A} = 0$ if $A = 1$

3. Basic Theorems (Laws)

1) Identity Laws : $A + 0 = A$, $A \cdot 1 = A$
2) Null Laws : $A + 1 = 1$, $A \cdot 0 = 0$
3) Idempotent Laws : $A + A = A$, $A \cdot A = A$
4) Complement Laws : $A + \overline{A} = 1$, $A \cdot \overline{A} = 0$
5) Involution Law : $\overline{\overline{A}} = A$
6) Commutative Laws : $A + B = B + A$, $A \cdot B = B \cdot A$
7) Associative Laws : $(A + B) + C = A + (B + C)$
   $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
8) Distributive Laws : $A \cdot (B + C) = A \cdot B + A \cdot C$
   $A + (B \cdot C) = (A + B) \cdot (A + C)$
9) DeMorgan's Laws : $\overline{A + B} = \overline{A} \cdot \overline{B}$, $\overline{A \cdot B} = \overline{A} + \overline{B}$

8. Useful Identities (Quick Reference)

- $A + \overline{A} = 1$
- $A \cdot \overline{A} = 0$
- $A + A = A$
- $A \cdot A = A$
- $A + 0 = A$, $A + 1 = 1$
- $A \cdot 0 = 0$, $A \cdot 1 = A$
- $\overline{\overline{A}} = A$
- $A + A \cdot B = A$
- $A\,(A + B) = A$
- $(A + B)\,(A + \overline{B}) = A$
- $A + B = B + A$, $A \cdot B = B \cdot A$

DeMorgan's Laws

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$
$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

Duality :

Every theorem has a dual obtained by interchanging $+ \leftrightarrow \cdot$ and $0 \leftrightarrow 1$.

9. Boolean Expression Forms

SOP (Sum of Products) : $F = m_1 + m_2 + m_5$
(OR of minterms)

POS (Product of Sums) : $F = (M_0)\,(M_2)\,(M_5)$
(AND of maxterms)

4. Truth Tables

A	$\overline{A}$
0	1
1	0

A	B	A + B	A · B
0	0	0	0
0	1	1	0
1	0	1	0
1	1	1	1

A	B	A ⊕ B (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

5. Derived Theorems

1) Absorption Laws : $A + A \cdot B = A$, $A\,(A + B) = A$
2) Consensus Theorem : $A \cdot B + \overline{A} \cdot C + B \cdot C = A \cdot B + \overline{A} \cdot C$
3) Covering Theorem : $(A + B)\,(A + \overline{B}) = A$
4) Principle of Duality : Interchanging + and $\cdot$, and 0 and 1 gives the dual.

6. Important Properties

- ☑ Each variable can take only two values (0 or 1).
- ☑ Laws and theorems remain valid for any number of variables.
- ☑ Used for simplification of Boolean expressions.
- ☑ Basis for logic minimization (K-maps, Quine-McCluskey).

7. Worked Examples

Example 1 : Simplify using Boolean algebra

$F = A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$

Step 1 : Take A common from first two terms
$= A\,(B + \overline{B}) + \overline{A} \cdot B$

Step 2 : Since $B + \overline{B} = 1$ → $= A \cdot 1 + \overline{A} \cdot B$

Step 3 : $A \cdot 1 = A$ → $F = A + \overline{A} \cdot B$

Step 4 : Use Absorption law $(A + \overline{A} \cdot B = A + B)$
$= A + B$

Answer : $\boxed{F = A + B}$

Example 2 : Simplify using DeMorgan's laws

$F = \overline{(A + B)\,(\bar{A} + C)}$

Step 1 : Apply DeMorgan's law $\overline{X\,Y} = \overline{X} + \overline{Y}$
$= \overline{(A + B)} + \overline{(\bar{A} + C)}$

Step 2 : Again apply DeMorgan
$= \overline{A} \cdot \overline{B} + (A \cdot \bar{C})$

Step 3 : Write in standard form
$= \overline{A} \cdot \overline{B} + A \cdot \overline{C}$

Answer : $\boxed{F = \overline{A} \cdot \overline{B} + A \cdot \overline{C}}$

Key Point

- + represents OR, · represents AND, bar represents NOT.
- Use laws to simplify expressions before logic implementation.
- Simplest form → minimum gates.

Applications

- Digital circuit design & optimization
- Logic minimization (K-maps, Quine-McCluskey)
- Computer arithmetic unit design
- Control logic & sequential circuit design

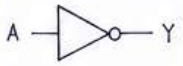
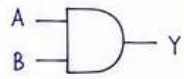
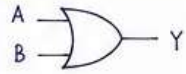
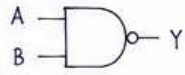
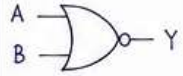
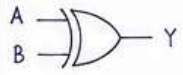
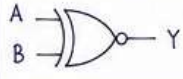
Common Mistakes

- Forgetting brackets in operations.
- Applying theorems without checking conditions.
- Confusing + with · or 0 with 1.

Author : Konda Tharun Vivekananda

Logic Gates & Implementation of Gates using Universal Gates NAND and NOR (Steps)

1. Logic Gates (Basic Gates)

Gate	Symbol	Boolean Exp.	Truth Table
1) NOT (Inverter)		$Y = \overline{A}$	A | Y 0 | 1 1 | 0
2) AND (2-input)		$Y = A \cdot B$	A B | Y 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
3) OR (2-input)		$Y = A + B$	A B | Y 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
4) NAND (2-input)		$Y = \overline{A \cdot B}$	A B | Y 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
5) NOR (2-input)		$Y = \overline{A + B}$	A B | Y 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
6) XOR (2-input)		$Y = A \oplus B$ $= \overline{A}B + A\overline{B}$	A B | Y 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
7) XNOR (2-input)		$Y = \overline{A} \oplus \overline{B}$ $= AB + \overline{A}\,\overline{B}$	A B | Y 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

Key Point

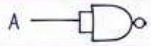
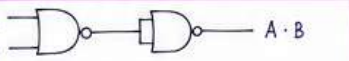
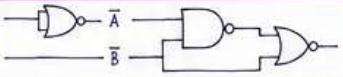
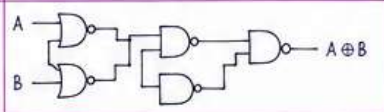
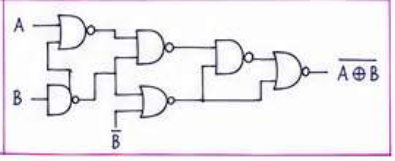
- NAND and NOR are Universal Gates.
- Any logic function can be implemented using only NANDs or only NORs.
- Fewer types of IC required in hardware.

2. Implementation using NAND Gates (Steps)

General Steps :

1) Write the Boolean expression using basic gates.
2) Replace every gate by equivalent NAND network.
3) Use NAND as inverter → $\overline{A}$ = A NAND A
4) Simplify if possible.
5) Draw the final circuit using only NAND gates.

Equivalent Conversions using NAND

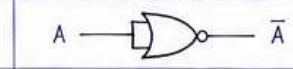
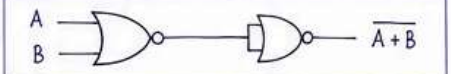
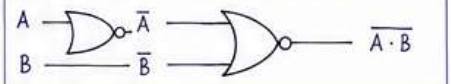
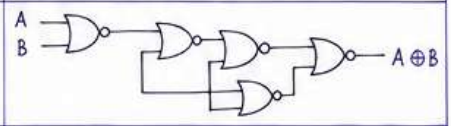
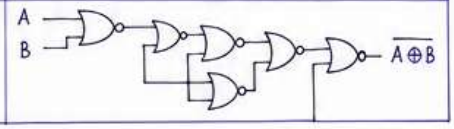
Gate	Expression	NAND Implementation
NOT A	$\overline{A}$	
A AND B	$A \cdot B$	
A OR B	$A + B$	
A XOR B (4 NORs)		
A XNOR B (5 NORs)		

3. Implementation using NOR Gates (Steps)

General Steps :

1) Write the Boolean expression using basic gates.
2) Replace every gate by equivalent NOR network.
3) Use NOR as inverter → $\overline{A}$ = A NOR A
4) Simplify if possible.
5) Draw the final circuit using only NOR gates.

Equivalent Conversions using NOR

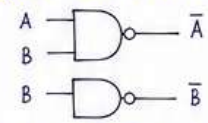
Gate	Expression	NOR Implementation
NOT A	$\overline{A}$	
A OR B	$A + B$	
A AND B	$A \cdot B$	
A XOR B (5 NORs)	$A \oplus B$	
A XNOR B (6 NORs)	$A \oplus B$	

4. Examples

Example 1 : Implement $Y = A + B$ using NAND gates

$$Y = A + B = \overline{\overline{A} \cdot \overline{B}}$$

Step 1 : Invert A and B



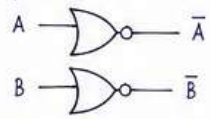
Step 2 : NAND the inverted outputs



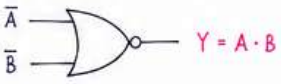
Example 2 : Implement $Y = A \cdot B$ using NOR gates

$$Y = A \cdot B = \overline{\overline{A} + \overline{B}}$$

Step 1 : Invert A and B



Step 2 : NOR the inverted outputs



5. Comparison

Parameter	NAND Implementation	NOR Implementation
Universal Gate	NAND	NOR
Inverter	A NAND A	A NOR A
AND	2 NANDs	2 NORs
OR	3 NANDs	2 NORs
XOR	4 NANDs	5 NORs
XNOR	5 NANDs	6 NORs
Speed	Generally fast	Slightly slower
Usage	Widely used in CMOS	Widely used ECL/others

Summary / Key Points

- ★ There are 7 basic logic gates.
- ★ NAND and NOR are universal gates.
- ★ Any Boolean function can be implemented using only NANDs or only NORs.
- ★ Use gate replacements and DeMorgan's laws for conversion.
- ★ NAND implementation is generally preferred in CMOS circuits.

Interview / Viva Point

- ★ Why NAND is universal ?
  → It can realize NOT, AND and OR functions using only NAND circuits.
- ★ Minimum number of gates to implement XOR using NAND ?
  → 4 NAND gates.

Author : Konda Tharun Vivekananda

Boolean Minimization

1. Boolean Minimization : Introduction

- Any Boolean expression can be written in many forms.
- Minimization means expressing a Boolean function in the simplest form with minimum number of literals (variables) and minimum number of gates.
- Minimization reduces cost, area, power and delay.

Key Point

Minimization does not change the function, only the form of expression.

Types of Minimization Techniques

S.No.	Technique	Description	Suitable for
1.	Boolean Properties	Using laws and theorems of Boolean algebra.	Small to medium expressions
2.	Karnaugh Map (K-Map)	Graphical method using Gray code cells and grouping.	2 to 6 variables
3.	Quine-McCluskey Method	Tabular (algorithmic) method.	Large number of variables
4.	Don't Care Conditions	Using don't care terms (X) to get further simplification.	When DC terms are available

Basic Rules for Minimization

1) Remove redundant terms.
2) Remove common literals.
3) Apply Boolean properties repeatedly.
4) Use complements to eliminate extra gates.
5) Verify by truth table (expression must be equivalent).

Advantages

- ★ Reduces number of gates and inputs.
- ★ Low cost and small circuit area.
- ★ Less power consumption.
- ★ Less propagation delay → faster circuits.
- ★ Easy realization and better reliability.

Summary / Key Points

- ★ Boolean minimization gives simplest equivalent expression.
- ★ Techniques : Boolean properties, K-map, Quine-McCluskey and Don't care.
- ★ Boolean properties are very useful for small/medium expressions.
- ★ Always verify the minimized expression using truth table.
- ★ Goal is minimum literals → minimum gates → efficient circuit.

2. Boolean Minimization using Boolean Properties

Important Boolean Properties (Laws)

1) Commutative : $A + B = B + A$, $A \cdot B = B \cdot A$
2) Associative : $(A + B) + C = A + (B + C)$
   $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
3) Distributive : $A \cdot (B + C) = A \cdot B + A \cdot C$
   $A + (B \cdot C) = (A + B) \cdot (A + C)$
4) Identity : $A + 0 = A$, $A \cdot 1 = A$
5) Null : $A + 1 = 1$, $A \cdot 0 = 0$
6) Idempotent : $A + A = A$, $A \cdot A = A$
7) Complement : $A + \overline{A} = 1$, $A \cdot \overline{A} = 0$
8) Involution : $\overline{\overline{A}} = A$
9) Absorption : $A + A \cdot B = A$, $A \cdot (A + B) = A$
10) DeMorgan's : $\overline{A + B} = \overline{A} \cdot \overline{B}$, $\overline{A \cdot B} = \overline{A} + \overline{B}$

Example 1 : Simplify using Boolean properties

$F = A \cdot B + A \cdot \overline{B} + \overline{A} \cdot B$

Step 1 : Take A common from first two terms
$= A\,(B + \overline{B}) + \overline{A} \cdot B$

Step 2 : Use Identity law $(B + \overline{B} = 1)$
$= A \cdot 1 + \overline{A} \cdot B$

Step 3 : Use Identity law $(A \cdot 1 = A)$
$\boxed{F = A + \overline{A} \cdot B}$

Step 4 : Apply Absorption law $(A + \overline{A} \cdot B = A + B)$
$\boxed{F = A + B}$

Truth Table (Check)

A	B	F (given)	F (A+B)
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Example 2 : Simplify using Boolean properties

$F = (A + B)(A + \overline{B}) + A \cdot \overline{B}$

Step 1 : Expand first term using distributive law
$= A \cdot A + A \cdot \overline{B} + B \cdot A + B \cdot \overline{B} + A \cdot \overline{B}$

Step 2 : Simplify using Idempotent and Null laws
$= A + A \cdot \overline{B} + A \cdot B + 0 + A \cdot \overline{B}$ $[A \cdot A = A,\ B \cdot \overline{B} = 0]$

Step 3 : Remove duplicate term and take A common
$= A + A \cdot \overline{B} + A \cdot B$
$= A\,(1 + \overline{B} + B)$

Step 4 : $(\overline{B} + B = 1!)$ → $1 + 1 = 1$
$\boxed{F = A \cdot 1 = A}$

Truth Table (Check)

A	B	F (given)	F (A)
0	0	0	0
0	1	0	0
1	0	1	1
1	1	1	1

Example 3 : Simplify using DeMorgan's law

$F = \overline{\overline{A} + \overline{B}} + \overline{\overline{A} + \overline{B}}$

Step 1 : Apply DeMorgan's law to both terms
$= \overline{A} \cdot B + \overline{A} \cdot \overline{B}$ $[\overline{X + Y} = \overline{X} \cdot \overline{Y}]$

Step 2 : Take $\overline{A}$ common
$= \overline{A}\,(B + \overline{B})$

Step 3 : $(B + \overline{B} = 1)$
$\boxed{F = \overline{A}}$

Truth Table (Check)

A	B	F (given)	F ($\overline{A}$)
0	0	1	1
0	1	1	1
1	0	0	0
1	1	0	0

When to use Boolean Properties ?

- → Expressions with small / medium number of literals.
- → When K-map is not convenient.
- → For initial simplification before using K-map.

Advantages

- ★ No diagrams or tables required.
- ★ Fast and easy for small expressions.
- ★ Useful for algebraic manipulation.

Limitations

- ★ Tedious for large expressions.
- ★ Difficult to visualize all possibilities.
- ★ K-map / Q-M method better for many variables.

Author : Konda Tharun Vivekananda

Boolean Minimization using K-Maps

1. Boolean Minimization using K-Maps – Steps

Karnaugh map (K-map) is a graphical method used to minimize Boolean expressions by grouping adjacent 1's (For SOP) or 0's (For POS).

Steps :

1) Determine the no. of variables and choose the appropriate K-map (2, 3, 4, 8 or 16 cells).
2) Write the minterms (1's) in the cells as per Gray code ordering.
3) Adjacent cells differ in only one variable. (Wrap around property holds for edges).
4) Group the 1's in powers of 2 → 1, 2, 4, 8, 16 ... (Largest possible groups first).
5) Groups may overlap, but include all 1's at least once.
6) For each group, write the term containing only the variables that remain constant in that group.
7) Sum of all terms → Minimal SOP expression.

Gray Code Sequences Used in K-Maps

Bits	Gray Code Order
2 bits	00 , 01 , 11 , 10
3 bits	000, 001, 011, 010, 110, 111, 101, 100
4 bits	0000, 0001, 0011, 0010, 0110, 0111, 1101, 1100, 1000, 1001, 1011, 1010, 1110, 1111, 1101, 1000

Key Points

- ★ Adjacent cells → differ in only one variable.
- ★ Edges are adjacent (wrap around).
- ★ Use largest possible groupings.
- ★ Don't leave any 1 uncovered.
- ★ Groups must be rectangular (in 2D K-map).

Summary / Key Points

- K-maps provide a visual and fast method for minimization.
- Number of cells = 2^n for n variables.
- Group 1's in largest possible powers of 2.
- Overlapping groups are allowed.
- K-map minimizes both SOP (1's) and POS (0's).
- Very useful for up to 4 variables; beyond that use algorithms (Quine-McCluskey).

Advantages

- Simple and fast for small no. of variables.
- Gives minimal expression with minimum literals.
- Reduces number of gates and cost.
- Easy to understand and implement.

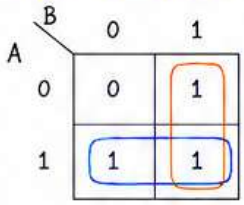
Limitations

- Not practical for more than 4 variables.
- Care needed while forming groups.
- Mistakes in Gray code can lead to wrong expression.

2. All Types of K-Map Examples

(a) 2-Variable K-map (4 cells)

$F(A,B) = \Sigma m(1,2,3)$



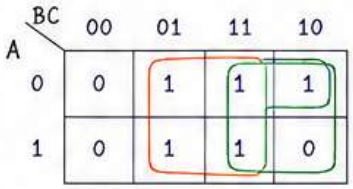
Groups : (1,3) → B

(2,3) → A

$F = A + B$

(b) 3-Variable K-map (8 cells)

$F(A,B,C) = \Sigma m(1,2,3,5,7)$



Groups :

(0,1,3,2) → $\overline{A}$

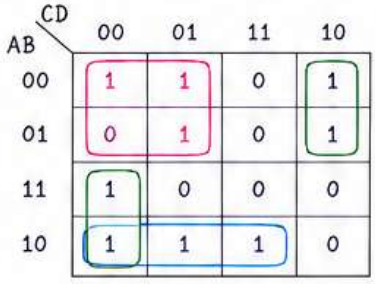
(1,5) → $\overline{B}C$

(3,7) → BC

$F = \overline{A} + BC + \overline{B}C$

(c) 4-Variable K-map (16 cells)

$F(A,B,C,D) = \Sigma m(0,1,2,5,8,9,10,14,15)$



Groups :

(0,1,8,9) → $\overline{B}\,\overline{C}$

(0,2,8,10) → $\overline{B}\,\overline{D}$

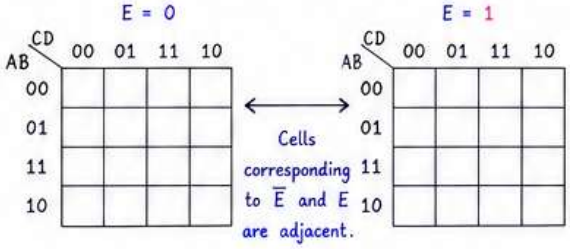
(5,7,13,15) → BD

(8,9,10,11) → $A\overline{C}$

$F = \overline{B}\,\overline{C} + \overline{B}\,\overline{D} + BD + A\overline{C}$

(d) 8-Variable K-map (256 cells)

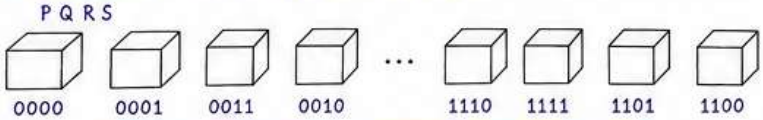
Representation : Use 2 four-variable K-maps with E as tabular variable.



Note : Each cell = minterm of 8 variables (A B C D E F G H).

(e) 16-Variable K-map (65536 cells)

Representation : Use 16 eight-variable K-maps.



Each small cube is an 8-variable K-map.
Adjacent cubes differ in only one of P, Q, R or S.

Grouping Rules in K-map

- Group sizes → 1, 2 (2^1), 4 (2^2), 8 (2^3), 16 (2^4) ...
- Groups must be rectangular.
- Wrap around allowed (top with bottom, left with right).

Interview / Viva Point

Q. Why Gray code is used in K-map ?

- To ensure that adjacent cells differ in only one variable, making grouping valid.

Boolean Minimization using K-Maps

1. Intro to Implicants and Types

An implicant is a product term that, when used in an OR expression, implies (produces 1 for) at least one minterm of the function.

1) Implicant :

Any product term that covers one or more minterms (having 1's) in the K-map.

Example : $\bar{A}\bar{B}$, AC , $\bar{B}\bar{D}$ are implicants.

2) Prime Implicant :

An implicant that cannot be combined (merged) with any other adjacent 1 (or don't care) to form a larger implicant.

Example : $A\bar{B}$, ACD , $\bar{B}\bar{D}\bar{C}$ are prime implicants.

3) Essential Prime Implicant (EPI) :

A prime implicant that is the only one covering at least one minterm (1) of the function.

It must be included in the final expression.

Example : If minterm m5 is covered only by $AC\bar{D}$, then $AC\bar{D}$ is an essential prime implicant.

4) Selective Prime Implicant :

A prime implicant that is not essential but is selected (along with other primes) to cover remaining minterms not covered by EPIs.

5) Redundant Prime Implicant :

A prime implicant that is not required in the final expression because all the minterms it covers are already covered by other selected prime implicants.

Key Point

- Every prime implicant is an implicant.
- Every essential prime implicant is a prime implicant.
- All EPIs must be selected in the final solution.
- Other primes are selected if required to cover remaining minterms.
- Redundant primes are not used in the final form.

Advantages of K-Map

- Simple and visual method.
- Gives minimal SOP directly.
- Easy for up to 4 variables.
- Helps in understanding groups and simplification.

Limitations

- Not practical for more than 4 variables.
- Care needed for correct grouping.
- Different possible but equivalent solutions.

Summary / Key Points

- Implicant → covers one or more 1's.
- Prime implicant → cannot be combined further.
- Essential prime implicant → only covering a minterm.
- Selective prime implicant → chosen to cover remaining 1's.
- Redundant prime implicant → not required.
- K-map minimizes Boolean expressions by grouping 1's (and don't cares) in largest possible powers of 2.

2. All Types of K-Map Examples

(a) 2-Variable K-Map (4 cells)

A \ B	0	1
0	1	0
1	0	1

$\Sigma m(0,3)$

$F = \bar{A}\bar{B} + AB$

(b) 3-Variable K-Map (8 cells)

A \ BC	00	01	11	10
0	1	0	1	0
1	0	1	1	0

$\Sigma m(0,3,5,7)$

$F = \bar{A}\bar{C} + AC$

(c) 4-Variable K-Map (16 cells)

AB \ CD	00	01	11	10
00	1	1	0	1
01	1	1	0	1
11	0	0	0	1
10	0	0	1	1

$\Sigma m(0,1,2,3,8,9,10,11,12,13,14,15)$

$F = \bar{A}\bar{C} + \bar{B}\bar{D} + AC$

(d) 8-Variable K-Map (256 cells)

Use 2 four-variable K-maps (one for E = 0 and one for E = 1) with variables A, B, C, D on both maps.

Each cell corresponds to minterms where only E differs.

(e) 16-Variable K-Map (65536 cells)

Use 16 eight-variable K-maps. Each map represents value of remaining variables (P, Q, R, S) in Gray code order.

Grouping is done across maps also when adjacent.

Detailed Example : 4-Variable K-Map with Don't Cares

Minimize $F(A,B,C,D) = \Sigma m(0,1,2,5,6,7,8,10,14,15) + d(3,9,13)$

Step 1 : Draw K-map

AB \ CD	00	01	11	10
00	1	1	0	d
01	1	0	1	1
11	0	d	d	d
10	1	0	1	0

Grouping Used :

- G1 (4 cells) → $\bar{A}\bar{C}$
- G2 (4 cells) → $\bar{A}D$
- G3 (2 cells) → BCD
- G4 (2 cells) → $A\bar{B}\bar{D}$

Step 2 : Write terms from groups

G1 : $\bar{A}\bar{C}$, G2 : $\bar{A}D$, G3 : BCD , G4 : $A\bar{B}\bar{D}$

Step 3 : Final minimized SOP expression

$$F = \bar{A}\bar{C} + \bar{A}D + BCD + A\bar{B}\bar{D}$$

Note

- Don't cares (d) are used only to make larger groups.

Grouping Rules in K-Map

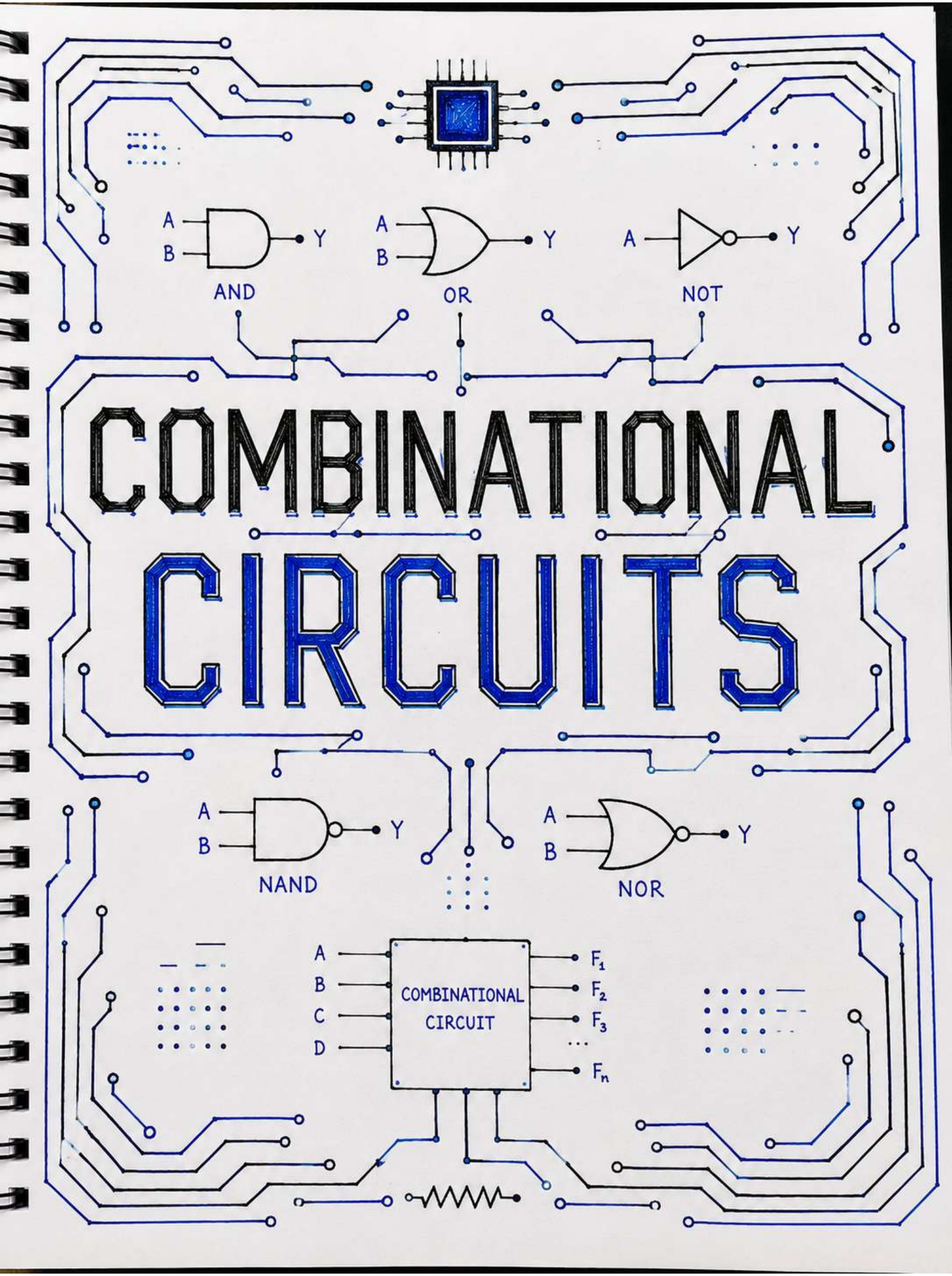
- Group sizes → $1, 2(2^1), 4(2^2), 8(2^3), 16(2^4)\ldots$
- Groups must be rectangular.
- Groups may overlap.
- Wrap around allowed (top-bottom, left-right).

Interview / Viva Point

Q. Why are don't cares used in K-map?

→ To form larger groups which may lead to minimal Boolean expression.

Author : Konda Tharun Vivekananda


A
B
Y
AND
A
B
Y
OR
A
Y
NOT
COMBINATIONAL
CIRCUITS
A
B
Y
NAND
A
B
Y
NOR
A
B
C
D
COMBINATIONAL
CIRCUIT
F1
F2
F3
...
Fn

Combinational Circuits

1. Introduction & Design Procedure

Introduction :

A combinational circuit is a logic circuit in which the output at any instant depends only on the present combination of inputs.



There is no memory element (flip-flop, latch) in it.

Example : Adders, MUX, Decoders, Comparators, etc.

Design Procedure of Combinational Circuits :

1) Understand the problem and list all inputs and outputs.
2) Determine the number of variables.
3) Prepare the truth table.
4) Obtain Boolean expressions for each output (SOP / POS form) using Boolean algebra or K-Map.
5) Simplify the expressions.
6) Draw the logic diagram using basic gates.
7) Verify the circuit with truth table.

Key Point

- Output depends only on present inputs.
- No feedback path.
- Does not have memory (stateless).
- Implementation uses logic gates only.

3. Advantages, Disadvantages & Applications

Advantages

- Simple and easy to understand.
- High speed operation (no clock waiting).
- No clock signal required.
- Design and testing are easy.
- Low power for small/medium circuits.
- Useful for arithmetic and logic operations.

Disadvantages

- Number of gates increases with number of inputs.
- Not suitable for tasks requiring memory.
- Limited to instant processing (no history).
- Can be complex for large number of variables.

Applications

- Adders, Subtractors, Multiplexers, De-Multiplexers.
- Encoders, Decoders, Priority Encoders.
- Comparators, Parity Generators, Checkers.
- Code Converters, Arithmetic Logic Unit (ALU).
- Control circuits in computers, processors, peripherals.

Summary / Key Points

- Combinational circuits are memoryless digital circuits.
- Output = f (Present Inputs) only.
- Designed using truth table, Boolean algebra and K-maps.
- Faster and simpler but cannot store data.
- Widely used in arithmetic, data routing and control circuits.

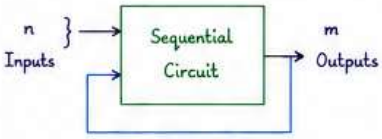
2. Combinational vs Sequential Circuits

Parameter	Combinational Circuits	Sequential Circuits
Output	Depends only on present inputs.	Depends on present inputs and past states.
Memory	No memory (stateless).	Has memory (stateful).
Feedback	No feedback path.	Has feedback path.
Clock	No clock required.	Usually requires clock.
Building Blocks	Logic gates (AND, OR, NOT, NAND, etc.)	Flip-flops, Latches, Registers + gates.
Design Method	Truth table, Boolean algebra, K-map, Logic diagram.	State table / State diagram, Flip-flop equations.
Speed	Generally faster.	Relatively slower.
Example	Adder, MUX, Decoder, Comparator.	Counter, Register, Sequence Detector.

Block Diagram Comparison :



(a) Combinational Circuit



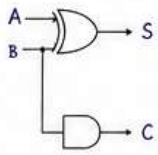
(b) Sequential Circuit

Examples of Combinational Circuits

1) Half Adder

$S = A \otimes B$, $C = A.B$

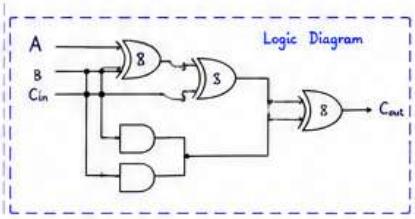
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



2) Full Adder

$S = A \otimes B \otimes C_{in}$

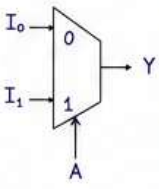
$C_{out} = AB + BC_{in} + AC_{in}$



3) 2 : 1 Multiplexer

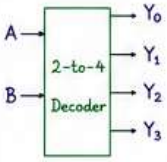
$Y = A' I_0 + A I_1$

A	I_0	I_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



4) 2 - to - 4 Decoder

A	B	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



General Points

- Combinational circuits do not store data.
- Each output is an instant function of inputs.
- Increasing inputs → more gates/complexity.
- Used in data path of most digital systems.

Interview / Viva Point

Q. Why combinational circuits are called memoryless circuits?

→ Because the output depends only on the current inputs, not on past inputs.

Arithmational Circuit Design : Adders

1. Design Arithmetic Circuit - Half - Adder (Combinational Circuit)

Objective : To design a Half-Adder which adds two 1-bit inputs A and B and produces Sum (S) and Carry (C).

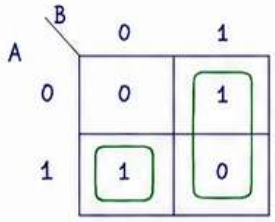
Step 1 : Truth Table

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Logic :
A + B = 2-bit result (C S)
S → LSB
C → MSB

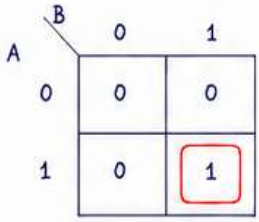
Step 2 : K-Map Solving

For $\hat{S}$ (Sum)



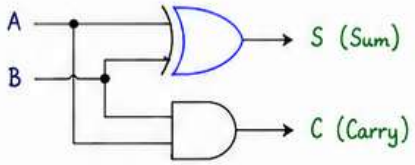
$S = \bar{A}\bar{B} + A\bar{B} = A \oplus B$

For C (Carry)



$C = AB$

Step 3 : Design of Logic Gates



$S = A \oplus B$
$C = A\ B$

Step 4 : Block Diagram



Key Points

- Half-Adder adds two 1-bit numbers.
- Sum is 1 when inputs are different.
- Carry is 1 only when both inputs are 1.
- Uses XOR for Sum and AND for Carry.

Advantages of Half-Adder

- Very simple adds due to implement.
- Requires only 2 basic gates (XOR, AND).
- Low hardware cost.
- Fast operation (no clock or memory).

Disadvantages

- Adds only two 1-bit numbers.
- Cannot handle carry input from previous stage.
- Not suitable for multi-bit addition alone.

Applications

- Building block of Full-Adder.
- Used in ALU, adders, counters, etc.
- Used in arithmetic and digital circuits.

2. Design Arithmetic Circuit - Full - Adder (Combinational Circuit)

Objective : To design a Full-Adder which adds three 1-bit inputs A, B and C_{in} and produces Sum (S) and Carry (C_{out}).

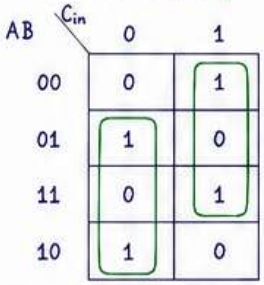
Step 1 : Truth Table

A	B	C_{in}	Sum (S)	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logic :
A + B + C_{in}
= 2-bit result (C_{out} S)
S → LSB
C_{out} → MSB

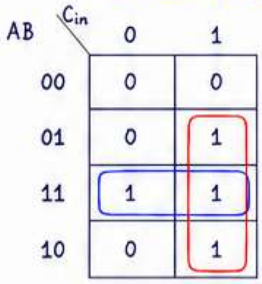
Step 2 : K-Map Solving (3-variable K-Map)

For $\hat{S}$ (Sum)



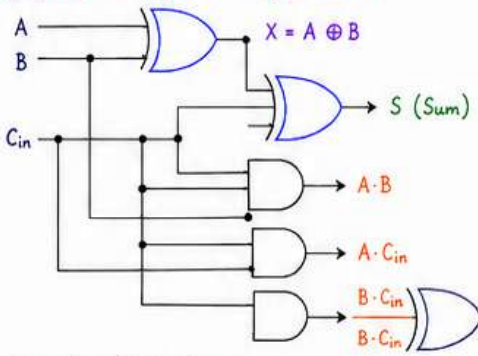
$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$
$= A \oplus B \oplus C_{in}$

For Cout (Carry)



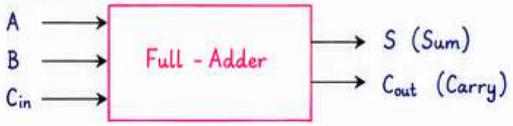
$C_{out} = AB + AC_{in} + BC_{in}$

Step 3 : Design of Logic Gates



$S = A \oplus B \oplus C_{in}$
$C_{out} = AB + AC_{in} + BC_{in}$

Step 4 : Block Diagram



Summary / Key Points

- Half-Adder adds two 1-bit inputs (A, B).
- Full-Adder adds three 1-bit inputs (A, B, C_{in}).
- Half-Adder → Uses XOR and AND gates.
- Full-Adder → Uses XOR, AND and OR gates.
- Full-Adder → Two Half-Adders & OR gate.
- Full-Adders are cascaded to form n-bit adders.

Interview / Viva Point

Q. How is Full-Adder different from Half-Adder ?
→ Half-Adder adds only two inputs and has no C_{in}.
Full-Adder considers C_{in} and produces C_{out}.

Arithmational Circuit Design : (Using Building Blocks)

1. Design Arithmetic Circuit - Full Adder using Half Adders (Combinational Circuit)

Objective : To design a Full-Adder using two Half-Adders.

Step 1 : Truth Table

A	B	C_{in}	Sum (S)	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logic :

$S = A \otimes B \otimes C_{in}$

$C_{out} = AB + A C_{in} + B C_{in}$

(3-input XOR for Sum)

Step 2 : K-Map Solving

For S (Sum)

AB \ C_{in}	00	01	11	10
00	0	0	1	0
01	0	0	1	1
11	1	1	x	1
10	1	0	x	1

$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + AB C_{in} + AB\bar{C}_{in}$

$= A \otimes B \otimes C_{in}$

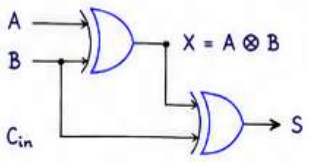
For Cout (Carry)

AB \ C_{in}	00	01	11	10
00	0	0	0	0
01	0	0	1	1
11	0	0	0	1
10	0	0	0	1

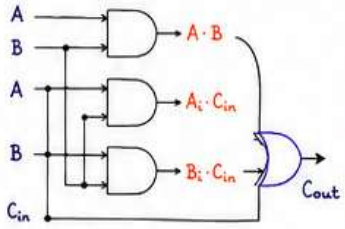
$C_{out} = AB + AC_{in} + BC_{in}$

Step 3 : Design of Logic Gates

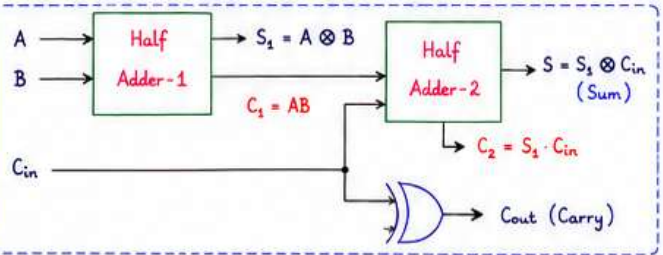
(i) Sum (S) = $A \otimes B \otimes C_{in}$



(ii) Carry (C_{out}) = $AB + AC_{in} + BC_{in}$



Step 4 : Block Diagram (Using Half-Adders)



Key Point

- Full-Adder can be realized using two Half-Adders and one OR gate.
- Requires minimum gates and is easy to implement.

2. Design Arithmetic Circuit - Ripple Carry Adder (Combinational Circuit)

Objective : To design an n-bit Adder using Full-Adders connected in cascade (Ripple Carry).

Step 1 : Concept

An n-bit Ripple Carry Adder consists of n Full-Adders. The carry out of each stage is connected to the carry in of the next higher stage.

For n-bit inputs

$A = A_{n-1} \ldots A_1 A_0$

$B = B_{n-1} \ldots B_1 B_0$

$C_{in} = C_0$

Outputs :

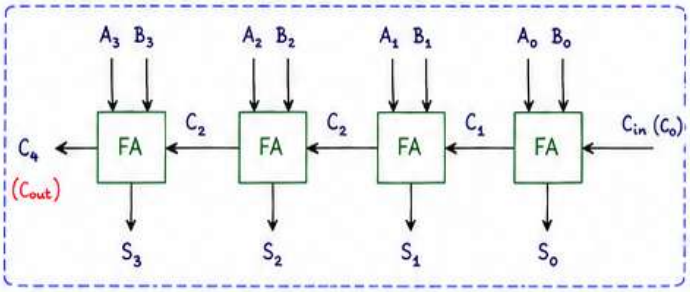
Sum = $S_{n-1} \ldots S_1 S_0$

Carry out = C_n

Step 2 : 1-Bit Full-Adder



Step 3 : 4-Bit Ripple Carry Adder Block Diagram

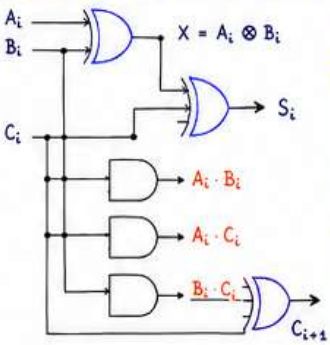


Step 4 : Boolean Expressions (1-bit FA)

$S_i = A_i \otimes B_i \otimes C_i$

$C_{i+1} = A_i B_i + A_i C_i + B_i C_i$

Step 5 : Logic Diagram of 1-bit FA



Step 6 : Example

Add 4-bit numbers A = 0101 and B = 0011

Stage	A_i	B_i	C_i (Input)	S_i (Output)	C_{i+1} (Output)
0 (LSB)	1	1	0	0	1
1	0	1	1	0	1
2	1	0	1	0	1
3 (MSB)	0	0	1	1	0
Final Cout = C_4 = 0			Result (S_3 S_2 S_1 S_0) = 1000		

Advantages of Ripple Carry Adder

- Simple and easy to design.
- Uses only Full-Adders (modular).
- Suitable for small number of bits.
- Scalable (n-bit by cascading).

Disadvantages

- Carry must ripple through all stages.
- Delay increases with number of bits.
- Not suitable for high-speed adders.

Applications

- ALU in computer systems.
- Counters and arithmetic operations.
- Address calculation, data routing.
- General purpose arithmetic units.

Interview / Viva Point

Q. Why is it called Ripple Carry Adder ?

→ Because the carry "ripples" from one stage to the next until the final carry out is produced.

Arithmational Circuit Design : Subtractors

1. Design Arithmatic Circuit - Half Subtractor (Combinational Circuit)

Objective : To design a Half Subtractor which subtracts B from A and produces Difference (D) and Borrow (B_{out}).

Step 1 : Truth Table

A	B	Difference (D)	Borrow (B_{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Logic :
A - B
If $A \geq B$
$D = A - B$, $B_{out} = 0$
If $A < B$
$D = 1$, $B_{out} = 1$

Step 2 : K-Map Solving

For D (Difference)

A \ B	0	1
0	0	1
1	1	0

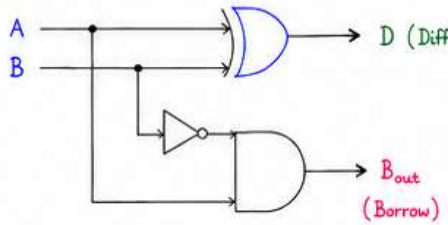
$D = \bar{A}B + A\bar{B} = A \otimes B$

For B_{out} (Borrow)

A \ B	0	1
0	0	1
1	0	0

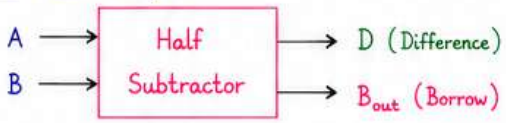
$B_{out} = \bar{A}B$

Step 3 : Design of Logic Gates



$D = A \otimes B$
$B_{out} = \bar{A}B$

Step 4 : Block Diagram



Key Point

- Half Subtractor works with two 1-bit inputs.
- Difference is 1 when inputs are different.
- Borrow is 1 only when A < B.
- Uses XOR gate for Difference and AND gate with inverter for Borrow.

2. Design Arithmatic Circuit - Full Subtractor (Combinational Circuit)

Objective : To design a Full Subtractor which subtracts B and borrow input (B_{in}) from A and produces Difference (D) and Borrow out (B_{out}).

Step 1 : Truth Table

A	B	B_{in}	Difference (D)	Borrow out (B_{out})
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Logic :
$A - B - B_{in}$
If $A \geq (B + B_{in})$
$D = A - B - B_{in}$
$B_{out} = 0$
If $A < (B + B_{in})$
$D = 1 + A - (B + B_{in})$
$B_{out} = 1$

Step 2 : K-Map Solving (3-variable K-Map)

For D (Difference)

A \ B B_{in}	00	01	11	10
0	0	1	0	1
1	1	0	1	0

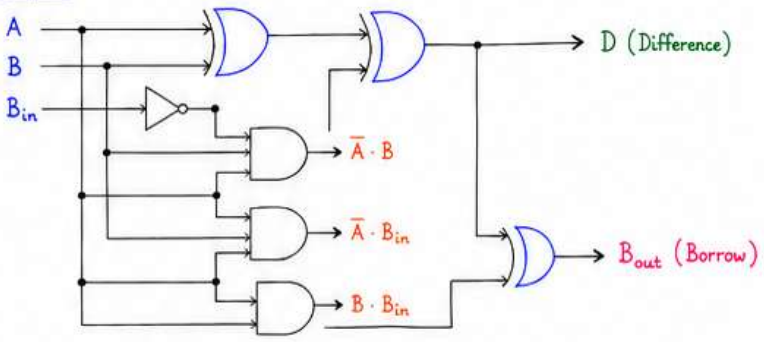
$D = A \otimes B \otimes B_{in}$

For B_{out} (Borrow out)

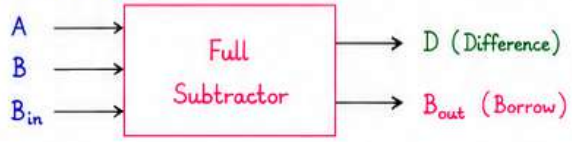
A \ B B_{in}	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$B_{out} = \bar{A}B + \bar{A}\bar{B}_{in} + BB_{in}$

Step 3 : Design of Logic Gates



Step 4 : Block Diagram



Advantages

- Essential for binary subtraction.
- Simple and fast combinational circuits.
- Used in processors, ALU, arithmetic units.
- Can be cascaded to build multi-bit subtractors.

Disadvantages

- Borrow propagation delay increases in multi-bit systems.
- More gates in Full Subtractor compared to Half Subtractor.
- Not suitable for very high speed applications without optimization.

Applications

- Used in ALU for subtraction operations.
- Used in digital calculators.
- Used in DSP and microprocessors.
- Used in counters and control units.

Interview / Viva Point

Q. Why is Full Subtractor more complex than Half Subtractor ?
→ Because it has an additional borrow input (B_{in}) and must consider three cases while producing Difference and Borrow output.

Summary / Key Points

- Half Subtractor has 2 inputs (A, B) and 2 outputs (D, B_{out}).
- Full Subtractor has 3 inputs (A, B, B_{in}) and 2 outputs (D, B_{out}).

$D_{HS} = A \otimes B$
$B_{out\ HS} = \bar{A}B$
$D_{FS} = A \otimes B \otimes B_{in}$
$B_{out\ FS} = \bar{A}B + \bar{A}\bar{B}_{in} + BB_{in}$

Multiplexer - 2X1 Design (using Logic Gates)

1. Design Arithmetic Circuit - Multiplexer - 2X1 (Combinational Circuit)

Objective : To design a 2X1 Multiplexer which selects one of the two inputs (D_0 or D_1) based on Select line (S) and produces output Y.

Step 1 : Truth Table

S	D_0 (0/1)	D_1 (0/1)	Y (Output) (0/1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Function :
If S = 0 → Y = D_0
If S = 1 → Y = D_1
(When S = 0, D_0 selected)
(When S = 1, I_1 selected)

Step 2 : K-Map Solving

For Y = D_0

S \	0	1
0	1	1
1	0	0

$Y = D_0 \overline{S}$

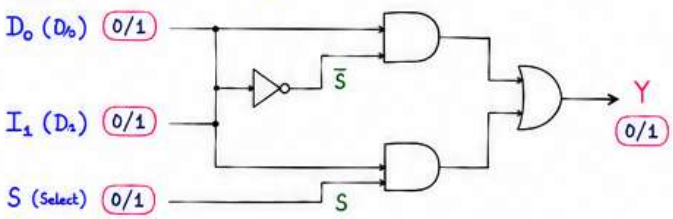
For Y = I_1

S \	0	1
0	0	0
1	1	1

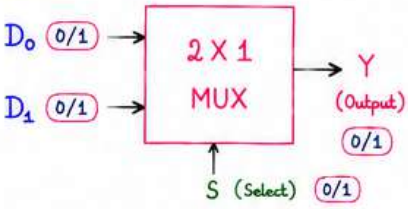
$Y = D_1 S$

Final Boolean Expression : $Y = D_0 \overline{S} + D_1 S$

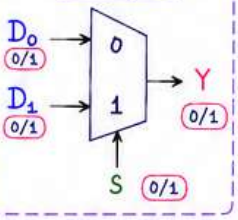
Step 3 : Design of Logic Gates



Step 4 : Block Diagram



MUX Symbol



Key Points

- Two data inputs D_0 and D_1 and one select line (S).
- If S = 0 → Y = D_0 , If S = 1 → Y = D_1.
- Boolean expression : $Y = D_0 \overline{S} + D_1 S$
- Implemented using 2 AND gates, 1 OR gate and 1 NOT gate.

2. Design Arithmetic Circuit - all Logic Gates Multiplexer - 2X1 (Mux Symbol Only) (Combinational Circuit)

Objective : To represent 2X1 Multiplexer using all basic logic gates separately, (Mux symbol only)

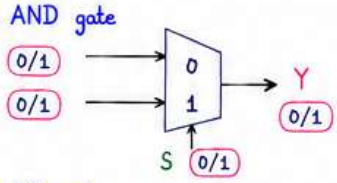
Truth Table (Same for all implementations)

S	D_0 (0/1)	D_1 (0/1)	Y (0/1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

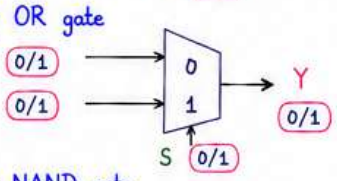
Function :
If S = 0 → Y = D_0
If S = 1 → Y = D_1

ALL Logic Gates Implementations (Mux Symbol Only)

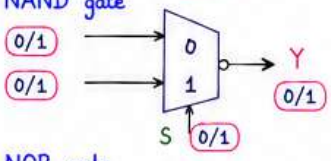
1) Using AND gate



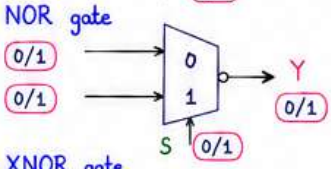
2) Using OR gate



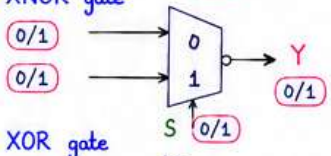
3) Using NAND gate



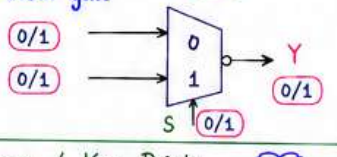
4) Using NOR gate



5) Using XNOR gate



6) Using XOR gate



Summary / Key Points

- 2X1 Multiplexer selects one of the two inputs based on select line.
- Boolean expression : $Y = D_0 \overline{S} + D_1 S$
- Can be implemented using different logic gates.
- Mux symbol remains same; only internal gates change.

Applications

- Data selection in digital circuits.
- Input selection in communication systems.
- Bus selection and data routing.
- Used in ALU, registers, etc.

Advantages

- Simple and easy to design.
- Reduces hardware for data selection.
- Fast operation.
- Widely used in digital systems.

Interview / Viva Point

Q. What does a 2X1 Multiplexer do ?
→ It selects one of two inputs (D_0 or D_1) based on select line (S) and gives the corresponding output Y.

Multiplexer - 4X1 Design (using Logic Gates)

1. Design Arithmetic Circuit - Multiplexer - 4X1 (Combinational Circuit)

Objective : To design a 4X1 Multiplexer which selects one of the four inputs (D_0, D_1, D_2 or D_3) based on Select lines (S_1, S_0) and produces output Y.

Step 1 : Truth Table

S_1	S_0	D_0 (0/1)	D_1 (0/1)	D_2 (0/1)	D_3 (0/1)	Y (out) (0/1)
0	0	1	0	0	0	1
0	0	0	1	0	0	0
0	1	0	0	1	0	1
0	1	0	0	0	1	0
1	0	0	0	1	0	1
1	0	0	0	0	1	0
1	1	0	0	0	0	1
1	1	0	0	0	1	0

Function :
- If $S_1S_0 = 00 \rightarrow Y = D_0$
- If $S_1S_0 = 01 \rightarrow Y = D_1$
- If $S_1S_0 = 10 \rightarrow Y = D_2$
- If $S_1S_0 = 11 \rightarrow Y = D_3$

(When 00 selected, D_0 passes)
(When 01 selected, D_1 passes)
(When 10 selected, D_2 passes)
(When 11 selected, D_3 passes)

Step 2 : K-Map Solving

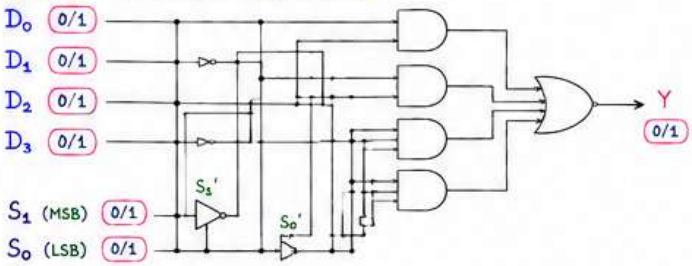
For Y

S_1 \ S_0	0	1
	D_0	D_1	D_2	D_3
0	1	0	1	0
1	1	0	1	0

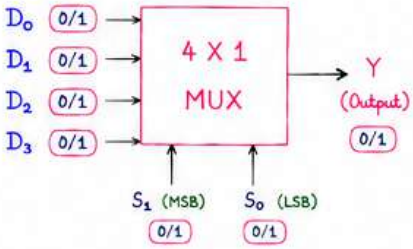
Groupings :
- $D_0 \rightarrow S_1' S_0'$
- $D_1 \rightarrow S_1' S_0$
- $D_2 \rightarrow S_1 S_0'$
- $D_3 \rightarrow S_1 S_0$

Final Boolean Expression :
$Y = D_0 S_1' S_0' + D_1 S_1' S_0 + D_2 S_1 S_0' + D_3 S_1 S_0$

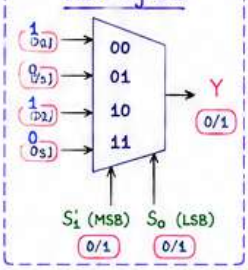
Step 3 : Design of Logic Gates



Step 4 : Block Diagram



MUX Symbol



Key Points

- 4 data inputs (D_0, D_1, D_2, D_3) and 2 select lines (S_1, S_0).
- $S_1S_0 = 00 \rightarrow D_0$, $01 \rightarrow D_1$; $10 \rightarrow D_2$; $11 \rightarrow D_3$.
- Boolean expression : $Y = D_0 S_1' S_0' + D_1 S_1' S_0 + D_2 S_1 S_0' + D_3 S_1 S_0$
- Implemented using 4 AND gates, 2 NOT gates and 1 OR gate.

2. Design Arithmetic Circuit - all Logic Gates Multiplexer - 4X1 (Mux Symbol Only) (Combinational Circuit)

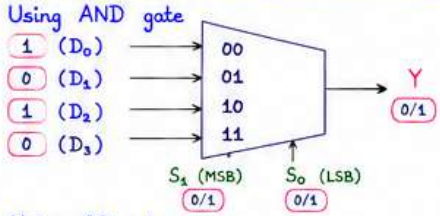
Objective : To represent 4X1 Multiplexer using all basic logic gates separately. (Mux symbol only)

Truth Table (Same for all implementations)

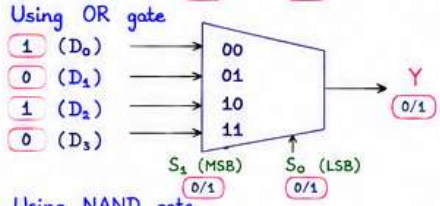
S_1	S_0	D_0 (0/1)	D_1 (0/1)	D_2 (0/1)	D_3 (0/1)	Y (0/1)
0	0	1	0	0	0	1
0	0	0	1	0	0	0
0	1	0	0	1	0	1
0	1	0	0	0	1	0
1	0	0	0	1	0	1
1	0	0	0	0	1	0
1	1	0	0	0	0	1
1	1	0	0	0	1	0

ALL Logic Gates Implementations (Mux Symbol Only)

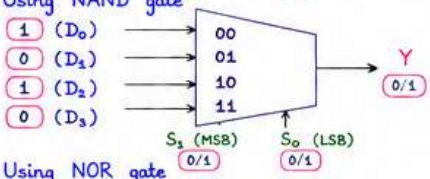
1) Using AND gate



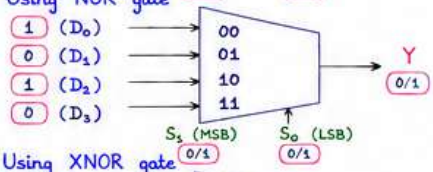
2) Using OR gate



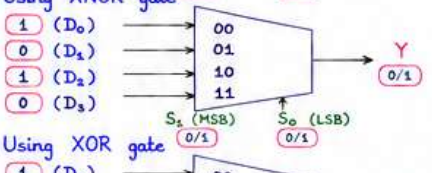
3) Using NAND gate



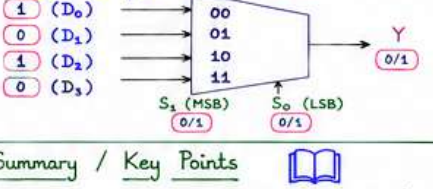
4) Using NOR gate



5) Using XNOR gate



6) Using XOR gate



Summary / Key Points

- 4X1 Multiplexer selects one of the four inputs based on S_1, S_0.
- Boolean expression : $Y = D_0 S_1' S_0' + D_1 S_1' S_0 + D_2 S_1 S_0' + D_3 S_1 S_0$
- Can be implemented using different logic gates.
- Mux symbol remains same; only internal gates change.

Applications

- Data selection in digital systems.
- Input selection in communication systems.
- Bus selection and data routing.
- Used in ALU, registers, memory, etc.

Advantages

- Simple and efficient hardware.
- Easy to design and implement.
- Fast operation.
- Widely used in digital systems.

Interview / Viva Point

Q. What does a 4X1 Multiplexer do ?
→ It selects one of the four inputs (D_0, D_1, D_2 or D_3) based on the select lines (S_1, S_0) and gives the corresponding output Y.

Author : Konda Tharun Vivekananda

Multiplexer - 8X1 and 16X1 Design (using Logic Gates)

1. Design Arithmetic Circuit - Multiplexer - 8X1 (Combinational Circuit)

Objective : To design a 8X1 Multiplexer which selects one of the eight inputs (D_0 to D_7) based on select lines (S_2, S_2, S_0) and produces output Y.

Step 1 : Truth Table

S_2	S_1	S_0	D_0 (0/1)	D_1 (0/1)	D_2 (0/1)	D_3 (0/1)	D_5 (0/1)	D_6 (0/1)	D_7 (0/1)	Y (0/1)
0	0	0	1	0	0	0	0	0	0	1
0	0	1	0	1	0	0	0	0	0	1
0	1	0	0	0	1	0	0	0	0	1
0	1	1	0	0	0	1	0	0	0	1
1	0	0	0	0	0	0	1	0	0	1
1	0	1	0	0	0	0	0	1	0	1
1	1	0	0	0	0	0	0	0	0	1
1	1	1	0	0	0	0	0	0	1	1

Function :

If $S_2\,S_1\,S_0$ = 000 → $Y = D_0$
001 → $Y = D_1$
010 → $Y = D_2$
011 → $Y = D_3$
100 → $Y = D_4$
101 → $Y = D_5$
110 → $Y = D_6$
111 → $Y = D_7$

Step 2 : K-Map Solving

S_2 \ S_1S_0	00	01	11	10
0	1 (D_0)	1 (D_1)	1 (D_3)	1 (D_2)
1	1 (D_4)	1 (D_5)	1 (D_7)	1 (D_6)

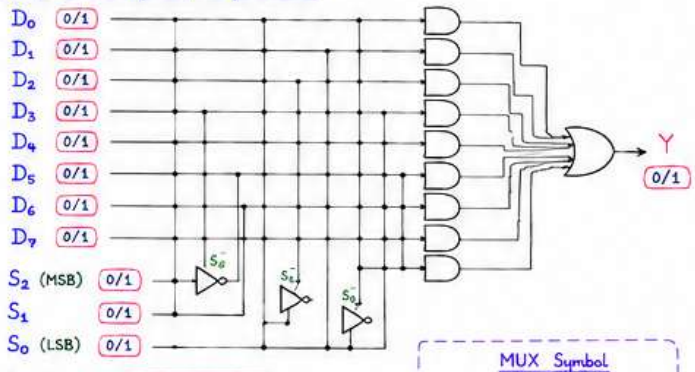
All cells are 1 (one each).

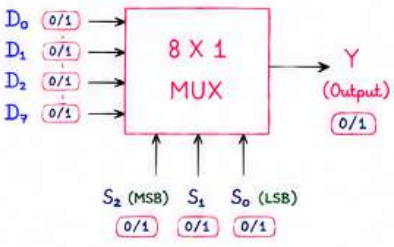
Groups :
Each cell forms a group of 1.

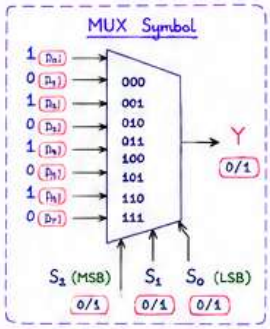
Final Boolean Expression :

$$Y = D_0\,S_2\,\bar{S_1}\,\bar{S_1}\,\bar{S_0} + D_1\,S_2\,\bar{S_1}\,\bar{S_1}\,S_0 + D_2\,S_2\,\bar{S_1}\,S_1\,\bar{S_0} + D_3\,S_2\,\bar{S_1}\,S_1\,S_0 + D_4\,S_2\,S_1\,\bar{S_1}\,\bar{S_0} + D_5\,S_2\,S_1\,\bar{S_1}\,S_0 + D_6\,S_2\,S_1\,S_1\,\bar{S_0} + D_7\,S_2\,S_1\,S_1\,S_0$$

Step 3 : Design of Logic Gates







Key Points (8X1)

- Eight data inputs D_0 - D_7 and three select lines S_2, S_1, S_0.
- When $S_2\,S_1\,S_0 = k$ (0 to 7), output $Y = D_k$.
- Boolean expression has 8 product terms.
- Implemented using 8 AND gates and 1 OR gate (with inverters).

2. Design Arithmetic Circuit - Multiplexer - 16X1 (Combinational Circuit)

Objective : To design a 16X1 Multiplexer which selects one of the sixteen inputs (D_0 to D_{15}) based on select lines (S_3, S_2, S_1, S_0) and produces output Y.

Step 1 : Truth Table

S_3	S_2	S_1	S_0	D_0 (0/1)	D_1 (0/1)	D_2 (0/1)	D_3 (0/1)	.....	D_{15} (0/1)	Y (out) (0/1)
0	0	0	0	1	0	0	0	....	0	1
0	0	0	1	0	1	0	0	....	0	1
0	0	1	0	0	0	1	0	....	0	1
0	0	1	1	0	0	0	1	....	0	1
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	....	⋮	⋮
1	1	1	1	0	0	0	0	....	1	1

Function :

If $S_3\,S_2\,S_1\,S_0$ = 0000 → $Y = D_0$
0001 → $Y = D_1$
⋮
1111 → $Y = D_{15}$

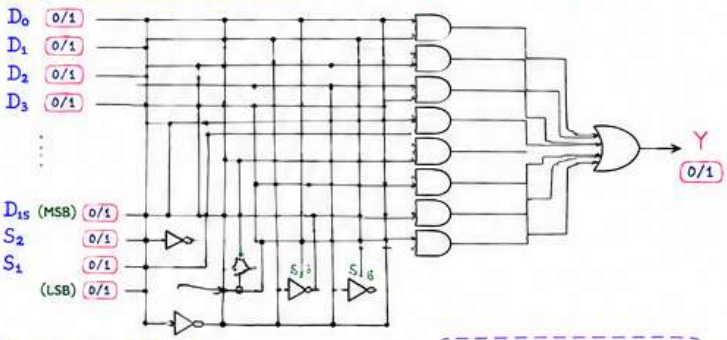
Step 2 : K-Map Solving

S_3 \ S_2S_1	00	01	11	10
00	1 (D_0)	1 (D_1)	1 (D_3)	1 (D_2)
01	1 (D_4)	1 (D_5)	1 (D_7)	1 (D_6)
11	1 (D_{12})	1 (D_{13})	1 (D_{15})	1 (D_{14})
10	1 (D_8)	1 (D_9)	1 (D_{11})	1 (D_{10})

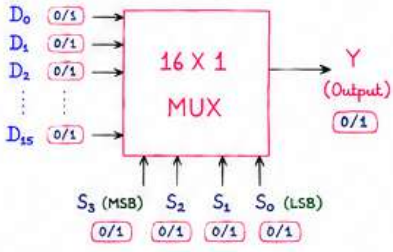
Final Boolean Expression :

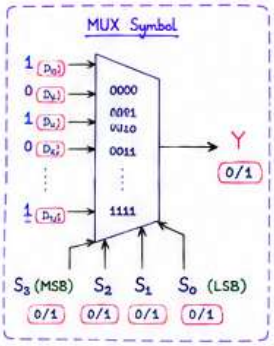
$$Y = D_0\,\bar{S_3}\,\bar{S_2}\,\bar{S_1}\,\bar{S_0} + D_1\,\bar{S_3}\,\bar{S_2}\,\bar{S_1}\,S_0 + \cdots + D_{15}\,S_3\,S_2\,S_1\,S_0$$

Step 3 : Design of Logic Gates (16 AND gates + 1 OR gate)



Step 4 : Block Diagram





Key Points (16X1)

- Sixteen data inputs D_0 - D_{15} and four select lines S_3, S_2, S_1, S_0.
- When $S_3\,S_2\,S_1\,S_0 = k$ (0 to 15), output $Y = D_k$.
- Boolean expression has 16 product terms.
- Implemented using 16 AND gates and 1 OR gate (with inverters).

Applications

- Data selection in digital systems.
- Input selection in communication systems.
- Bus selection and data routing.
- Used in ALU, registers, memory, etc.

Advantages

- High speed data selection.
- Simple and regular structure.
- Scalable for higher inputs.
- Widely used in digital circuits.

Interview / Viva Point

Q. What does a 8X1 / 16X1 Multiplexer do ?
→ It selects one of the 8 / 16 inputs based on the select lines and gives the corresponding input as output Y.

Author : Konda Tharun Vivekananda

MULTIPLEXER (MUX) - INTRODUCTION, APPLICATIONS, ADVANTAGES, DISADVANTAGES

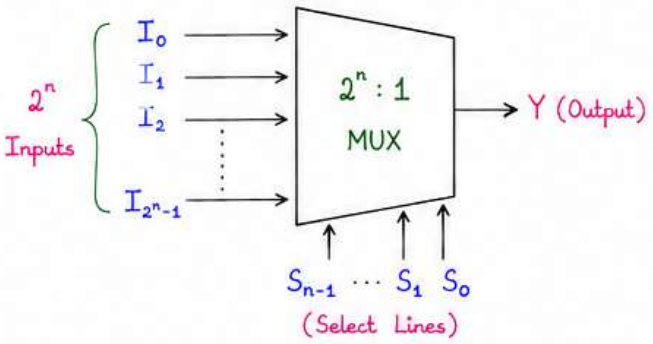
1. INTRODUCTION

→ A Multiplexer (MUX) is a combinational circuit that selects one of the many input lines and forwards the selected input to a single output.

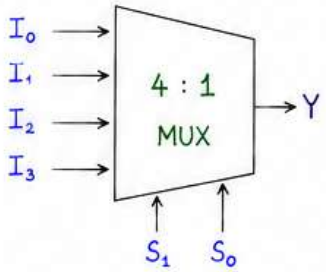
→ It is also called a "Data Selector".

→ A MUX with n select lines can select one from 2^n input lines.

GENERAL STRUCTURE OF A MUX



EXAMPLE : 4X1 MUX (n = 2)



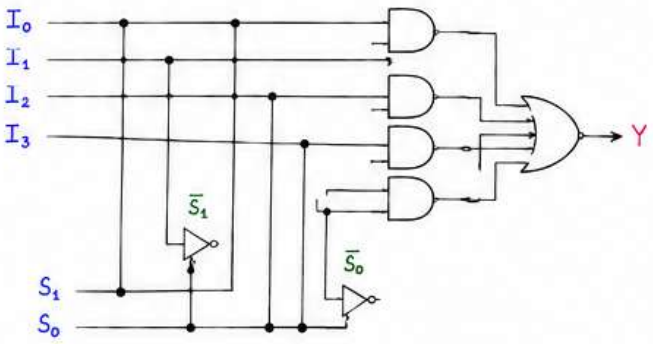
TRUTH TABLE (4X1 MUX)

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

BOOLEAN EXPRESSION (4X1 MUX)

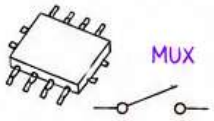
$$Y = \overline{S_1}\ \overline{S_0}\ I_0 + \overline{S_1}\ S_0\ I_1 + S_1\ \overline{S_0}\ I_2 + S_1\ S_0\ I_3$$

REALIZATION USING LOGIC GATES (4X1 MUX)



KEY POINTS

- Acts as an electronic switch.
- n select lines control 2^n inputs.
- Output is equal to the selected input.
- Widely used in digital systems.



2. APPLICATIONS

- Data selection in digital communication systems.
- Parallel to serial data conversion.
- Time division multiplexing in communication.
- Selection of memory locations or data sources.
- Switching and routing in digital systems.
- Used in ALU, CPUs, microcontrollers, FPGAs.
- Used in digital signal processing.

3. ADVANTAGES



- Reduces number of data lines.
- Simplifies digital circuit design.
- High speed operation.
- Less hardware for data selection.
- Flexible and easy to expand.
- Can be cascadea to build higher order MUX.
- Low power consumption (for modern ICs).

4. DISADVANTAGES



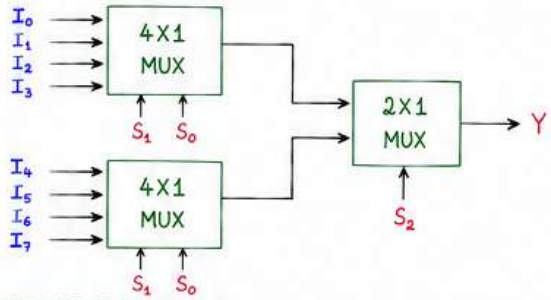
- More select lines are required as inputs increase.
- Propagation delay increases with size.
- Fan-in of gates increases (in gate level implementation).
- Large MUX occupies more chip area.
- Complexity increases for very high order MUX.

COMPARSION OF MUX SIZES

MUX Type	No. of Inputs	No. of Select Lines (n)	Selection Capacity
2 X 1 MUX	2	1	1 of 2
4 X 1 MUX	4	2	1 of 4
8 X 1 MUX	8	3	1 of 8
16 X 1 MUX	16	4	1 of 16

INTERESTING NOTE

Two 4X1 MUXs can be used to design an 8X1 MUX. Similarly, two 8X1 MUXs can be used to design a 16X1 MUX.



As the number of inputs doubles, the number of select lines increases by one.



MUX TREE (MULTIPLEXER TREE)

1. MULTIPLEXER TREE

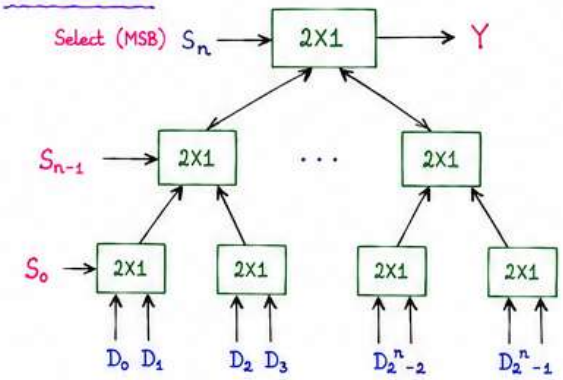
Introduction :

→ A MUX Tree is a hierarchical arrangement of small multiplexers to realize a higher order multiplexer.

→ Instead of using a single large MUX (8X1, 16X1, etc.), multiple 2X1 MUX blocks are connected in a tree structure.

→ This reduces chip area, delay and complexity.

General Idea :



Formulae :

- Number of data inputs = 2^n
- Number of select lines = n
- Number of 2X1 MUX used = $2^n - 1$
- Number of levels in tree = n
- Propagation delay ≈ $n \times t_{2x1}$

(assuming equal delay for each 2X1 MUX)

Key Point

At each level, the number of MUX reduces to half compared to previous level.

Applications :

- Data selection in digital communication systems.
- Parallel to serial data conversion.
- Time division multiplexing (TDM).
- Memory addressing and data routing.
- ALU, CPU, Microcontrollers, DSP, FPGA.
- Router / switching fabrics in networks.
- Selection of multiple sensor outputs.

Advantages :

- Uses only simple 2X1 MUXs → easy to design.
- Requires less hardware ($2^n - 1$ instead of complex large MUX).
- Lower propagation delay (n stages only).
- Modular and scalable.
- Easy to expand or modify.
- Better for VLSI implementation.

Disadvantages :

- More number of levels → delay increases with n.
- More select lines are required as order increases.
- Power consumption slightly increases with more stages.
- Wiring becomes complex for very high order.

2. EXAMPLE : 8X1 MUX USING 2X1 MUX

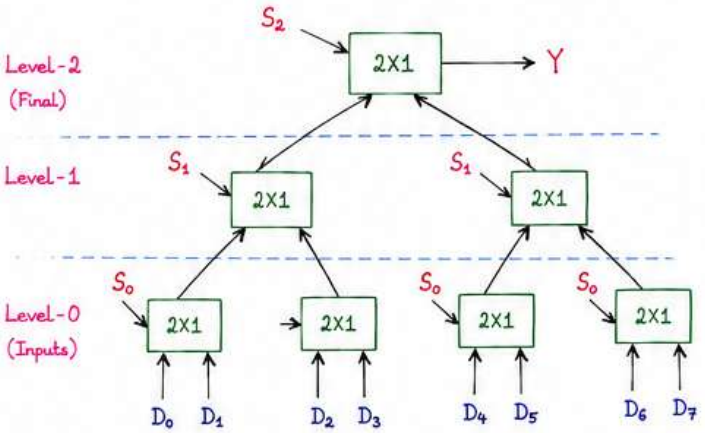
Objective : Design an 8X1 Multiplexer using only 2X1 MUXs in tree structure.

Given : 8 data inputs → D_0 to D_7
Select lines → S_2 (MSB), S_1, S_0 (LSB)
Output → Y

Construction Steps :

1. Pair the inputs and connect to first level 2X1 MUXs using S_0.
2. The outputs of first level are connected to second level 2X1 MUXs using S_1.
3. Outputs of second level are connected to final 2X1 MUX using S_2 to get Y.

Mux Tree Diagram (8X1 using 2X1 MUXs) :



Connections Summary :

- S_0 → Selects between (D_0, D_1), (D_2, D_3), (D_4, D_5), (D_6, D_7)
- S_1 → Selects between outputs of first level pairs.
- S_2 → Selects final output.

Number of 2X1 MUX used :

= $2^3 - 1$
= 8 − 1
= 7 MUXs

Operation Example :

If $S_2 = 1$, $S_1 = 0$, $S_0 = 1$ → Binary (1 0 1) = 5
Then output Y = D_5

Truth Table (8X1 MUX)

S_2 (MSB)	S_1	S_0 (LSB)	Selected Input	Output Y
0	0	0	D_0	D_0
0	0	1	D_1	D_1
0	1	0	D_2	D_2
0	1	1	D_3	D_3
1	0	0	D_4	D_4
1	0	1	D_5	D_5
1	1	0	D_6	D_6
1	1	1	D_7	D_7

Key Points

- Each path from input to output passes through 'n' 2X1 MUXs.
- All 2X1 MUXs at same level use same select line.
- This structure is called Multiplexer Tree.

Note :

Similarly, 16X1 MUX can be realized using 15 ($2^4 - 1$) number of 2X1 MUXs with 4 select lines (S_3, S_2, S_1, S_0).

Author : Konda Tharun Vivekananda

MUX TREE - EXAMPLES

1. MUX TREE - EXAMPLE OF 16X1 MUX USING 4X1 MUX

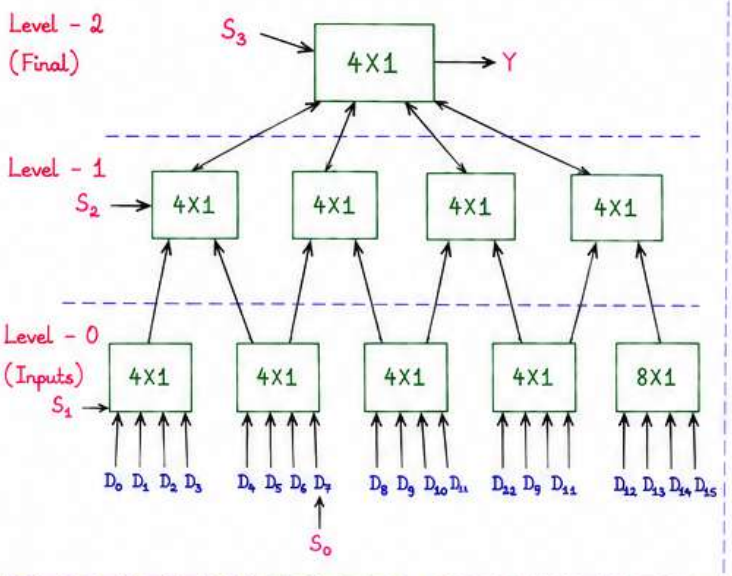
Objective : Realize a 16X1 MUX using 4X1 MUX blocks.

Given : 16 data inputs D_0 to D_{15}

Select lines : S_3 (MSB), S_2, S_1, S_0 (LSB)

Output : Y

Construction using 4X1 MUX blocks :



Connections Summary :

- S_0 → select line of all Level-0 (4X1) MUXs
- S_1 → select line of all Level-1 (4X1) MUXs
- S_2 → select line of all Level-2 (4X1) MUXs
- S_3 → select line of final (top) 4X1 MUX

General Formulae (using 4X1 MUX)

- Total inputs = $16 = 4^2$
- No. of levels = 2 (since 4^2)
- No. of 4X1 MUXs
  $= 8^2 - 1$
  $= 16 - 1 = 15$ MUXs
- Propagation delay $\approx 2 \times t_{8x1}$ (Two stages only)

Boolean Expression :

$$Y = \overline{S_3}\ Y_0 + S_3\ Y_1$$

where,

$$Y_0 = \overline{S_2}\ Y_{00} + S_2\ Y_{01}\ ,\quad Y_1 = \overline{S_2}\ Y_{10} + S_2\ Y_{11}$$

and each Y_{ij} (selected by S_1, S_0) is

$$Y_{00} = \overline{S_1}\ \overline{S_0}\ D_0 + \overline{S_1}\ S_0\ D_1 + S_1\ \overline{S_0}\ D_2 + S_1\ S_0\ D_3$$

$$Y_{01} = \overline{S_1}\ \overline{S_0}\ D_4 + \overline{S_1}\ S_0\ D_5 + S_1\ \overline{S_0}\ D_6 + S_1\ S_0\ D_7$$

$$Y_{10} = \overline{S_1}\ \overline{S_0}\ D_8 + \overline{S_1}\ S_0\ D_9 + S_1\ \overline{S_0}\ D_{10} + S_1\ S_0\ D_{11}$$

$$Y_{11} = \overline{S_1}\ \overline{S_0}\ D_{12} + \overline{S_1}\ S_0\ D_{13} + S_1\ \overline{S_0}\ D_{14} + S_1\ S_0\ D_{15}$$

Advantages

- Uses only smaller 4X1 MUXs.
- Reduces chip area and cost.
- Easy to design and modular.
- Low propagation delay (2 stages).
- Easy to expand to higher order.

Disadvantages

- More number of MUXs required.
- More interconnections.
- Slightly higher power usage.
- Delay increases with more stages.

2. MUX TREE - EXAMPLE OF 64X1 MUX USING 8X1 MUX

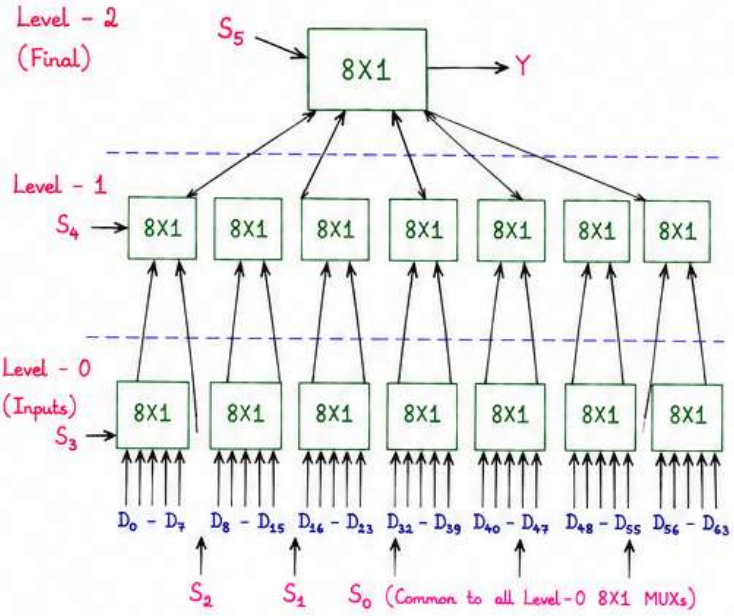
Objective : Realize a 64X1 MUX using 8X1 MUX blocks.

Given : 64 data inputs D_0 to D_{63}

Select lines : S_5 (MSB), S_4, S_3, S_2, S_1, S_0 (LSB)

Output : Y

Construction using 8X1 MUX blocks :



Connections Summary :

- S_0, S_1, S_2 → select lines of all Level-0 (8X1) MUXs
- S_3 → select line of all Level-1 (8X1) MUXs
- S_4 → select line of all Level-2 (8X1) MUXs
- S_5 → select line of final (top) 8X1 MUX

General Formulae (using 8X1 MUX)

- Total inputs = $64 = 8^2$
- No. of levels = 2 (since 8^2)
- No. of 8X1 MUXs
  $= 8^2 - 1$
  $= 64 - 1 = 63$ MUXs
- Propagation delay $\approx 2 \times t_{8x1}$

Boolean Expression (Hierarchical Form)

$$Y = \overline{S_5}\ A_0 + S_5\ A_1$$

where,

$$A_0 = \overline{S_4}\ B_{00} + S_4\ B_{01}\ ,\quad A_1 = \overline{S_4}\ B_{10} + S_4\ B_{11}$$

Each B_{ij} is an 8-to-1 function selected by (S_3, S_2, S_1, S_0) for inputs $D_{8i} - D_{8i+7}$.

Applications

- Large data bus selection.
- Communication systems.
- Memory and register selection.
- Digital switching networks.
- Used in FPGAs, CPU data paths, etc.

Key Points

- Tree structure reduces complexity.
- Uses only small MUX blocks.
- Number of MUXs = $(\text{Base})^2 - 1$.
- Number of levels = $\log_{\text{Base}}(N)$.

DEMULTIPLEXER (De MUX)

1. DEMULTIPLEXER (De MUX)

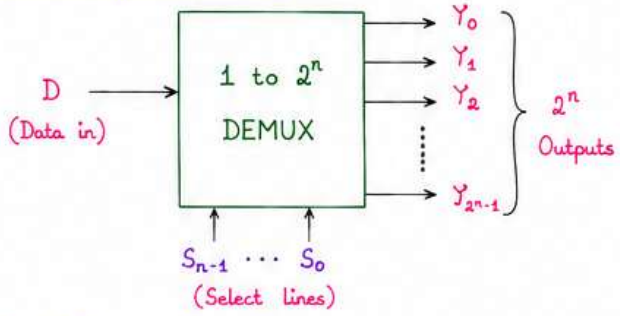
Introduction :

→ A Demultiplexer is a combinational circuit that takes one input and routes it to one of the 2^n outputs based on select lines.

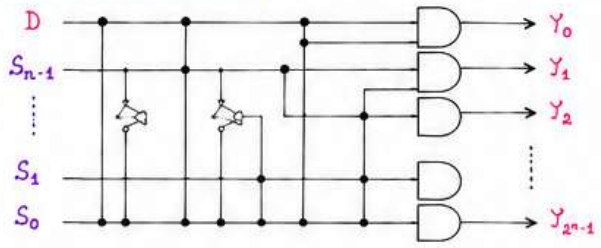
→ It is the reverse operation of Multiplexer.

→ It has 1 data input (D), n select lines ($S_{n-1} \cdots S_0$) and 2^n outputs (Y_0 to Y_{2^n-1}).

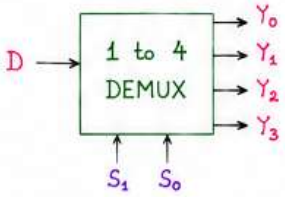
Block Diagram (General)



Circuit Diagram (General)



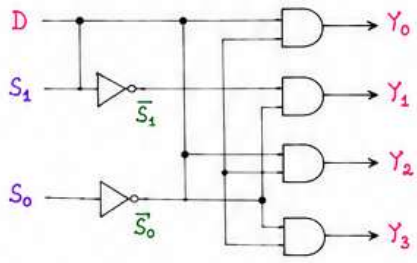
Example : 1 to 4 DEMUX (n = 2)



Truth Table

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

Logic Circuit (1 to 4 DEMUX)



Equations :

$Y_0 = D \bar{S}_1 \bar{S}_0$

$Y_1 = D \bar{S}_1 S_0$

$Y_2 = D S_1 \bar{S}_0$

$Y_3 = D S_1 S_0$

Advantages

- Simple and easy to implement.
- Expands one data line into multiple lines.
- Useful in data distribution, memory decoding, etc.
- Can be designed using AND gates only.

Disadvantages

- Number of outputs grows exponentially (2^n).
- More hardware for large n.
- Not suitable for very high speed distribution.

2. De MUX - 1 x 2

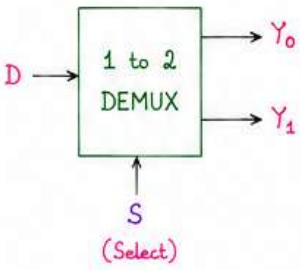
Introduction :

→ 1 to 2 Demultiplexer has one data input (D), one select line (S) and two outputs (Y_0, Y_1).

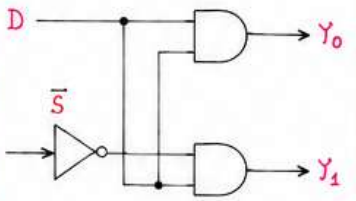
→ When S = 0, input D appears at Y_0.

→ When S = 1, input D appears at Y_1.

Block Diagram



Circuit Diagram



Truth Table

S	Y_0	Y_1
0	D	0
1	0	D

K-Map (for each output)

$Y_0 = D\bar{S}$

S \	0	1
S	1	0

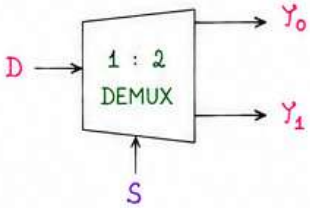
$Y_0 = D\bar{S}$

$Y_1 = DS$

S \	0	1
1	0	1

$Y_1 = DS$

Symbol



Equations :

$Y_0 = D\bar{S}$

$Y_1 = DS$

Key Points (1 x 2 De MUX)

- If S = 0 → Y_0 = D, Y_1 = 0
- If S = 1 → Y_0 = 0, Y_1 = D
- Only two AND gates and one inverter required.
- It is the basic building block for higher order De MUX.

Real World Applications

- Serial data to two channel distribution.
- Memory chip select decoding.
- Data routing in communication systems.
- Time division data demultiplexing.

Interview / Viva Point ?

Q. What is the difference between MUX and De MUX ?

Ans: MUX selects one input and forwards it to one output. De MUX takes one input and routes it to one of many outputs.

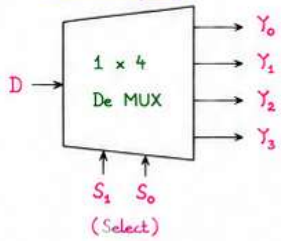
DEMULTIPLEXER (De MUX) - 1 x 4 and 1 x 8

1. De MUX - 1 x 4

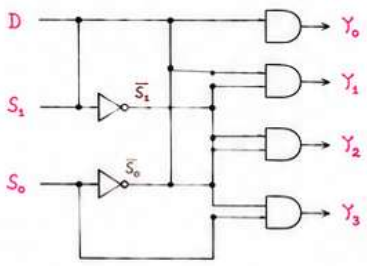
Introduction :

A 1 x 4 Demultiplexer has one data input (D), two select lines (S_1, S_0) and four outputs (Y_0 - Y_3).
Only one output is active (D) at a time depending on the select lines.

Block Diagram



Circuit Diagram



Truth Table

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

K-Map (for each output)

$Y_0 = D\,\overline{S_1}\,\overline{S_0}$

S_1 \ S_0	0	1
0	D	0
1	0	0

$Y_1 = D\,\overline{S_1}\,S_0$

S_1 \ S_0	0	1
0	0	D
1	0	0

$Y_2 = D\,S_1\,\overline{S_0}$

S_1 \ S_0	0	1
0	0	0
1	D	0

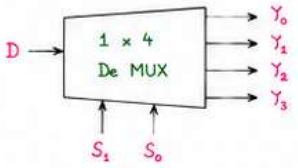
$Y_3 = D\,S_1\,S_0$

S_1 \ S_0	0	1
0	0	0
1	0	D

Equations

$Y_0 = D\,\overline{S_1}\,\overline{S_0}$
$Y_1 = D\,\overline{S_1}\,S_0$
$Y_2 = D\,S_1\,\overline{S_0}$
$Y_3 = D\,S_1\,S_0$

Symbol



Example :

If $S_1 = 1$, $S_0 = 0$ and $D = 1$
Then $Y_2 = D = 1$ and
$Y_0 = Y_1 = Y_3 = 0$.

Advantages

- Simple and easy to implement.
- Used for data distribution.
- Requires less hardware than higher order De MUX.
- Widely used in digital systems.

Disadvantages

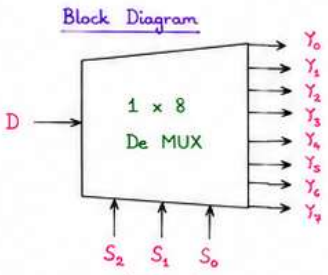
- More outputs as order increases (2^n).
- Fan-out increases.
- Delay increases with number of levels for high order.

2. De MUX - 1 x 8

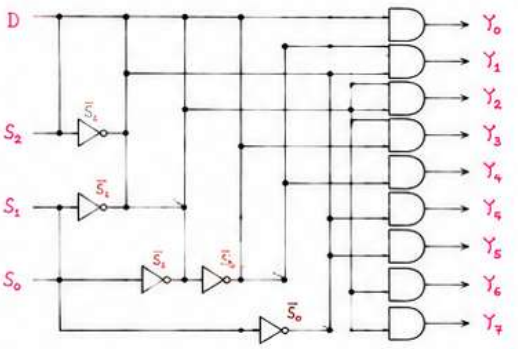
Introduction :

A 1 x 8 Demultiplexer has one data input (D), three select lines (S_2, S_1, S_0) and eight outputs (Y_0 - Y_7).

Block Diagram



Circuit Diagram



Truth Table

S_2	S_1	S_0	Active Output
0	0	0	Y_0
0	0	1	Y_1
0	1	0	Y_2
0	1	1	Y_3
1	0	0	Y_4
1	0	1	Y_5
1	1	0	Y_6
1	1	1	Y_7

K-Map (for each output)

$Y_0 = D\,\overline{S_2}\,\overline{S_1}\,\overline{S_0}$

S_2 \ S_1S_0	00	01	11	10
0	D	0	0	0
1	0	0	0	0

$Y_1 = D\,\overline{S_2}\,\overline{S_1}\,S_0$

S_2 \ S_1S_0	00	01	11	10
0	0	D	0	0
1	0	0	0	0

$Y_2 = D\,\overline{S_2}\,S_1\,\overline{S_0}$

S_2 \ S_1S_0	00	01	11	10
0	0	D	0	0
1	0	0	0	0

$Y_3 = D\,\overline{S_2}\,S_1\,S_0$

S_2 \ S_1S_0	00	01	11	10
0	0	D	0	0
1	0	0	0	0

$Y_4 = D\,S_2\,\overline{S_1}\,\overline{S_0}$

S_2 \ S_1S_0	00	01	11	10
0	0	D	0	0
1	0	0	0	0

$Y_5 = D\,S_2\,\overline{S_1}\,S_0$

S_2 \ S_1S_0	00	01	11	10
0	0	0	0	0
1	0	D	0	0

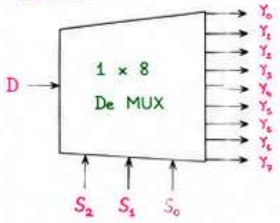
$Y_6 = D\,S_2\,S_1\,\overline{S_0}$

S_2 \ S_1S_0	00	01	11	10
0	0	0	0	0
1	0	0	0	D

$Y_7 = D\,S_2\,S_1\,S_0$

S_2 \ S_1S_0	00	01	11	10
0	0	0	0	0
1	0	0	0	D

Symbol



Equations

$Y_0 = D\,\overline{S_2}\,\overline{S_1}\,\overline{S_0}$
$Y_1 = D\,\overline{S_2}\,\overline{S_1}\,S_0$
$Y_2 = D\,\overline{S_2}\,S_1\,\overline{S_0}$
$Y_3 = D\,\overline{S_2}\,S_1\,S_0$
$Y_4 = D\,S_2\,\overline{S_1}\,\overline{S_0}$
$Y_5 = D\,S_2\,\overline{S_1}\,S_0$
$Y_6 = D\,S_2\,S_1\,\overline{S_0}$
$Y_7 = D\,S_2\,S_1\,S_0$

Example :

If $S_2 = 1$, $S_1 = 0$, $S_0 = 1$ and $D = 1$
Then $Y_5 = D = 1$ and all other outputs = 0.

Applications

- Data distribution in communication.
- Memory bank selection.
- High speed digital systems.

Interview / Viva Point

Q. What is the difference between MUX and De MUX ?
Ans. MUX selects one of many inputs and forwards it to one output.
De MUX takes one input and routes it to one of many outputs.

Key Points (Both 1 x 4 and 1 x 8 De MUX)

- Demultiplexer routes one input to one of the 2^n outputs based on select lines.
- Only one output is active at a time.
- Dataout equations are of the form : $Y_i = D \times$ (select line terms).

DECODER - INTRODUCTION & 2 : 4 DECODER

1. DECODER

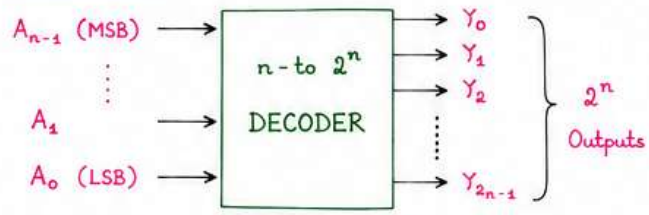
Introduction :

→ A Decoder is a combinational circuit that converts an n-bit binary input into one of the 2^n output lines.

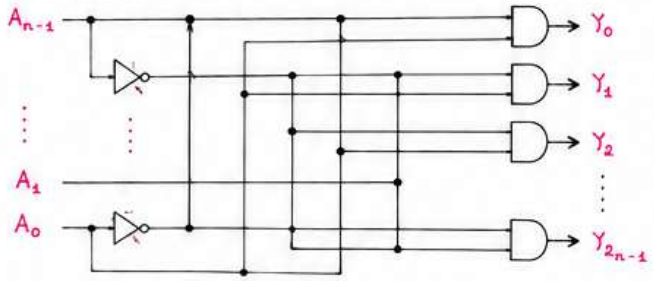
→ Only one output is HIGH (1) at a time corresponding to the binary input.

→ It is also called "n-to-2^n Decoder".

Block Diagram (General n-to-2^n Decoder)



Circuit Diagram (General using AND gates)



K-Map (for each output)

Each output is 1 for only one binary combination. So, it reduces to a single minterm.

Example : $Y_0 = \bar{A}_1 \bar{A}_0$

A_1 \ A_0	0	1
0	(1)	0
1	0	0

Example : $Y_3 = A_1 A_0$

A_1 \ A_0	0	1
0	0	0
1	0	(1)

Types of Decoders

★ 1-to-2^n Decoder → One input line, n select lines, 2^n outputs.

★ BCD to Decimal Decoder → 4-to-10 Decoder.

★ Seven Segment Decoder → BCD input to 7 outputs (a-g).

★ Priority Decoder → Multiple inputs, encoded output with priority.

Advantages

★ Simple and reliable.

★ One output active → easy decoding.

★ Used in memory decoding, address decoding, display drivers etc.

Disadvantages

★ Number of outputs increases exponentially (2^n).

★ Requires more hardware for large n.

★ Not suitable for very high speed if n is large.

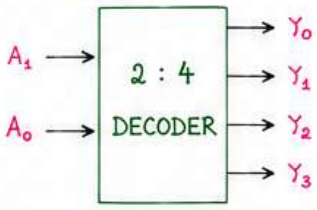
2. DECODER - 2 : 4

Introduction :

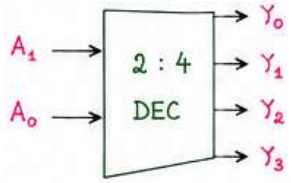
→ A 2 : 4 Decoder has 2 input lines (A_1, A_0) and 4 outputs ($Y_0 - Y_3$).

→ When $A_1\ A_0$ = 00 → $Y_0 = 1$
01 → $Y_1 = 1$
10 → $Y_2 = 1$
11 → $Y_3 = 1$

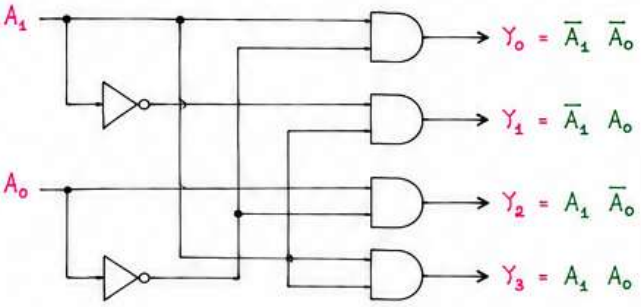
Block Diagram



Symbol



Circuit Diagram (using AND gates & inverters)



Truth Table

A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

K-Map for each output

$Y_0 = \bar{A}_1 \bar{A}_0$

A_1 \ A_0	0	1
1	(1)	0
0	0	0

$Y_1 = \bar{A}_1 A_0$

A_1 \ A_0	0	1
1	0	0
0	0	(1)

$Y_2 = A_1 \bar{A}_0$

A_1 \ A_0	0	1
1	0	0
0	(1)	0

$Y_3 = A_1 A_0$

A_1 \ A_0	0	1
1	0	0
0	0	(1)

Equations

$Y_0 = \bar{A}_1 \bar{A}_0$

$Y_1 = \bar{A}_1 A_0$

$Y_2 = A_1 \bar{A}_0$

$Y_3 = A_1 A_0$

Example :

If $A_1 = 1$, $A_0 = 0$

Binary (1 0)

→ Only $Y_2 = 1$

All other outputs = 0.

Applications

★ Memory address decoding.

★ Data distribution.

★ Seven segment display drivers.

★ Demultiplexing in networks.

Key Points

★ Exactly one output is HIGH at any time.

★ Number of outputs = 2^n.

★ It is the reverse of MUX.

Author : Konda Tharun Vivekananda

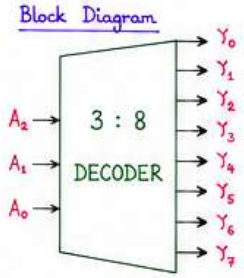
DECODER – 3 : 8 and 4 : 16

1. DECODER – 3 : 8

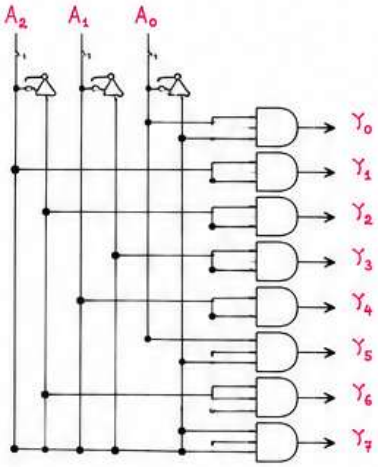
Introduction :

- → 3 to 8 Decoder converts a 3-bit binary input into one of the eight outputs.
- → Eight outputs ($Y_0 - Y_7$), only one output is HIGH (1) at a time.
- → Used in memory decoding, data routing, etc.

Block Diagram



Circuit Diagram (using AND gates & inverters)



Truth Table

A_2	A_1	A_0	Output Active
0	0	0	Y_0
0	0	1	Y_1
0	1	0	Y_2
0	1	1	Y_3
1	0	0	Y_4
1	0	1	Y_5
1	1	0	Y_6
1	1	1	Y_7

K-Map for each output

$Y_0 = \bar{A}_2\,\bar{A}_1\,\bar{A}_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	0	0	0	0

$Y_1 = \bar{A}_2\,\bar{A}_1\,A_0$

A_2 \ A_1A_0	00	01	11	10
0	0	1	0	0
1	0	0	0	0

$Y_2 = \bar{A}_2\,A_1\,\bar{A}_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	0	0	0	0

$Y_3 = \bar{A}_2\,A_1\,A_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	0	0	0	0

$Y_4 = A_2\,\bar{A}_1\,\bar{A}_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	1	0	0	0

$Y_5 = A_2\,\bar{A}_1\,A_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	0	0	0	0

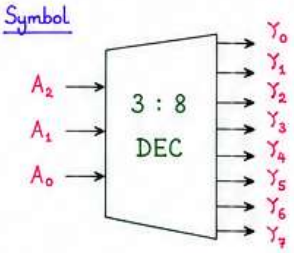
$Y_6 = A_2\,A_1\,\bar{A}_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	1	0	0	0

$Y_7 = A_2\,A_1\,A_0$

A_2 \ A_1A_0	00	01	11	10
0	0	0	0	0
1	0	0	0	1

Symbol



Equations :

$Y_0 = \bar{A}_2\,\bar{A}_1\,\bar{A}_0$
$Y_1 = \bar{A}_2\,\bar{A}_1\,A_0$
$Y_2 = \bar{A}_2\,A_1\,\bar{A}_0$
$Y_3 = \bar{A}_2\,A_1\,A_0$
$Y_4 = A_2\,\bar{A}_1\,\bar{A}_0$
$Y_5 = A_2\,\bar{A}_1\,A_0$
$Y_6 = A_2\,A_1\,\bar{A}_0$
$Y_7 = A_2\,A_1\,A_0$

Advantages

- Simple and reliable.
- Only one active output.
- Widely used in address decoding, multiplexing, chip select, etc.
- Easy to implement using gates.

Disadvantages

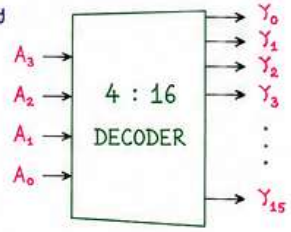
- Number of outputs = 2^n (increases exponentially).
- More hardware for large n.
- Not suitable for very high speed when n is large.

2. DECODER – 4 : 16

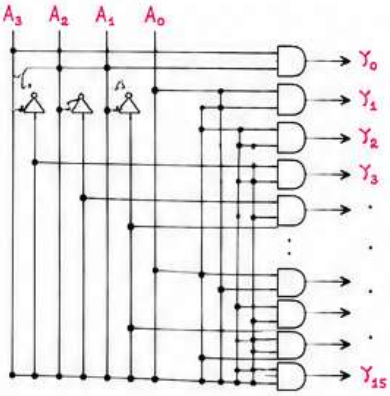
Introduction :

- → 4 to 16 Decoder converts a 4-bit binary input into one of sixteen outputs.
- → Sixteen outputs ($Y_0 - Y_{15}$), only one output is HIGH (1) at a time.
- → Commonly used in memory systems.

Block Diagram



Circuit Diagram (using AND gates & inverters)



Truth Table

A_3	A_2	A_1	A_0	Active Output
0	0	0	0	Y_0
0	0	0	1	Y_1
0	0	1	0	Y_2
0	0	1	1	Y_3
.	.	.	.	.
1	1	1	1	Y_{15}

K-Map for each output

$Y_0 = \bar{A}_3\,\bar{A}_2\,\bar{A}_1\,\bar{A}_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	0	0	0	0
10	0	0	0	0

$Y_1 = \bar{A}_3\,\bar{A}_2\,\bar{A}_1\,A_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

$Y_2 = \bar{A}_3\,\bar{A}_2\,A_1\,\bar{A}_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	0	0	0
10	0	0	0	0

$Y_3 = \bar{A}_3\,\bar{A}_2\,A_1\,A_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	0	0	0
10	0	0	0	0

. . . . . .

$Y_{12} = A_3\,A_2\,\bar{A}_1\,\bar{A}_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	1	0

$Y_{13} = A_3\,A_2\,\bar{A}_1\,A_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	1	0

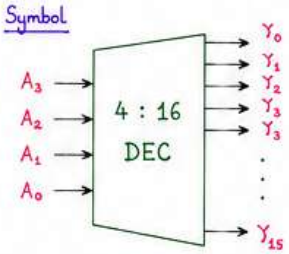
$Y_{14} = A_3\,A_2\,A_1\,\bar{A}_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	1	0

$Y_{15} = A_3\,A_2\,A_1\,A_0$

A_3 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	1	0
10	0	0	1	0

Symbol



Equations :

General form : Y_i = product (minterm i)

e.g. $Y_5 = \bar{A}_3\,A_2\,\bar{A}_1\,A_0$

$Y_{14} = A_3\,A_2\,A_1\,\bar{A}_0$

. .

$Y_{15} = A_3\,A_2\,A_1\,A_0$

Applications

- Memory address decoding.
- Chip select generation.
- Data routing and distribution.
- Digital systems and microprocessors.

Key Points (Both Decoders)

- n input → 2^n outputs.
- Exactly one output is HIGH (1).
- Outputs are mutually exclusive.
- Implemented using inverters and AND gates.

Author : Konda Tharun Vivekananda

ENCODER

1. ENCODER

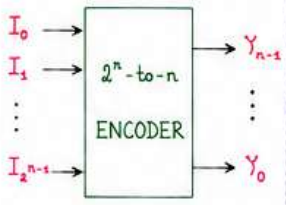
Introduction :

→ Encoder is a combinational circuit that converts one of the 2^n input lines which is HIGH (1) into an n-bit binary code.

→ Only one input is assumed to be HIGH at a time (valid input).

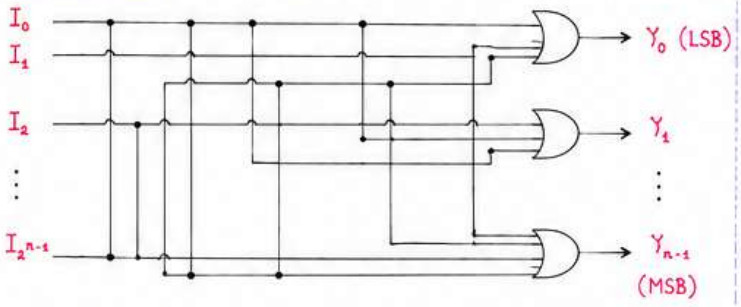
→ Used for data conversion, priority coding, data compression, etc.

Block Diagram (General)



(n-bit code output)

Circuit Diagram (General using OR gates)



Truth Table (General 2^n-to-n Encoder)

Inputs					Outputs				Decimal	Code
I_{2^n-1}	I_{2^n-2}	...	I_1	I_0	Y_{n-1}	...	Y_1	Y_0		
0	0	...	0	1	0	...	0	0	0	000...0
0	0	...	1	0	0	...	0	1	1	000...1
⋮	0	...	0	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1	0	...	0	0	1	...	1	1	2^n-1	111...1

(Only one input is HIGH (1) at a time)

Types of Encoders :

★ 1. 2-to-1 Encoder → Two inputs to one bit.
★ 2. 4-to-2 Encoder → Four inputs to two bits.
★ 3. 8-to-3 Encoder → Eight inputs to three bits.
★ 4. 16-to-4 Encoder → Sixteen inputs to four bits.
★ 5. Priority Encoder → When multiple inputs HIGH, highest priority input is encoded.

Advantages

★ Reduces number of lines from 2^n inputs to n output lines.
★ Simplifies data handling.
★ Saves hardware, space and cost.
★ Useful in communication & digital systems.

Disadvantages

★ Only one input can be active at a time.
★ If multiple inputs HIGH, output becomes invalid (except in priority encoder).
★ Increase in propagation delay for higher order encoders.

Applications

★ Keyboard encoding (key-press to code).
★ Decimal to binary code conversion.
★ Data compression in communication systems.
★ Priority interrupt systems using priority encoder.
★ Used in arithmetic and control circuits.

2. ENCODER – 4 : 2

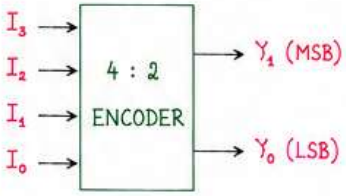
Introduction :

→ 4 : 2 Encoder has four inputs (I_3 - I_0) and two outputs (Y_1, Y_0).

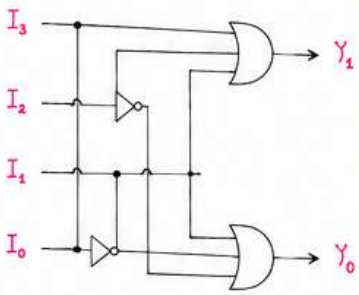
→ If $I_3 = 1$ → Code = 00
$I_2 = 1$ → Code = 01
$I_1 = 1$ → Code = 10
$I_0 = 1$ → Code = 11

→ Only one input is HIGH at a time.

Block Diagram



Circuit Diagram (using OR gates)



Truth Table

Inputs				Outputs	
I_3	I_2	I_1	I_0	Y_1	Y_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	2
1	0	0	0	1	3

(Only one input HIGH (1) at a time)

K-Map for each output

Y_1 (MSB) = $I_3 + I_2$

I_2 \ I_3	0	1
0	0	1
1	1	1

Y_0 (LSB) = $I_1 + \overline{I_3}$

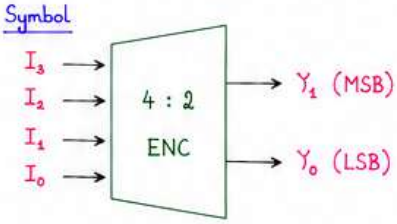
I_1 \ I_3	0	1
0	0	1
1	1	0

Equations

$Y_1 = I_3 + I_2$

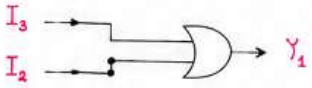
$Y_0 = \overline{I_3} + I_1$

Symbol

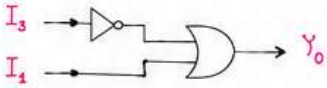


Implementation using basic gates

$Y_1 = I_3 + I_2$



$Y_0 = I_1 + \overline{I_3}$



Key Points (4 : 2 Encoder)

★ Converts 4 input lines to 2-bit binary code.
★ Only one input should be HIGH at a time.
★ Y_1 is MSB and Y_0 is LSB.
★ Logic is implemented using 2 OR gates and 1 inverter.

Applications (4 : 2 Encoder)

★ Keyboard encoder for 4 keys.
★ Decimal to binary code conversion (0-3).
★ Data compression in small systems.
★ Use in interrupt control and address encoding.

Interview / Viva Point

Q. Why only one input HIGH in Encoder ?

Ans: If more than one input is HIGH, outputs become ambiguous and do not represent a valid binary code.

Author : Konda Tharun Vivekananda

ENCODER - 8 : 3 and PRIORITY ENCODER - 4 : 2

1. ENCODER - 8 : 3

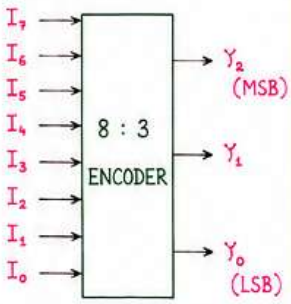
Introduction :

→ 8 : 3 Encoder converts 1 of the eight input lines ($I_7 - I_0$) into 3-bit binary code ($Y_2\ Y_1\ Y_0$).

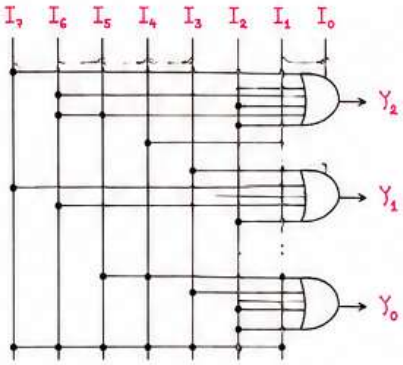
→ Only one input is HIGH (1) at a time.

→ Used in data compression, keyboard encoding, etc.

Block Diagram



Circuit Diagram (using 8 gates)



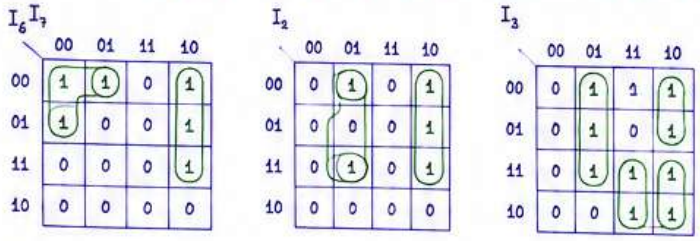
Truth Table

Inputs					Outputs		
I_7	I_6	...	I_1	I_0	Y_2	Y_1	Y_0
1	0	...	0	0	1	1	1
0	1	...	0	0	1	1	0
0	0	...	1	0	1	0	1
0	0	...	0	1	1	0	0
⋮	⋮	...	⋮	⋮	⋮	⋮	⋮
0	0	...	1	0	0	0	1
0	0	...	0	0	0	0	0

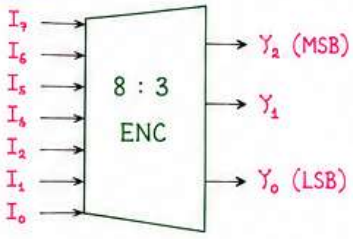
(Only one input HIGH (1) at a time)

K-Map for each output

Y_2 (MSB) = $I_4 + I_5 + I_6 + I_7$ $\quad$ $Y_1 = I_2 + I_3 + I_6 + I_7$ $\quad$ Y_0 (LSB) = $I_1 + I_3 + I_5 + I_7$



Symbol



Equations :

$Y_2 = I_4 + I_5 + I_6 + I_7$

$Y_1 = I_2 + I_3 + I_6 + I_7$

$Y_0 = I_1 + I_3 + I_5 + I_7$

Advantages

- Reduces 2^n input lines to n-bit code.
- Simplifies data handling.
- Saves hardware and cost.
- Useful for data compression and coding.

Disadvantages

- Only one input can be HIGH at a time.
- Output becomes invalid if multiple inputs are HIGH.

Applications

- Keyboard encoding (key code generation).
- Data compression.
- Telecommunication systems.
- Barcode and BCD encoding.



2. PRIORITY ENCODER - 4 : 2

Introduction :

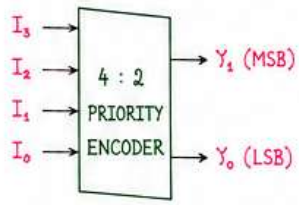
→ 4 : 2 Priority Encoder has 4 inputs ($I_3 - I_0$) and 2 outputs (Y_1, Y_0).

→ If more than one input is HIGH, the input with highest priority (highest index) is encoded.

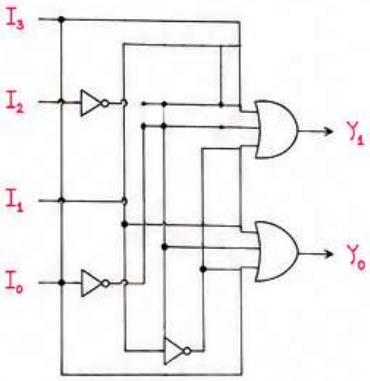
→ Priority order : $I_3 > I_2 > I_1 > I_0$

→ Used in interrupt handling, CPU systems, etc.

Block Diagram



Circuit Diagram (using OR gates & inverters)



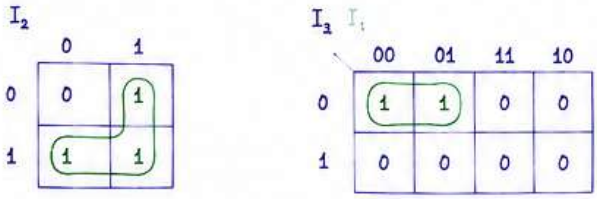
Truth Table

Inputs				Outputs		Encoded Input
I_3	I_2	I_1	I_0	Y_1	Y_0	
0	0	0	0	0	0	None
0	0	0	1	0	0	I_0
0	0	1	0	0	1	I_1
0	1	0	0	1	0	I_2
1	0	0	0	1	1	I_3
0	1	0	1	1	0	I_2
1	0	1	1	1	1	I_3
1	1	1	1	1	1	I_3

(Highest priority input is encoded)

K-Map for each output

Y_1 (MSB) = $I_3 + I_2$ $\quad$ Y_0 (LSB) = $\bar{I}_3 I_2 + \bar{I}_3 \bar{I}_2 I_1 + \bar{I}_3 \bar{I}_2 \bar{I}_1 I_0$

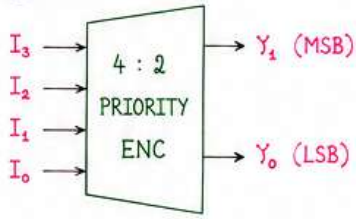


Equations :

$Y_1 = I_3 + I_2$

$Y_0 = \bar{I}_3 I_2 + \bar{I}_3 \bar{I}_2 I_1 + \bar{I}_3 \bar{I}_2 \bar{I}_1 I_0$

Symbol



Advantages

- Handles multiple HIGH inputs.
- Encodes highest priority input.
- Useful in interrupt systems and control circuits.

Disadvantages

- More hardware than simple encoder.
- Propagation delay increases.
- Logic is more complex.

Applications

- Interrupt processing in CPUs.
- Keyboard / key priority detection.
- Data arbitration in communication systems.
- Alarm and fault detection systems.

Key Points

- Encoder → assumes only one input HIGH.
- Priority Encoder → handles multiple HIGH inputs and encodes highest priority one.

COMPARATOR

1. COMPARATOR (GENERAL)

Introduction :

→ Comparator is a combinational circuit that compares two binary numbers or logic levels.

→ It has three outputs :

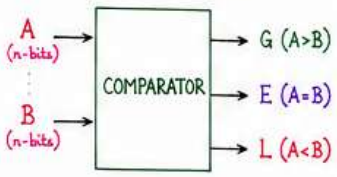
A > B → G (Greater)

A = B → E (Equal)

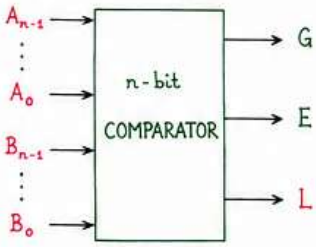
A < B → L (Less)

→ Used in decision making, control, sorting, data processing etc.

Block Diagram



General Circuit Diagram



Truth Table (2 numbers)

A	B	G (A>B)	E (A=B)	L (A<B)
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

Advantages

- Simple and efficient.
- High speed operation.
- Direct comparison of two numbers.
- Used in many digital systems.

Disadvantages

- For n-bit comparison, hardware increases.
- Propagation delay increases with number of bits.
- More power for large n.

Applications

- Magnitude comparison in ALU.
- Data sorting and sequencing.
- Digital voltmeters, ADCs.
- Control and decision making.
- Error detection in communication.

Types of Comparators

Type	Description	Outputs	Remarks
1. Magnitude Comparator	Compares two multi-bit numbers.	A>B, A=B, A<B	Most widely used.
2. Equality Comparator	Checks whether A = B or not.	A=B	Used in matching.
3. Difference Comparator	Indicates if A ≠ B.	A≠B	Used in error detect.
4. 1-bit Comparator	Compares two single bits.	A>B, A=B, A<B	Basic building block.

Key Points

- Comparator compares two binary numbers and gives 3 outputs.
- Comparator is the main building block of ALU.
- For multi-bit numbers, comparison is done from MSB to LSB.
- Equality output is HIGH only when all bits are same.
- Different types are used for different purposes.

2. COMPARATOR – 1 BIT

Introduction :

→ 1-bit Comparator compares two single bits A and B.

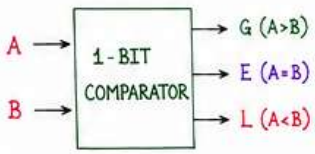
→ It produces three outputs :

A > B → G

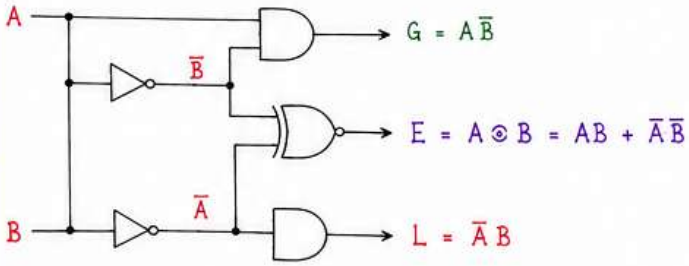
A = B → E

A < B → L

Block Diagram



Circuit Diagram (using basic gates)



Truth Table

A	B	G (A>B)	E (A=B)	L (A<B)
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

K-Map for each output

$G = A\overline{B}$

A \	0	1
0	0	0
1	(1)	0

$E = AB + \overline{A}\,\overline{B}$

A \	0	1
0	(1)	0
1	0	(1)

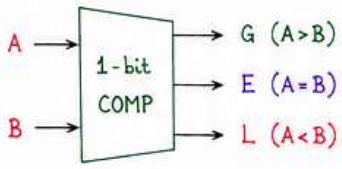
Equations :

$G = A\overline{B}$

$E = AB + \overline{A}\,\overline{B}$ (XNOR)

$L = \overline{A}B$

Symbol



Advantages

- Very simple circuit.
- Uses minimal number of gates.
- High speed.
- Low cost and power.

Disadvantages

- Only single bit comparison.
- Not suitable for multi-bit numbers directly.

Summary / Key Points

- Comparator compares two binary numbers and indicates whether A is greater, equal or less than B.
- 1-bit comparator is the basic building block.
- Multi-bit comparators are built using cascading of 1-bit comparators or gate logic.
- Used widely in ALU, calculators, processors and control systems.

Interview / Viva Point

Q. What are the outputs of a comparator ?

Ans: Greater (A > B), Equal (A = B) and Less (A < B). Only one of them is HIGH at a time.

Author : Konda Tharun Vivekananda

COMPARATOR - 2 BIT and 4 BIT

1. COMPARATOR - 2 BIT

Introduction :

→ 2-bit comparator compares two numbers $A(A_1A_0)$ and $B(B_1B_0)$.

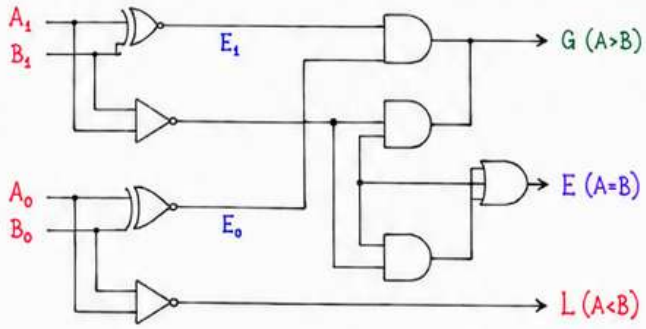
→ Three outputs are produced :

G (A>B) , E (A=B) , L (A<B)

Block Diagram



Circuit Diagram (using basic gates)



Truth Table

A_1 A_0	B_1 B_0	G(A>B)	E(A=B)	L(A<B)
0 0	0 0	0	1	0
0 0	0 1	0	0	1
0 0	1 0	0	0	1
: :	:	:	:	:
1 1	1 0	1	0	0
1 1	1 1	0	1	0

K-Map for each output

G (A>B)

B_1B_0 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	1	1	0
10	0	1	1	1

E (A<B)

B_1B_0 \ A_1A_0	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

L (A<B)

B_1B_0 \ A_1A_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	0
10	1	1	0	0

Simplified Equations

$G = A_1\overline{B_1} + (A_1 \oplus B_1)\,\overline{A_0}\,\overline{B_0}$

$E = (A_1 \oplus B_1)\,\overline{(A_0 \oplus B_0)}$

$L = \overline{A_1}\,B_1 + (A_1 \oplus B_1)\,A_0\,\overline{B_0}$

Advantages

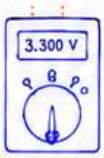
- Simple and fast.
- Requires less hardware.
- Widely used in CPUs, ALU, processors.

Disadvantages

- More outputs → more wiring for larger bit widths.
- Delay increases with cascade in higher bits.

Applications

- Arithmetic units (ALU) for comparison.
- Digital voltmeter, ADCs.
- Data sorting and data routing.
- Control & decision making in digital systems.



2. NORMAL COMPARATOR - 4 BIT

Introduction :

→ 4-bit normal comparator compares $A(A_3A_2A_1A_0)$ and $B(B_3B_2B_1B_0)$.

→ Outputs : G (A>B) , E (A=B) , L (A<B)

How it works ?

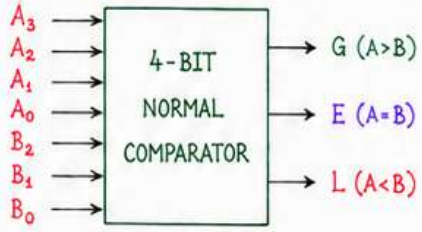
Start comparsion from MSB (A_3 with B_3).

If equal → go to next lower bit.

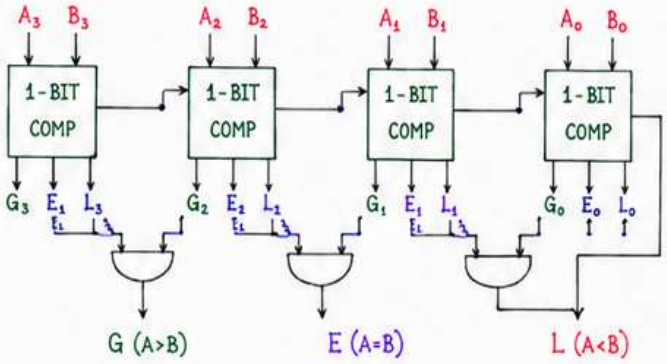
If not equal, decision is taken at that bit.

Used cascading of 1-bit comparators.

Block Diagram



Circuit Diagram (using cascade of 1-bit comparators)



Truth Table (general idea)

A B	G	E	L
A > B	1	0	0
A = B	0	1	0
A < B	0	0	1

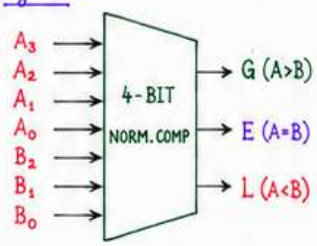
K-Map / Equations (general form)

$G = A_3\overline{B_3} + (A_3 \oplus B_3)\,A_2\overline{B_2} + (A_3 \oplus B_3)\,(A_2 \oplus B_2)\,.\,A_1\overline{B_1} + (A_3 \oplus B_3)\,(A_2 \oplus B_2)\,(A_1 \oplus B_1)\,A_0\overline{B_0}$

$E = (A_3 \oplus B_3)\,(A_2 \oplus B_2)\,(A_1 \oplus B_1)\,\overline{(A_0 \oplus B_0)}$

$L = \overline{A_3}\,\overline{B_3} + (A_3 \oplus B_3)\,.\,A_2\overline{B_2} + (A_3 \oplus B_3)\,(A_2 \oplus B_2)\,.\,\overline{A_1}\,\overline{B_1} + (A_3 \oplus B_3)\,(A_2 \oplus B_2)\,(A_1 \oplus B_1)\,A_0\overline{B_0}$

Symbol



IC Used

74LS85 → 4-bit comparator (comparator IC)

Pin details (74LS85) :

- Inputs : A_3, A_2, A_1, A_0, B_2, B_1, B_0
- Cascading inputs : I_{GT}, I_{EQ}, I_{LT}
- Outputs : A > B , A = B , A < B

Advantages

- Handles 4-bit numbers.
- Cascadable for higher bits.
- Fast and reliable.
- Available as IC (74LS85).

Disadvantages

- More hardware than 2-bit comparator.
- Power consumption increases with number of bits.

Applications

- CPU / ALU for arithmetic operations.
- Digital systems for address comparions.
- Data sorting, searching algorithms.
- Used in calculators, control systems.



Key Point



Comparator gives one of the three mutually exclusive outputs G, E or L. any time.

Author : Konda Tharun Vivekananda.

8 BIT COMPARATOR USING 4-BIT COMPARATOR

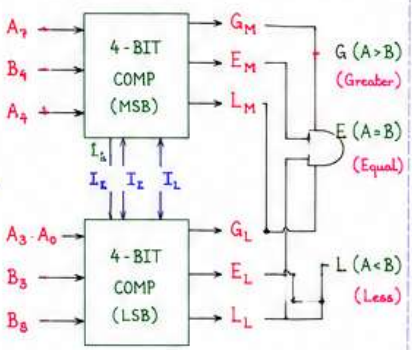
1. 8-BIT COMPARATOR USING 4-BIT COMPARATOR

Introduction :

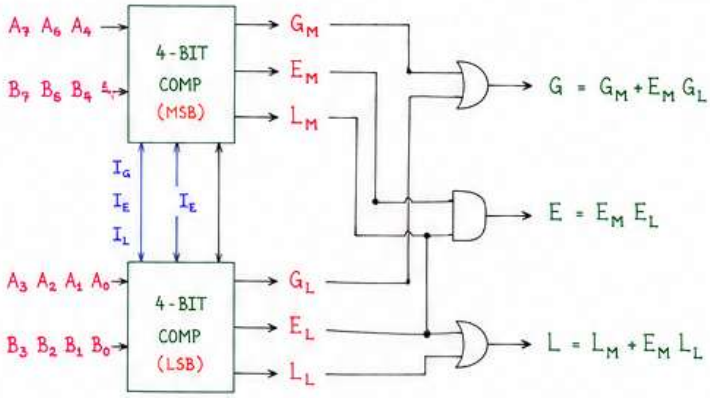
→ 8-bit comparator compares two 8-bit numbers $A_7 \ldots A_0$ and $B_7 \ldots B_0$ and gives three outputs G (A>B), E (A=B) and L (A<B).

→ Implemented using cascade of two 4-bit magnitude comparators.

Block Diagram



Circuit Diagram (using cascade of 4-bit comparators)



Truth Table (8-bit comparator using 4-bit blocks)

Condition	MSB Output			LSB Output			Final Output		
	G_M	E_M	L_M	G_L	E_L	L_L	G	E	L
$A_7 \ldots A_4 > B_7 \ldots B_4$	1	0	0	×	×	×	1	0	0
$A_7 \ldots A_4 = B_7 \ldots B_4$ and $A_3 \ldots A_0 > B_3 \ldots B_0$	0	1	0	1	0	0	1	0	0
$A_7 \ldots A_4 = B_7 \ldots B_4$ and $A_3 \ldots A_0 = B_3 \ldots B_0$	0	1	0	0	1	0	0	1	0
$A_7 \ldots A_4 = B_7 \ldots B_4$ and $A_3 \ldots A_0 < B_3 \ldots B_0$	0	1	0	0	0	1	0	0	1
$A_7 \ldots A_4 < B_7 \ldots B_4$	0	0	1	×	×	×	0	0	1

(× = don't care)

Cascade Inputs

$I_G = G_M$ $\quad I_E = E_M$ $\quad I_L = L_M$

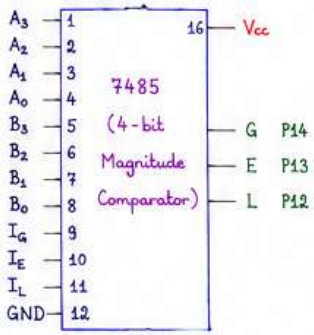
Equations (8-bit comparator)

$$G = G_M + E_M G_L$$

$$E = E_M E_L$$

$$L = L_M + E_M L_L$$

IC USED



Use two 7485 ICs for 8-bit comparator.

Advantages

- Simple and modular design.
- Uses standard ICs.
- Fast comparison.
- Easy to cascade for higher bits.

Disadvantages

- More hardware (two ICs).
- Propagation delay increases with cascading.
- Power consumption more.

Applications

- Used in CPU / ALU.
- Data sorting & searching.
- Digital voltmeters.
- Decision making & control.

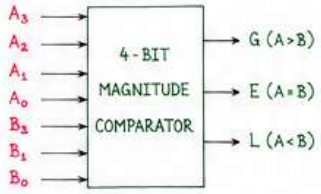
2. MAGNITUDE COMPARATOR - 4 BIT

Introduction :

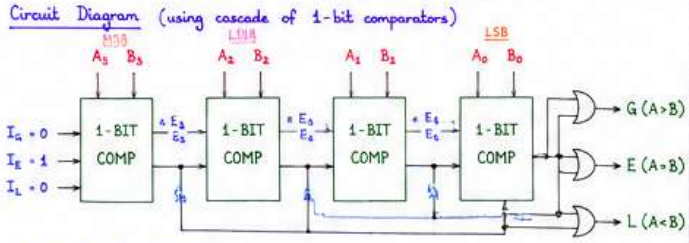
→ 4-bit Magnitude Comparator compares two 4-bit binary numbers $A_3 A_2 A_1 A_0$ and $B_3 B_2 B_1 B_0$.

→ Outputs are : G (A>B), E (A=B), L (A<B).

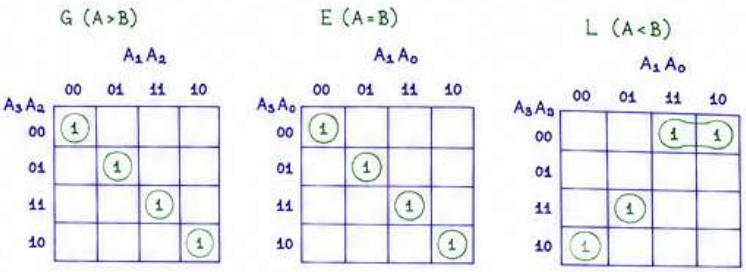
Block Diagram



Circuit Diagram (using cascade of 1-bit comparators)



K-Map for each output



IC USED

☆ 7485 → 4-bit magnitude comparator.

★ Pin details given in above 8-bit section.

★ Can be cascaded by providing I_G, I_E, I_L.

Truth Table (4-bit comparator)

Inputs		Outputs		
A_3 A_2 A_1 A_0	B_3 B_2 B_1 B_0	G (A>B)	E (A=B)	L (A<B)
A > B	× × × × ×	1	0	0
A = B	× × × × (Same)	0	1	0
A < B	× × × ×	0	0	1

(× = don't care)

Advantages

- Fast comparison of 4-bit numbers.
- Only 3 outputs.
- Easy to implement.
- Standard IC available (7485).

Disadvantages

- Limited to 4 bits.
- Not suitable for very large word length without cascading.
- Propagation delay increases when cascaded.

Applications

- Comparison in ALU.
- Digital measuring instruments.
- Sorting and searching.
- Control and decision circuits.

Key Points

- Comparator compares two numbers and gives three outputs : G, E, L.
- 8-bit comparator is built using two 4-bit comparators.
- Cascading is done from MSB to LSB.
- 7485 is a widely used magnitude comparator IC.

Interview / Viva Points

Q. What are the outputs of a comparator ?

Ans : Greater (A>B), Equal (A=B) and Less (A<B). Only one output is HIGH at a time.

Author : Konda Tharun Vivekananda

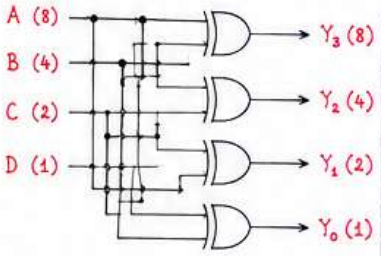
CODE CONVERTORS

1. BCD TO DECIMAL

Introduction :

- → Converts 4-bit BCD code (8421) into equivalent decimal number (0 - 9) in binary form.
- → Invalid BCD codes (10 - 15) are treated as don't cares.

Circuit Diagram



Truth Table

BCD Inputs				Decimal Output				Decimal
A	B	C	D	Y_3	Y_2	Y_1	Y_0	
0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1	1
0	0	1	0	0	0	1	0	2
0	0	1	1	0	0	1	1	3
0	1	0	0	0	1	0	0	4
0	1	0	1	0	1	0	1	5
0	1	1	0	0	1	1	0	6
0	1	1	1	0	1	1	1	7
1	0	0	0	1	0	0	0	8
1	0	0	1	1	0	0	1	9
10 - 15				x	x	x	x	Don't care

Boolean Equations

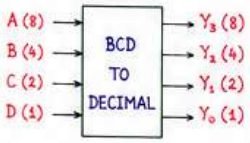
$$Y_3 = A$$

$$Y_2 = \bar{A}B + AC$$

$$Y_1 = \bar{A}C + BD$$

$$Y_0 = \bar{A}D + B\bar{C} + C\bar{D}$$

Symbol



Advantages

- ★ Simple and fast.
- ★ Easy to design with basic gates.

Disadvantages

- ★ Only valid for BCD codes (0-9).
- ★ Not suitable for large coded systems.

Applications

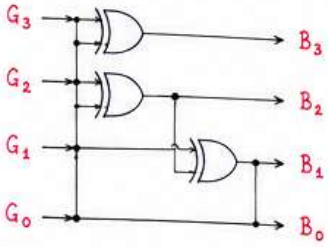
- ★ BCD calculators & digital meters.
- ★ BCD counters.
- ★ Data entry in microcontroller systems.

2. GRAY TO BINARY

Introduction :

- → Gray code has only one bit change between consecutive numbers.
- → Converter changes 4-bit Gray code ($G_3\ G_2\ G_1\ G_0$) to 4-bit binary code ($B_3\ B_2\ B_1\ B_0$).

Circuit Diagram



Truth Table

Gray Inputs				Binary Output			
G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
0	1	0	1	0	1	1	1
0	1	0	0	0	1	1	0
1	1	1	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	1	1	0	0	0
1	1	1	0	1	0	0	1
1	1	1	1	1	1	1	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	0	0	1	1	1	0	0
1	0	0	0	1	1	0	1

Boolean Equations

$$B_3 = G_3$$

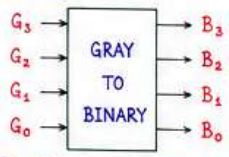
$$B_2 = B_3 \oplus G_2$$

$$B_1 = B_2 \oplus G_1$$

$$B_0 = B_1 \oplus G_0$$

$(\oplus$ = XOR)

Symbol



Advantages

- ★ Only one bit changes between numbers.
- ★ Less error in transmission.
- ★ Easy conversion.

Disadvantages

- ★ Not directly usable in arithmetic circuits.
- ★ Extra hardware required for conversion.

Applications

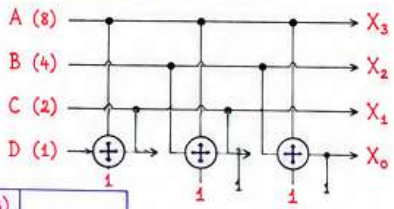
- ★ Rotary encoders.
- ★ Position & motion detection.
- ★ Error reduction in data communication.

3. BCD TO EXCESS-3 (XS-3) CODE

Introduction :

- → Excess-3 code is a self-complementing code.
- → It is obtained by adding $(0011)_2$ to the BCD digit.
- → Useful in computer arithmetic & decimal complement operations.

Circuit Diagram



Truth Table

BCD Inputs				Excess-3 Output (XS-3)				Decimal
A	B	C	D	X_3	X_2	X_1	X_0	
0	0	0	0	0	0	1	1	3
0	0	0	1	0	1	0	0	4
0	0	1	0	0	1	0	1	5
0	0	1	1	0	1	1	0	6
0	1	0	0	0	1	1	1	7
0	1	0	1	1	0	0	0	8
0	1	1	0	1	0	0	1	9
0	1	1	1	1	0	1	0	10 (A)
1	0	0	0	1	0	1	1	11 (B)
1	0	0	1	1	1	0	0	12 (C)
10 - 15				—				Not valid

Boolean Equations

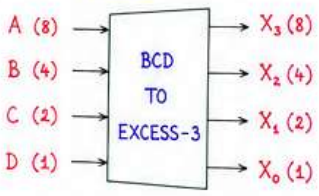
$$X_3 = A$$

$$X_2 = B \oplus 1 = \bar{B}$$

$$X_1 = C \oplus 1 = \bar{C}$$

$$X_0 = D \oplus 1 = \bar{D}$$

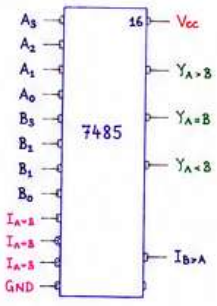
Symbol



IC USED

74LS85 / 7485

- ★ 4-bit magnitude comparator.
- ★ Can be used to compare BCD or XS-3 codes.
- ★ Features :
  - Compares two 4-bit numbers.
  - Outputs : A > B, A = B, A < B.
  - Cascade inputs : $I_{A>B}$, $I_{A=B}$, $I_{A<B}$ for multi-bit comparison.
  - Pin 16 : Vcc, Pin 8 : GND.



Applications

- ★ Used in decimal arithmetic units.
- ★ Digital calculators.
- ★ Data transmission with error detection.
- ★ Microprocessor based systems.

Advantages

- ★ XS-3 is self-complementing code.
- ★ Simplifies subtraction using decimal complement.
- ★ Useful in computer arithmetic.

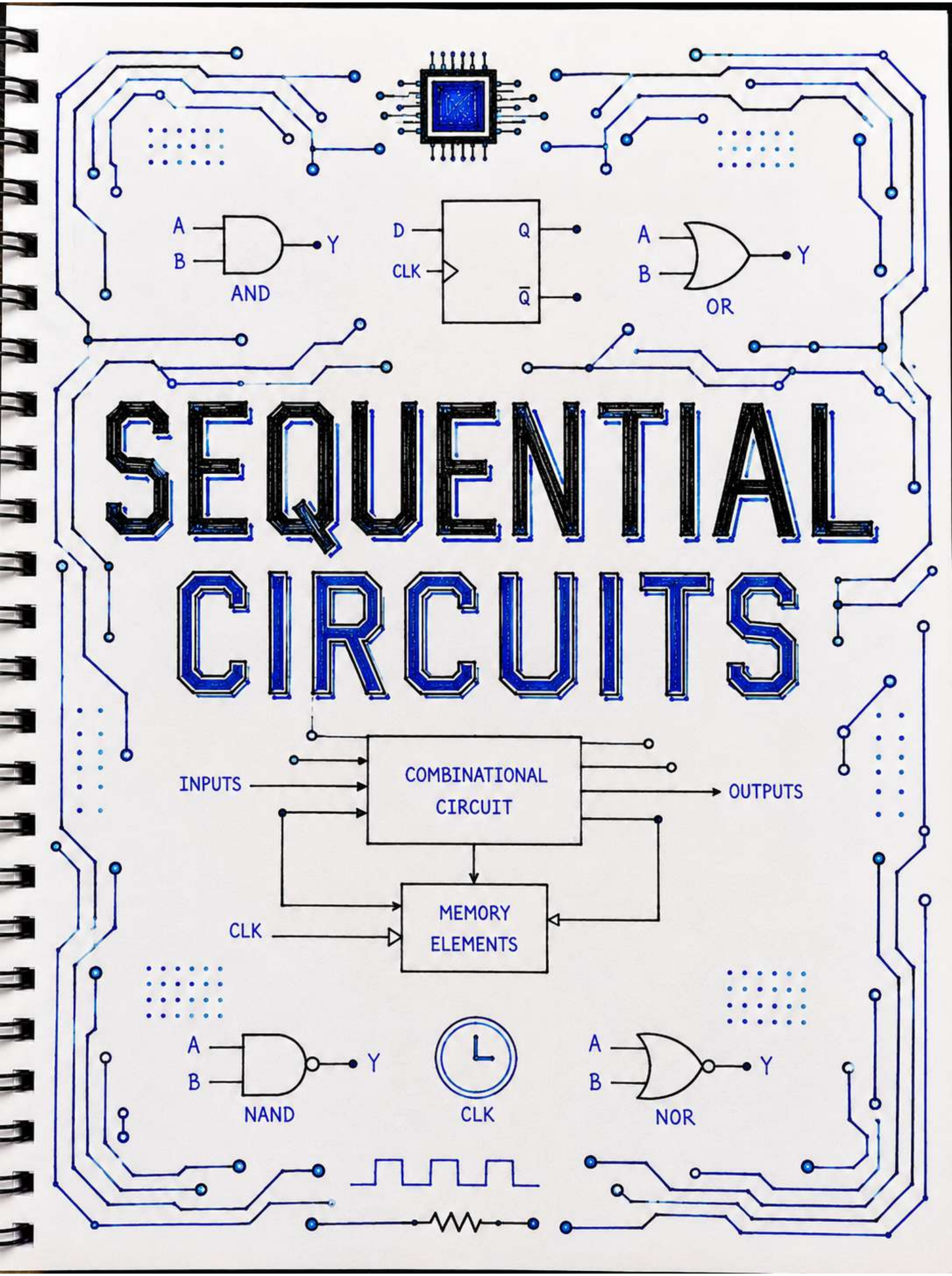
Disadvantages

- ★ Requires extra hardware for conversion.
- ★ Invalid for 10-15 in BCD inminput.

Key Points

- ★ Code convertors are essential for interfacing different coded systems.
- ★ Cascading of 4-bit comparator is used for higher bit comparisons.

Author : Konda Tharun Vivekananda


A
B
Y
AND
D
CLK
Q
$\bar{Q}$
A
B
Y
OR
SEQUENTIAL
CIRCUITS
INPUTS
COMBINATIONAL
CIRCUIT
OUTPUTS
CLK
MEMORY
ELEMENTS
A
B
Y
NAND
CLK
A
B
Y
NOR

SEQUENTIAL CIRCUITS

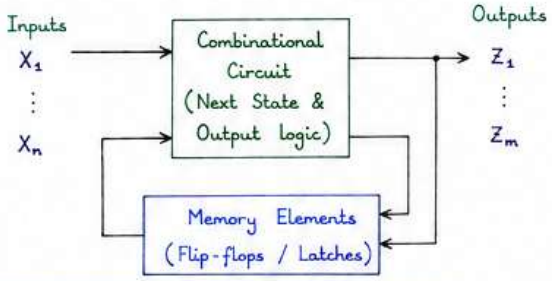
1. INTRODUCTION

→ Sequential circuits are digital circuits whose outputs depend on both present inputs and previous states.

→ It has memory element (flip-flops / latches) to store the state.

→ Used in counters, registers, controllers, memory, communication systems etc.

Basic Structure of Sequential Circuit



Key Terms

★ State : The information stored in memory elements.
★ Present State : Current state stored in flip-flops.
★ Next State : State that will be stored at next clock.
★ Output : Depends on inputs and present state.
★ Feedback : Connection from output of memory to input.

2. COMBINATIONAL vs SEQUENTIAL CIRCUITS

Parameter	Combinational Circuit	Sequential Circuit
1. Output	Depends only on present inputs.	Depends on present inputs and past state.
2. Memory	No memory element.	Has memory (FF / Latch).
3. Feedback	No feedback path.	Has feedback path.
4. Clock	Not required.	Required in synchronous type.
5. Example	Adder, MUX, Decoder.	Counter, Register, FSM.
6. Timing	No timing issues.	Depends on clock & delays.
7. Complexity	Less complex.	More complex.
8. Application	Arithmetic, data routing.	Control, sequence, memory.

Important Observation

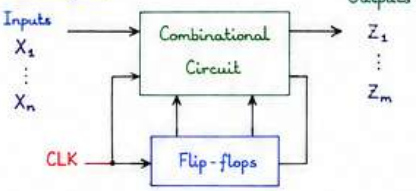
★ Every sequential circuit contains combinational part + memory.
★ Sequential circuits are built using flip-flops, latches and feedback.
★ They are used where order or history of events is important.

3. TYPES OF SEQUENTIAL CIRCUITS

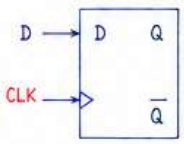
A. SYNCHRONOUS SEQUENTIAL CIRCUITS (SYNC)

→ All memory elements are driven by a common clock.
→ State changes occur only on clock edge (rising/falling).
→ Easy to design, test and analyze.

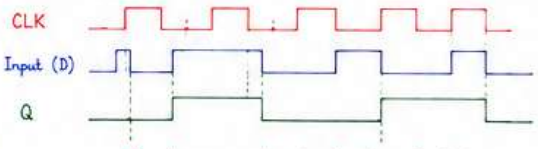
Block Diagram



Clocked Flip-Flop



Timing Diagram

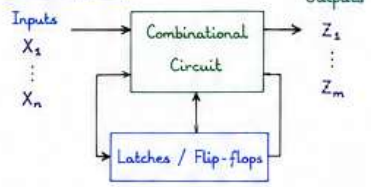


Q changes only at ↑ edge of clock

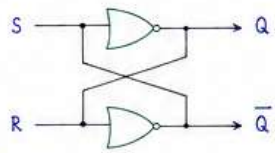
B. ASYNCHRONOUS SEQUENTIAL CIRCUITS (ASYNC)

→ No global clock signal.
→ State changes occur immediately due to input changes.
→ More complex and difficult to design.

Block Diagram



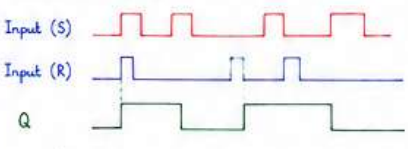
SR Latch (Example)



Operation

S	R	Q(next)	Description
0	0	Q(hold)	No change
0	1	0	Reset
1	0	1	Set
1	1	Invalid	Not allowed

Timing Diagram (No clock)

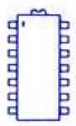


Q changes immediately with input change

ICs USED IN SEQUENTIAL CIRCUITS

Synchronous Circuits :

★ 74LS74 → Dual D Flip-Flop
★ 74LS76 → JK Flip-Flop with Clear & Preset
★ 74LS175 → Quad D Flip-Flop
★ 74LS163 → 4-bit Synchronous Counter



Asynchronous Circuits :

★ 74LS00 → Quad 2-input NAND (for latches)
★ 74LS02 → Quad 2-input NOR (for SR Latch)
★ 74LS279 → Quad SR Latch

Applications

★ Counters & Timers
★ Registers & Memories
★ Control Systems
★ Sequence Generators
★ Digital Communication
★ FSM Based Systems

Advantages (Sequential Circuits)

★ Can store information (memory).
★ Useful in complex digital systems.
★ Required for control & timing applications.
★ Can perform counting and sequencing.

Disadvantages

★ More complex than combinational circuits.
★ Requires clock (in synchronous circuits).
★ Propagation delay & timing issues.
★ More hardware (flip-flops, feedback).

Key Points

★ Sequential circuit = Combinational circuit + Memory + Feedback.
★ Two main types : Synchronous and Asynchronous.
★ Synchronous → with clock (safe & predictable).
★ Asynchronous → without clock (faster but complex).

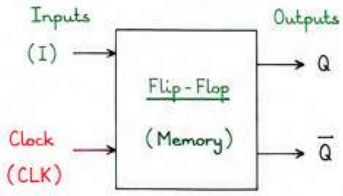
FLIPFLOPS (Sequential Circuits)

1. DEFINITION

→ A flip-flop is a basic sequential circuit which can store one bit of information (0 or 1).

→ It is a bistable multivibrator having two stable states.

→ It changes state according to inputs and clock.



2. TYPES OF FLIP-FLOPS

Flip-flops are mainly classified based on the inputs and operation.

- ★ SR Flip-Flop
- ★ JK Flip-Flop
- ★ D Flip-Flop (Delay)
- ★ T Flip-Flop (Toggle)

All are of two types based on triggering method:
(i) Edge Triggered
(ii) Level Triggered

3. COMPARISON : COMBINATIONAL vs SEQUENTIAL CIRCUITS

Parameter	Combinational Circuit	Sequential Circuit
1. Output	Depends only on present inputs.	Depends on present inputs and past state.
2. Memory	No memory element.	Has memory element (FF/Latch).
3. Feedback	No feedback path.	Has feedback path.
4. Clock	Not required.	Required in synchronous type.
5. Example	Adder, MUX, Decoder.	Counter, Register, FSM.
6. Timing	No timing issues.	Depends on clock and delays.
7. Complexity	Less complex.	More complex.
8. Application	Arithmetic, data routing.	Control, sequence, memory.

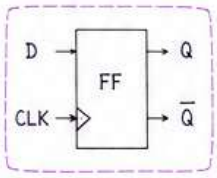
4. EDGE TRIGGERED FLIP-FLOP

→ Output changes only at the instant of clock transition (rising or falling edge).

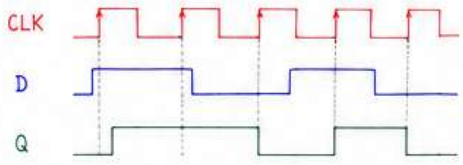
→ Between edges, output remains unchanged.

→ More reliable and less sensitive to clock pulse width.

Symbol



Timing Waveform (positive edge triggered D-FF)



Q changes only at ↑ edge of CLK.

Characteristics
- ★ Triggering on rising or falling edge.
- ★ Better noise immunity.
- ★ Requires precise edge.

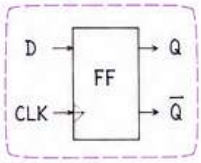
5. LEVEL TRIGGERED FLIP-FLOP

→ Output follows input as long as clock is at a particular level (HIGH or LOW).

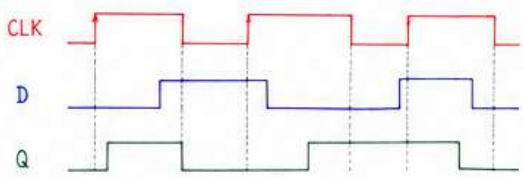
→ Output can change any time during the active level.

→ Sensitive to clock pulse width and delays.

Symbol



Timing Waveform (level triggered D latch - Active HIGH)



Q follows D while CLK = 1.

Characteristics
- ★ Triggering on HIGH or LOW level.
- ★ Simple design.
- ★ More susceptible to glitches.

TRUTH TABLES (D FLIP-FLOP)

Edge Triggered (at ↑ edge)

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

Q(t+1) = D (sampled at edge)

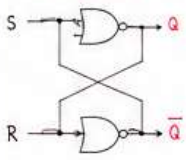
Level Triggered (Active HIGH)

CLK	D	Q(next)
0	X	Q(hold)
1	0	0
1	1	1

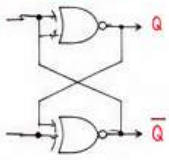
X = Don't care

BASIC FLIP-FLOP CIRCUITS

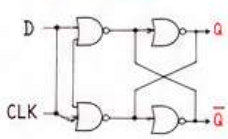
SR Flip-Flop



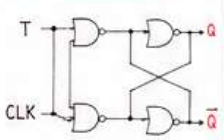
JK Flip-Flop



D Flip-Flop



T Flip-Flop



APPLICATIONS
- ★ Counters & Registers
- ★ Data storage
- ★ Sequence generators
- ★ Control systems
- ★ Digital communication
- ★ Microprocessor based systems

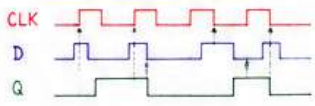
ADVANTAGES
- ★ Can store 1 bit of data.
- ★ Essential for sequential operations.
- ★ Easy to cascade for multi-bit registers.
- ★ Edge triggered FFs have better stability.

DISADVANTAGES
- ★ More complex than combinational circuits.
- ★ Requires clock (for synchronous).
- ★ Propagation delay & timing issues.
- ★ Sensitive to noise (level triggered).

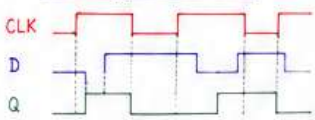
KEY POINTS
- ★ Flip-flop is the basic memory element.
- ★ Types: SR, JK, D, T.
- ★ Triggering: Edge or Level.
- ★ Used everywhere in digital systems.

GRAPHS SUMMARY

Edge Triggered (D-FF)



Level Triggered (D Latch)

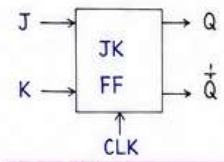


FLIPFLOPS (Sequential Circuits)

Definition (General) : A flipflop is a bistable multivibrator used to store 1 bit of information. It has two stable states (0 and 1) and changes state on the basis of inputs and clock.

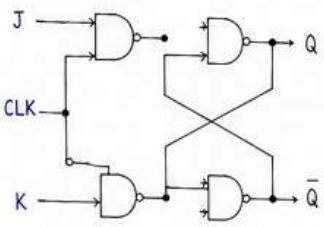
1. JK FLIP-FLOP

Symbol



Definition :
JK flip-flop is a clocked bistable circuit that stores 1 bit and has four possible operations : No change, Reset, Set and Toggle.

Circuit (NAND Implementation)



CLK enables when HIGH.

Truth Table

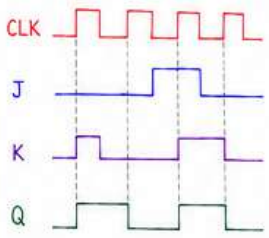
J	K	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
0	1	Q	0	Reset
1	0	0	1	Set
1	1	0	$\bar{Q}$	Toggle

(Forbidden state does not exist)

Characteristic Table

J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

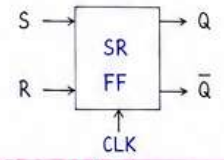
Timing Diagram



Q changes on active HIGH level of CLK.

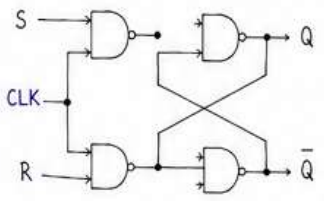
2. SR FLIP-FLOP

Symbol



Definition :
SR flip-flop is a clocked bistable circuit with two inputs Set (S) and Reset (R). It stores 1 bit and has three valid operations.

Circuit (NAND Implementation)



CLK enables when HIGH.

Truth Table

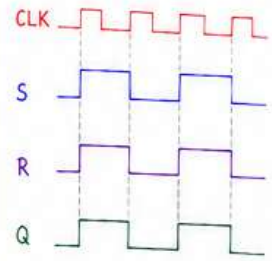
S	R	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
1	0	1	1	Set
0	1	0	0	Reset
1	1	X	X	Forbidden (Invalid)

X → Indeterminate state

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
1	0	0	1
1	0	1	1
0	1	0	0
0	1	1	0
1	1	0	X
1	1	0	X
1	1	1	X

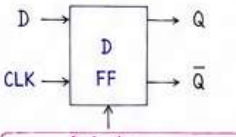
Timing Diagram



S & R should not be HIGH together.

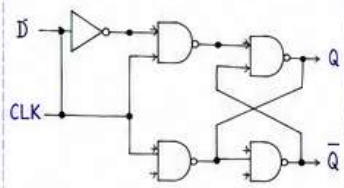
3. D FLIP-FLOP

Symbol



Definition :
D flip-flop stores the value of single data input (D) at the instant of active clock and outputs the same until the next active clock.

Circuit (NAND Implementation)



Output follows D when CLK = HIGH.

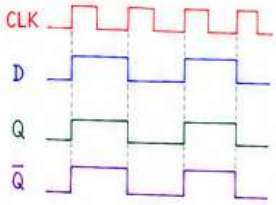
Truth Table

D	Q(t)	Q(t+1)	Operation
0	0	0	Reset
0	1	0	Reset
1	0	1	Set
1	1	1	Set

Characteristic Table

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

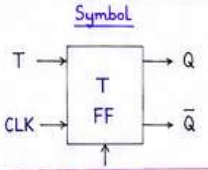
Timing Diagram



Q takes the value of D on active HIGH level.

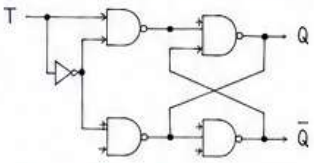
4. T FLIP-FLOP

Symbol



Definition :
T flipflop has a single input (T). It toggles its state when T = 1 on the active clock and holds state when T = 0.

Circuit (NAND Implementation)



If T = 0 → No change
If T = 1 → Toggle

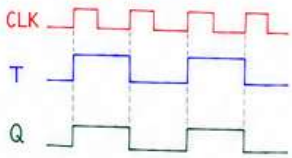
Truth Table

T	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	1	No change
1	0	1	Toggle
1	1	0	Toggle

Characteristic Table

T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Timing Diagram



Q toggles when T = 1 on active HIGH level.

SUMMARY / KEY POINTS

- Flipflops are 1-bit memory elements.
- All are level triggered (active HIGH).
- Stores state and changes only when CLK is active.
- Used for registers, counters, FSMs, etc.

APPLICATIONS

- Counters & sequence generation
- Registers & shift registers
- Data storage in digital systems
- Control units in FSM
- Frequency division
- Synchronization

ADVANTAGES

- Simple & easy to design
- Reliable storage of data
- Low power (with CMOS)
- High speed operation
- Available in many ICs

DISADVANTAGES

- SR FF has invalid state (S = R = 1).
- Sensitive to noise & glitches.
- Propagation delay limitation.

QUICK COMPARISON

Flipflop	Inputs	Operation
JK	J, K	Toggle, Set, Reset, No change
SR	S, R	Set, Reset, No change (S = R = 1 invalid)
D	D	Output follows D
T	T	T = 1 Toggle, T = 0 No change

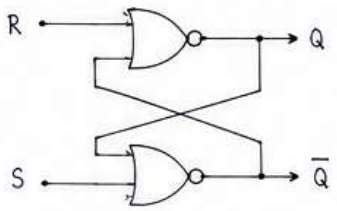
LATCHES (Sequential Circuits)

Definition : A latch is a bistable multivibrator which stores 1 bit of information.
It is level sensitive i.e., output follows input when enable level is active.

Types of Latches
- ★ SR Latch
- ★ Gated SR Latch
- ★ D Latch
- ★ Gated D Latch

1. SR LATCH (Basic)

Circuit (Cross Coupled NOR)



No clock. Inputs act directly.
When S = R = 0 → Hold (No change).

Truth Table

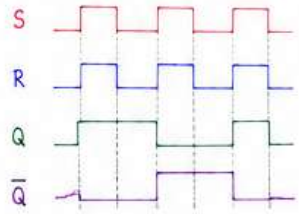
S	R	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
0	1	0	0	Reset
1	0	1	1	Set
1	1	–	–	Forbidden

(Both outputs go to 0)

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X
1	1	1	X

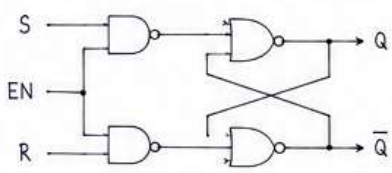
Timing Diagram



Outputs follow inputs immediately (Level sensitive).

2. GATED SR LATCH (Enable)

Circuit



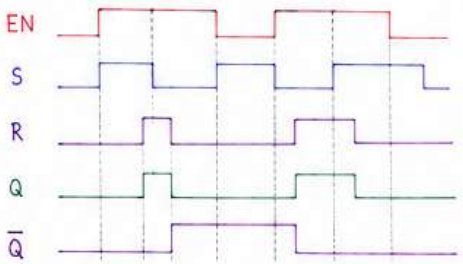
When EN = 0 → Latch holds
When EN = 1 → Acts like SR Latch

Truth Table

EN	S	R	Q(t)	Q(t+1)	Operation
0	X	X	Q	Q	Hold
1	0	0	Q	Q	
1	1	1	0	0	Reset
1	1	0	1	1	Set
1	1	1	–	–	Forbidden

X → Don't care

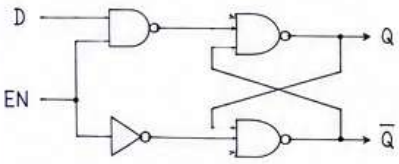
Timing Diagram



Q changes only when EN = 1.
When EN = 0, outputs hold previous state.

3. D LATCH (Transparent Latch)

Circuit



When EN = 0 → Q follows D
When EN = 0 → Latch holds

Truth Table

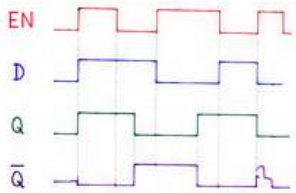
EN	D	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	0	X	0	Reset
1	1	X	1	Set

X → Don't care

Characteristic Table

EN	D	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	0	0	1
1	1	0	1
1	1	1	1

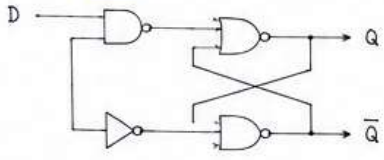
Timing Diagram



When EN = 1, Q = D.
When EN = 0, Q holds.

4. GATED D LATCH (Enable / Non-Inverting)

Circuit

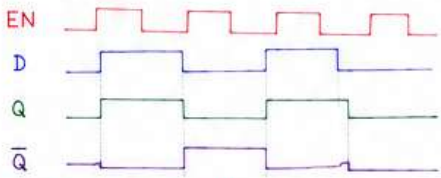


Same as D Latch (level sensitive).

Truth Table

EN	D	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	0	X	0	Reset
1	1	X	1	Set

Timing Diagram



Q follows D when EN = 1.
Otherwise holds previous state.

APPLICATIONS
- ★ Data storage in registers and memory elements.
- ★ Used in shift registers.
- ★ Counters and sequence generators.
- ★ Data synchronization.
- ★ Control circuits and FSMs.

ADVANTAGES
- ★ Simple and easy to design.
- ★ Requires fewer gates.
- ★ Low power consumption.
- ★ Fast operation.
- ★ To store 1 bit of data.

DISADVANTAGES
- ★ Level sensitive (may cause race around problems).
- ★ SR latch has invalid state (S = R = 1).
- ★ Sensitive to glitches and noise.
- ★ Not suitable for high speed without care.

KEY POINTS
- ★ Latches are level sensitive.
- ★ Outputs change as long as enable is active.
- ★ Used as building blocks for flip-flops.
- ★ Flip-flops are edge triggered, latches are level triggered.

QUICK COMPARISON

Latch Type	Active Condition
SR Latch	Always active (No clock)
Gated SR Latch	EN = 1
D Latch	EN = 1
Gated D Latch	EN = 1

Author : Konda Tharun Vivekananda

LATCHES (Sequential Circuits)

Types of Latches
- SR Latch
- Gated SR Latch
- D Latch (Transparent)
- Gated D Latch
- JK Latch
- T Latch

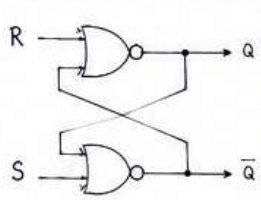
Definition : A latch is a bistable multivibrator used to store 1 bit of information. It is level sensitive i.e., when enable (or control) level is active, output follows input. When enable is inactive, it holds the previous state.

1. SR LATCH (Basic Latch)

Definition :

SR latch is a basic latch with two inputs S (Set) and R (Reset). It has two stable states and is built using cross coupled NOR gates.

Circuit (Cross Coupled NOR)



Truth Table

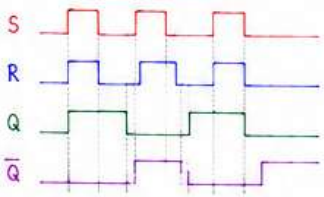
S	R	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
0	1	0	0	Reset
1	0	1	1	Set
1	1	–	–	Forbidden (Invalid)

(Both outputs go to 0)

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X
1	1	1	X

Timing Diagram



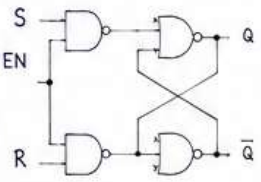
Outputs follow inputs when active (level high)
When S = R = 0, output holds previous state.

2. GATED SR LATCH

Definition :

Gated SR latch is SR latch with an enable input (EN). When EN = 1, it behaves like SR latch. When EN = 0, it holds the state.

Circuit



Truth Table

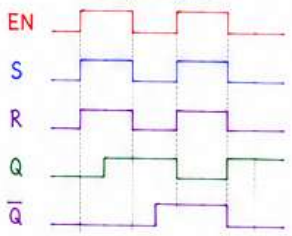
EN	S	R	Q(t)	Q(t+1)	Operation
0	X	X	Q	Q	Hold
1	0	0	Q	Q	No change
1	0	1	0	0	Reset
1	1	0	1	1	Set
1	1	1	–	–	Forbidden (Invalid)

X → Don't care

Characteristic Table

EN	S	R	Q(t)	Q(t+1)
0	X	X	0	0
0	X	X	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	0	0	1	1
1	1	1	0	X
1	1	1	1	X
1	1	1	1	X

Timing Diagram



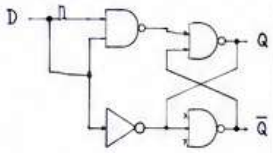
When EN = 1 → outputs follows S & R
When EN = 0 → outputs hold previous state.

3. D LATCH (Transparent Latch)

Definition :

D latch has one data input D and an enable input (EN). Q follows D when EN = 1 (transparent) and holds when EN = 0.

Circuit



Truth Table

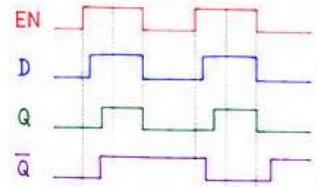
EN	D	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	1	X	1	Set

X → Don't care

Characteristic Table

EN	D	Q(t)	Q(t+1)
0	X	0	0
0	X	1	1
1	0	0	0
1	0	0	0
1	1	1	0
1	1	1	1
1	1	1	1

Timing Diagram



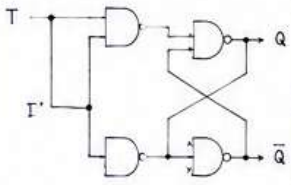
When EN = 1 → Q follows D
When EN = 0 → Q holds.

4. T LATCH

Definition :

D latch has one input T and enable (EN). When EN = 1, output toggles if T = 1 and holds if T = 0. When EN = 0, it holds.

Circuit



If T = 0 → No change
If T = 1 → Toggle

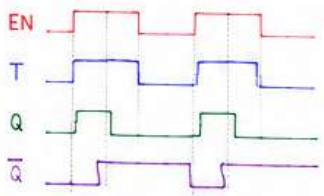
Truth Table

EN	T	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	0	Q	Q	No change
1	1	1	$\bar{Q}$	Toggle

Characteristic Table

EN	T	Q(t)	Q(t+1)
0	X	0	0
0	X	1	1
1	0	0	0
1	0	1	1
1	0	1	1
1	1	0	1
1	1	1	0

Timing Diagram



T = 1 with EN = 1 → Output toggles.
T = 0 → No change, Hold when EN = 0.

APPLICATIONS
- Data storage in registers and memory elements.
- Used in shift registers.
- Counters and sequence generators.
- Control circuits.
- Switch debouncing.

ADVANTAGES
- Simple structure.
- Requires fewer gates (low hardware cost).
- Low power consumption.
- Fast operation.
- Reliable storage of 1 bit data.

DISADVANTAGES
- Level sensitive → may cause race around problems.
- Not suitable for high speed without careful design.
- Forbidden state in SR (and gated SR).

KEY POINTS
- Latches are level sensitive.
- Outputs change as long as enable is active.
- Flip-flops are edge triggered, latches are level triggered.
- Used as building blocks for sequential circuits.

QUICK COMPARISON

Latch Type	Inputs	Condition (when active)
SR Latch	S, R	S,R control (Set/Reset)
Gated SR Latch	S, R, EN	EN = 1 → SR op
D Latch	D, EN	Q follows D
Gated D Latch	D, EN	EN = 1 → Q follows D
JK Latch	J, K, EN	J,K control (like FF JK)
T Latch	T, EN	T = 1 → Toggle

ALL FLIP-FLOP CONVERSIONS

(From JK Flip-Flop)

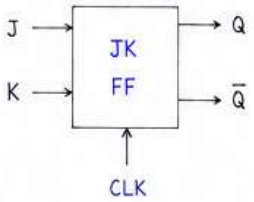
1. INTRODUCTION

→ A flip-flop conversion means realizing the behavior of one flip-flop using another flip-flop by proper input connections.

→ JK flip-flop is the most versatile. Hence it can be converted to SR, T and D flip-flops.

→ Clocked conversion is shown here. (Applies for edge triggered also)

JK Flip-Flop (Reference)



JK Flip-Flop Truth Table (Reference)

J	K	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
0	1	Q	0	Reset
1	0	0	1	Set
1	1	Q	$\bar{Q}$	Toggle

Key Points

★ J = 1 → Set condition
★ K = 1 → Reset condition
★ J = K = 1 → Toggle
★ $\bar{Q}$ is complement of Q

2. JK TO SR FLIP-FLOP CONVERSION

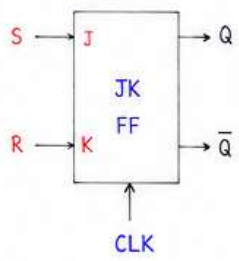
Idea :

Make J and K behave like S and R.

Connections

J = S
K = R

Circuit



Truth Table (Equivalent to SR)

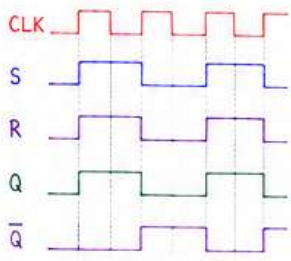
S	R	Q(t)	Q(t+1)	Operation
0	0	Q	Q	Hold
0	1	Q	0	Reset
1	0	0	1	Set
1	1	1	Q	Forbidden

(Same as SR flip-flop)

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0
1	1	1	0

Timing Diagram



Outputs follow inputs when CLK is active (level high).

3. JK TO T FLIP-FLOP CONVERSION

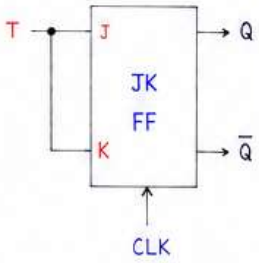
Idea :

T flip-flop has single input T.
Toggle when T = 1,
No change when T = 0.

Connections

J = T
K = T

Circuit



Truth Table (Equivalent to T)

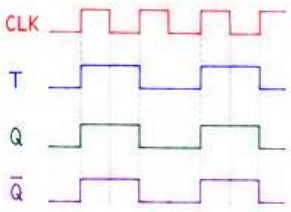
T	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	1	No change
1	0	1	Toggle
1	1	0	Toggle

(Same as T flip-flop)

Characteristic Table

T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	1

Timing Diagram



Q toggles only when T = 1 during active level of CLK.

4. JK TO D FLIP-FLOP CONVERSION

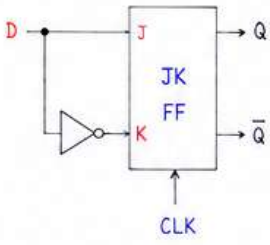
Idea :

D flip-flop has single input D.
Output follows D.
So, set J and K using D and Q.

Connections

J = D
K = $\bar{D}$

Circuit



Truth Table (Equivalent to D)

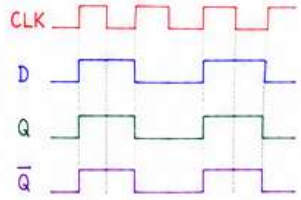
D	Q(t)	Q(t+1)	Operation
0	0	0	Reset
0	1	0	Reset
1	0	1	Set
1	1	1	Set

(Same as D flip-flop)

Characteristic Table

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

Timing Diagram



Q follows D when CLK is active (transparent).

SUMMARY / KEY POINTS

★ JK flip-flop is universal.
★ By proper input connections, it can be converted to other flip-flops.
★ Conversions shown are for level sensitive latches.
★ For edge triggered FFs, same relations apply.

ADVANTAGES

★ Uses only JK flip-flop IC.
★ Reduces variety of ICs in design.
★ Easy to implement.
★ Flexible and universal behavior.

DISADVANTAGES

★ Extra gates (like inverter) may be required.
★ Slight increase in propagation delay.
★ More power if additional gates are used.

APPLICATIONS

★ Digital registers
★ Counters
★ Sequence generators
★ Controllers
★ Data storage systems
★ Communication systems

QUICK COMPARISON

Target FF	Input(s)	Connection
SR Latch	S, R	J = S, K = R
T Latch	T	J = T, K = T
D Latch	D	J = D, K = $\bar{D}$

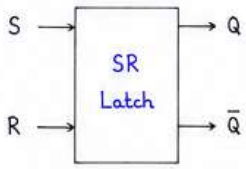
ALL FLIP-FLOP CONVERSIONS

(From SR Flip-Flop)

1. INTRODUCTION

- → A flip-flop conversion means realizing the behavior of one flip-flop using another flip-flop by proper input connections.
- → SR flip-flop is the most basic flip-flop. Hence, it can be converted to JK, T and D flip-flops.
- → Conversions shown here are for level sensitive latches.

SR Flip-Flop (Reference)



SR Latch Truth Table (Reference)

S	R	Q(t)	Q(t+1)	Operation
0	0	Q	Q	Hold
0	1	Q	0	Reset
1	0	Q	1	Set
1	1	–	–	Forbidden (Invalid)

Key Points

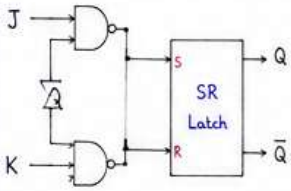
- ★ S = 1 → Set
- ★ R = 1 → Reset
- ★ S = R = 0 → Hold
- ★ S = R = 1 → Forbidden
- ★ $\bar{Q}$ is complement of Q

2. SR TO JK FLIP-FLOP CONVERSION

Idea :
Make SR inputs using J, K and present state (Q).

Connections :
$S = J \cdot \bar{Q}$
$R = K \cdot Q$

Circuit



Two AND gates + SR Latch

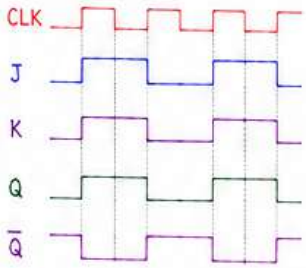
Truth Table (Equivalent to JK)

J	K	Q(t)	Q(t+1)	Operation
0	0	Q	Q	No change
0	1	Q	0	Reset
1	0	Q	1	Set
1	1	–	$\bar{Q}$	Toggle

Characteristic Table

J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Timing Diagram



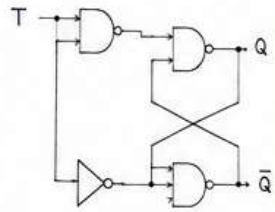
Outputs change according to J & K when CLK is HIGH.

3. SR TO T FLIP FLOP CONVERSION

Idea :
T flip-flop toggles when T = 1 and holds when T = 0.

Connections :
$S = T \cdot \bar{Q}$
$R = T \cdot Q$

Circuit



Two AND gates + SR Latch

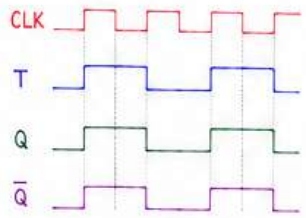
Truth Table (Equivalent to T)

T	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	1	No change
1	0	1	Toggle
1	1	0	Toggle

Characteristic Table

T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Timing Diagram



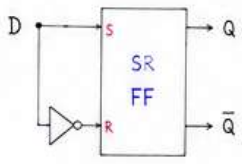
Q toggles only when T = 1 is active (active level of CLK).

4. SR TO D FLIP-FLOP CONVERSION

Idea :
D flip-flop output follows the input D (when CLK is active).
Hence, Set when D = 1 and Reset when D = 0.

Connections :
$S = D$
$R = \bar{D}$

Circuit



Inverter + SR Latch

Truth Table (Equivalent to D)

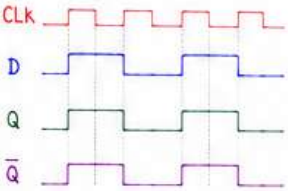
D	Q(t)	Q(t+1)	Operation
0	X	0	Reset
1	X	1	Set

(X → Don't care)

Characteristic Table

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

Timing Diagram



Q follows D when CLK is active (transparent).

ADVANTAGES

- ★ Conversions use minimum additional gates.
- ★ Allows using any available FF for required function.
- ★ Easy to implement with combinational logic.
- ★ Useful for design flexibility.

DISADVANTAGES

- ★ Extra gates increase propagation delay.
- ★ More hardware → more power.
- ★ SR latch forbidden state must be avoided.

APPLICATIONS

- ★ Design of counters, registers.
- ★ Control circuits in FSM.
- ★ Data storage and buffering.
- ★ Communication and interface circuits.
- ★ General purpose sequential circuit design.

SUMMARY / KEY POINTS

- ★ SR latch is the basic level sensitive storage element.
- ★ Conversions: SR → JK, T, D
- ★ SR inputs realized using combinational logic.
- ★ Behaviors are equivalent in terms of Q(t+1).

QUICK COMPARISON

Target FF	Connections (From SR)
JK Flip-Flop	$S = J\bar{Q}$, $R = KQ$
T Flip-Flop	$S = T\bar{Q}$, $R = TQ$
D Flip-Flop	$S = D$, $R = \bar{D}$

ALL FLIP-FLOP CONVERSIONS

(From D Flip-Flop)

1. INTRODUCTION

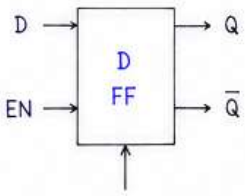
→ Flip-flop conversion means realizing the same behavior of one flip-flop using another flip-flop with proper input connections.

→ D flip-flop is the most versatile.

→ Hence, it can be converted to JK, T and SR flip-flops.

→ Conversions shown here are for level sensitive latches (active HIGH Enable).

D Flip-Flop (Reference)



(Level sensitive latch)

D Flip-Flop Truth Table (Reference)

EN	D	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	0	Q	0	Reset
1	1	Q	1	Set

X → Don't care

Key Points

★ EN = 1 → Latch is active.

★ EN = 0 → Latch holds previous state.

★ $\overline{Q}$ is complement of Q.

★ All conversions use D FF as reference.

2. D TO JK FLIP-FLOP CONVERSION

Idea :

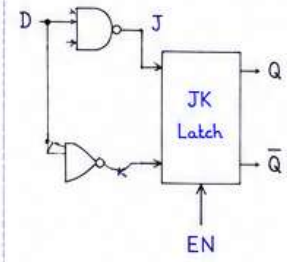
Find J and K in terms of D and present state Q. Then apply to JK latch.

Connections :

$J = D.\overline{Q}$

$K = \overline{D}.Q$

Circuit



Truth Table (Equivalent to Truth)

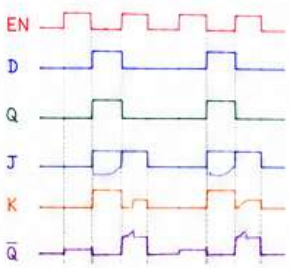
Q	D	J	K	Q(t+1)	Operation
0	0	0	0	0	Reset
1	0	1	1	0	Reset
1	0	0	0	1	Set
1	1	1	0	1	Forbidden

$(J = D.\overline{Q},\ K = \overline{D}.Q)$

Characteristic Table

J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0
1	1	1	0

Timing Diagram



When EN = 1, Q follows D

J and K are generated as $J = D.\overline{Q}$, $K = \overline{D}.Q$.

3. D TO T FLIP-FLOP CONVERSION

Idea :

Find T in terms of D and present state Q.

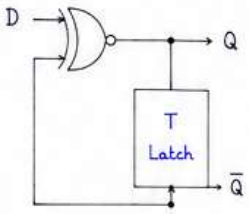
T = 1 when Q at state.

Connection :

$T = D \oplus Q$

$= D.\overline{Q} + \overline{D}.Q$

Circuit



Truth Table (Equivalent to T)

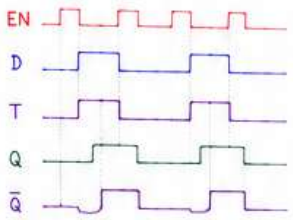
Q	Q(t)	T	Q(t+1)	Operation
0	0	0	0	No change
0	1	1	1	Toggle
1	0	1	1	Toggle
1	1	0	0	No change

$(T = D \oplus Q)$

Characteristic Table

T	Q(t)	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Timing Diagram



Q toggles only when T = 1 (i.e., D ≠ Q) during EN = 1.

4. D TO SR FLIP-FLOP CONVERSION

Idea :

Generate S = 1 only when we need to set (D = 1 and Q = 0).

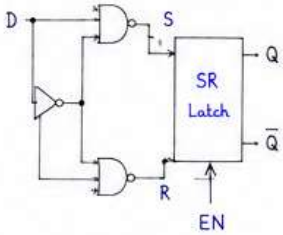
Proper S = 1 when we need to reset (D = 0 and Q = 1).

Connections :

$S = D.\overline{Q}$

$R = \overline{D}.Q$

Circuit



Truth Table (Equivalent to SR)

Q	D	S	R	Q(t+1)	Operation
0	0	0	0	0	No change
0	1	1	0	1	Set
1	0	0	1	0	Reset
1	1	0	0	1	No change

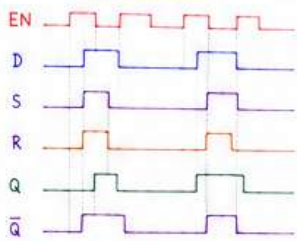
$(S = D.\overline{Q},\ R = \overline{D}.Q)$

(S = R = 1 never occurs)

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	0	1
1	1	1	1
1	1	–	–

Timing Diagram



During EN = 1, $S = D.\overline{Q}$, $R = \overline{D}.Q$ and Q follows D.

SUMMARY / KEY POINTS

★ D FF is used as reference.

★ Conversions are for level sensitive latches (EN = 1).

★ Proper input combinations reproduce same behavior.

★ Feedback from Q and $\overline{Q}$ is used to generate inputs.

ADVANTAGES

★ Uses only D flip-flop IC.

★ Reduces IC variety.

★ Easy to implement.

★ Saves space and cost.

★ Flexible and universal.

DISADVANTAGES

★ More gates required.

★ Propagation delay increases slightly.

★ Extra power due to additional gates.

APPLICATIONS

★ Design of counters, registers.

★ Control circuits in FSM.

★ Data synchronization.

★ Communication systems.

★ General purpose sequential circuits.

QUICK RECAP (FORMULAS)

★ JK : $J = D.\overline{Q}$, $K = \overline{D}.Q$

★ T : $T = D \oplus Q = D.\overline{Q} + \overline{D}.Q$

★ SR : $S = D.\overline{Q}$, $R = \overline{D}.Q$

CONVERSION SUMMARY TABLE

Target FF	Inputs in terms of D and Q
JK Flip-Flop	$J = D.\overline{Q}$, $K = \overline{D}.Q$
T Flip-Flop	$T = D \oplus Q$
SR Flip-Flop	$S = D.\overline{Q}$, $R = \overline{D}.Q$

ALL FLIP-FLOP CONVERSIONS

(From T Flip-Flop)

1. INTRODUCTION

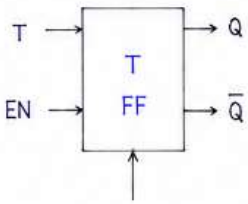
→ A flip-flop conversion means realizing the same behavior of one flip-flop using another flip-flop with proper input connections.

→ T flip-flop is a single input flip-flop. It toggles when T = 1 and holds when T = 0.

→ Hence, it can be converted to JK, D and SR flip-flops.

→ Conversions shown here are for level sensitive latches (active HIGH).

T Flip-Flop (Reference)



(Level sensitive latch)

T Flip-Flop Truth Table (Reference)

EN	T	Q(t)	Q(t+1)	Operation
0	X	Q	Q	Hold
1	0	Q	Q	No change
1	1	Q	$\bar{Q}$	Toggle

X → Don't care

Key Points

★ EN = 1 → Latch is active.
★ EN = 0 → Holds previous state.
★ T = 0 → No change.
★ T = 1 → Toggle.
★ $\bar{Q}$ is complement of Q.
★ All conversions use T FF as reference.

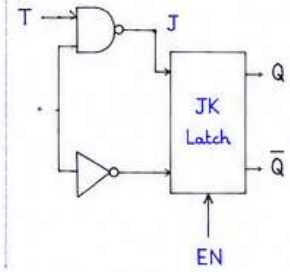
2. T TO JK FLIP-FLOP CONVERSION

Idea :

Generate J and K in terms of T and present state Q.
When T = 0 → J = K = 0 (No change)
When T = 1 → J = $\bar{Q}$, K = Q

Connections :
J = T . $\bar{Q}$
K = T . Q

Circuit



Truth Table (Equivalent to JK)

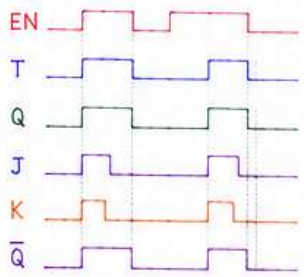
T	Q	J	K	Q(t+1)	Operation
0	0	0	0	0	No change
1	1	0	0	0	No change
1	0	1	1	0	Set
1	1	1	0	0	No change

(J = T . $\bar{Q}$, K = T . Q)

Characteristic Table

J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Timing Diagram



When EN = 1, J & K generated from T and Q as shown.

3. T TO D FLIP-FLOP CONVERSION

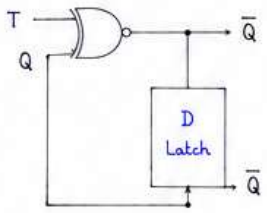
Idea :

For T flip-flop, Q(t+1) = D.
When T = 0 → D = Q
When T = 1 → D = $\bar{Q}$

Connection :
D = T ⊕ Q

(Exclusive OR gate)

Circuit



Truth Table (Equivalent to D)

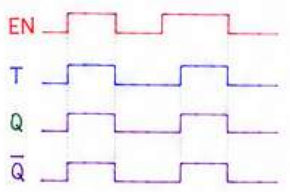
T	Q	D	Q(t+1)	Operation
0	0	0	0	No change
0	1	1	1	No change
1	0	1	1	Toggle
1	1	0	0	Toggle

(D = T ⊗ Q)

Characteristic Table

D	Q(t)	Q(t+1)
0	0	0
0	1	0
1	0	1
1	1	1

Timing Diagram



D is obtained by XOR of T and Q.
Q follows D when EN = 1.

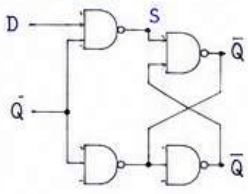
4. T TO SR FLIP-FLOP CONVERSION

Idea :

Generate S and R using T and present state Q.
When T = 0 → S = R = 0 (No change)
When T = 1 → S = $\bar{Q}$, R = Q

Connections :
S = T . $\bar{Q}$
R = T . Q

Circuit



Truth Table (Equivalent to SR)

T	Q	S	R	Q(t+1)	Operation
0	0	0	0	0	No change
0	0	1	0	1	Set
1	1	0	1	0	Reset

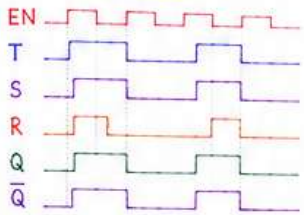
(S = T . $\bar{Q}$, R = T . Q)

Characteristic Table

S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	1	1
1	1	–	–

Forbidden

Timing Diagram



S = R = 0 → Hold,
S = 1 → Set, R = 1 → Reset

SUMMARY / KEY POINTS

★ T FF is used as reference.
★ T FF toggles when T = 1 and holds when T = 0.
★ Conversions are for level sensitive latches (EN = 1)
★ Proper input equations reproduce same behavior.

ADVANTAGES

★ Reduces IC variety.
★ Uses minimum additional gates.
★ Simple and systematic design.
★ Helpful in exam problems and practical design.

DISADVANTAGES

★ Extra gates → increase propagation delay.
★ More hardware → more power and cost.
★ Care required for forbidden conditions in SR.

APPLICATIONS

★ Counters (using T FF).
★ Frequency division.
★ Control circuits.
★ State machines.
★ Data storage systems.

QUICK COMPARISON

Target FF	Input Equations (from T FF)
JK Flip-Flop	J = T . $\bar{Q}$, K = T . $\bar{Q}$
D Flip-Flop	D = T ⊗ Q
SR Flip-Flop	S = T . $\bar{Q}$, R = T . $\bar{Q}$

Counters

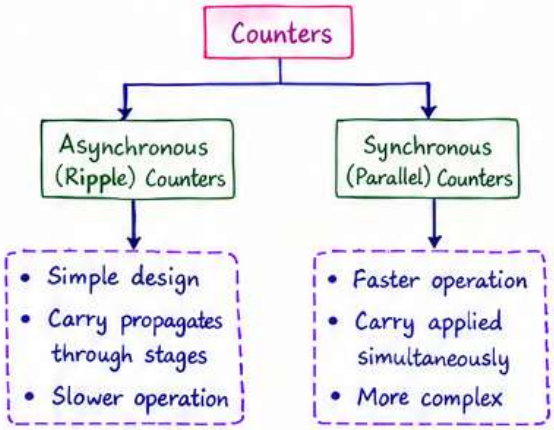
1. Introduction

- A counter is a sequential circuit that advances through a predetermined sequence of states in response to clock pulses.
- It is used for counting, timing, and frequency division.
- Counters operate based on flip-flops and logic gates.

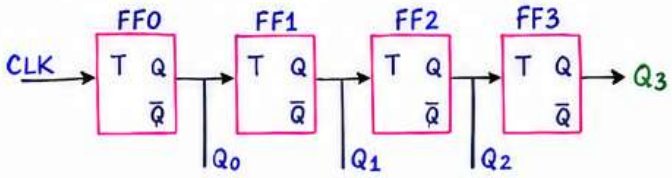
Key Point

- Counters are broadly classified as Asynchronous (Ripple) and Synchronous (Parallel) counters.

2. Classification of Counters



3. Basic Counter Design (Asynchronous)



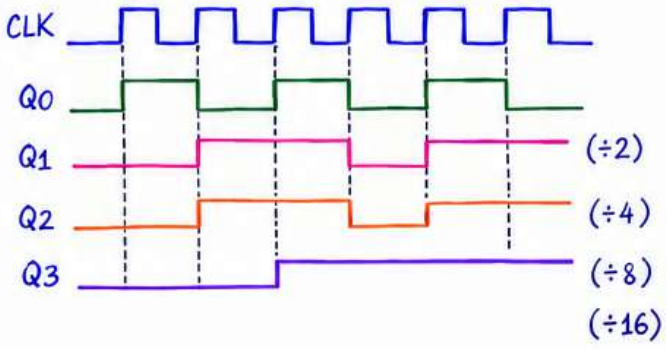
4. Synchronous Counter

- All flip-flops are triggered by the same clock.
- All the is is determined by the present state.
- Requires additional logic (gates).

5. Truth Table (4-bit Binary Counter)

Present State				Next State			
Q_3	Q_2	Q_1	Q_0	Q_3	Q_2	Q_1	Q_0
0	0	0	0 →	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
...	...	...	...	...	...	...	...
1	1	1	1	0	0	0	0

6. Waveform of 4-bit Counter



7. Applications of Counters

- Digite timers and clocks.
- Frequency measurement.
- Sequence generation.
- Address generation in memory systems.
- Event counting.

8. Advantages ✓

- Simple and economical.
- Useful for timing and control.
- Can be designed for dared modulus.

9. Disadvantages ✗

- Ripple counters are relatively slow.
- Propagation delay accumulates.
- Limited speed.

10. Summary / Key Points

- Counters are sequential circuits used for counting and timing.
- Asynchronous (ripple) counters are simple but slower.
- Synchronous (parallel) counters are faster but more complex.
- Used in timers, frequency dividers and sequence generators.

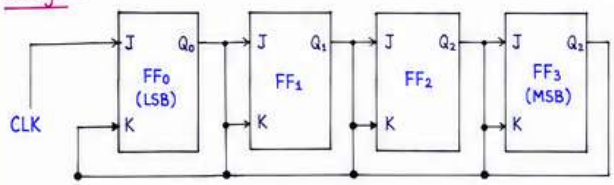
11. Interview / Viva Point

- Ripple counters are simple but suffer from propagation delay, while synchronous counters are faster.

COUNTERS - SYNCHRONOUS COUNTERS

1. 4 BIT SYNCHRONOUS UP COUNTER (MOD-16)

Design :



{ All FFs share common CLK → synchronous operation }

Characteristic Table (JK FF)

J	K	Q(t+1)	Operation
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}(t)$	Toggle

Excitation Table (JK FF)

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
X	X	X	1
X	X	X	0

X → Don't care

State Table (MOD-16)

PS				NS				Decimal
Q_3	Q_2	Q_1	Q_0	Q_{3+}	Q_{2+}	Q_{1+}	Q_{0+}	
0	0	0	0	→	0	0	1	0
0	0	0	1	→	0	0	0	1
0	0	1	0	→	0	0	1	2
0	0	1	1	→	0	1	0	3
0	1	0	0	→	1	0	1	4
0	1	0	1	→	1	0	0	5
0	1	1	0	→	1	0	1	6
0	1	1	1	→	1	1	0	7
1	0	0	0	→	1	0	1	8
1	0	0	1	→	1	0	0	9
1	0	1	0	→	1	0	1	10
1	0	1	1	→	1	1	0	11
1	1	0	0	→	1	0	1	12
1	1	0	1	→	1	0	0	13
1	1	1	0	→	1	0	1	14
1	1	1	1	→	0	0	0	15

Transition Table

Present State	Next State	Transition
0 0 0 0	0 0 0 1	0 → 1
0 0 0 1	0 0 1 0	1 → 2
0 0 1 0	0 0 1 1	2 → 3
0 0 1 1	0 1 0 0	3 → 4
0 1 0 0	0 1 0 1	4 → 5
0 1 0 1	0 1 1 0	5 → 6
0 1 1 0	0 1 1 1	6 → 7
0 1 1 1	1 0 0 0	7 → 8
1 0 0 0	1 0 0 1	8 → 9
1 0 0 1	1 0 1 0	9 → 10
1 0 1 0	1 0 1 1	10 → 11
1 0 1 1	1 1 0 0	11 → 12
1 1 0 0	1 1 0 1	12 → 13
1 1 0 1	1 1 1 0	13 → 14
1 1 1 0	1 1 1 1	14 → 15
1 1 1 1	0 0 0 0	15 → 0

K-Maps for JK Inputs

J_0 (Q_3Q_2 rows, Q_1Q_0 columns)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

K_0

00	01	11	10
1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

J_1 (Q_3Q_2 rows, Q_1Q_0 columns)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

K_1

00	01	11	10
0	1	0	1
0	1	0	1
0	1	0	1
0	1	0	1

J_2 (Q_3Q_2 rows, Q_1Q_0 columns)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

K_2

00	01	11	10
0	0	1	1
0	0	1	1
0	0	1	1
0	0	1	1

J_3 (Q_3Q_2 rows, Q_1Q_0 columns)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	1	1	1	1
10	1	1	1	1

K_3

00	01	11	10
0	0	0	0
0	0	0	0
1	1	1	1
1	1	1	1

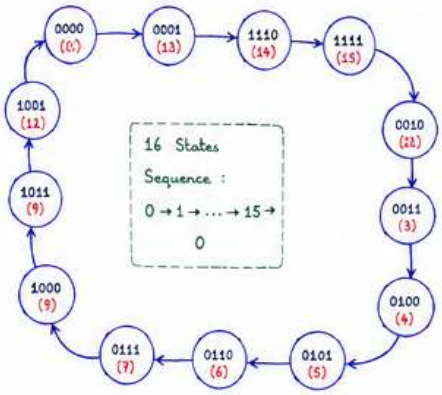
$J_0 = 1$, $K_0 = 1$

$J_1 = Q_0$, $K_1 = Q_0$

$J_2 = Q_1 \cdot Q_0$, $K_2 = Q_1 \cdot Q_0$

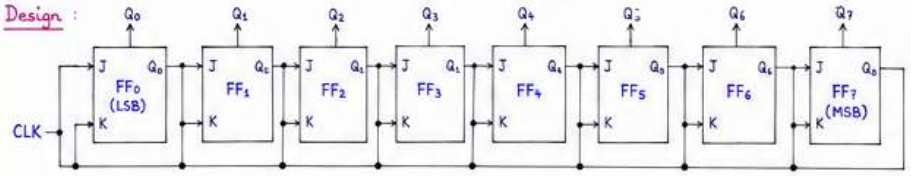
$J_3 = Q_2 \cdot Q_1 \cdot Q_0$, $K_2 = Q_2 \cdot Q_1 \cdot Q_0$

FSM (State Diagram)



2. 8 BIT SYNCHRONOUS UP COUNTER (MOD-256)

Design :



- 8 FFs (JK type)
- Common clock → synchronous
- Counts from 00000000_2 to 11111111_2

State Sequence

00000000 → 00000001
00000001 → 00000010
⋮
11111110 → 11111111
11111111 → 00000000

Total states = 256

Excitation / Input Expressions

Bit	J_i	K_i	Comment
J_0	1	1	Toggles every count
J_1	Q_0	Q_0	Toggles when $Q_0 = 1$
J_2	$Q_1 \cdot Q_0$	$Q_1 \cdot Q_0$	Toggles when $Q_1 \cdot Q_0 = 1$
J_3	$Q_2 \cdot Q_1 \cdot Q_0$	$Q_2 \cdot Q_1 \cdot Q_0$	Toggles when $Q_2 \cdot Q_1 \cdot Q_0 = 1$
J_4	$Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0$	$Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0$	...
⋮	⋮	⋮	⋮
J_7	$Q_6 \cdot Q_5 \cdot Q_4 \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0$	$Q_6 \cdot Q_5 \cdot Q_4 \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0$	Toggles when lower 7 bits = 1

Generalization

For i^{th} FF $(0 \le i \le 7)$:

$J_i = Q_{i-1} \cdot Q_{i-2} \cdot \dots \cdot Q_0$

$K_i = Q_{i-1} \cdot Q_{i-2} \cdot \dots \cdot Q_0$

(For $i = 0 \rightarrow J_0 = K_0 = 1$)

K-Map Pattern (Typical)

For J_3 and K_3 (function of $Q_2 Q_1 Q_0$)

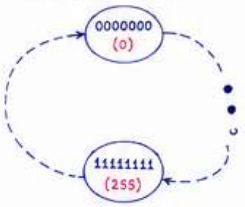
Q_2 \ Q_1Q_0	00	01	11	10
0	0	0	0	0

$J_3 = Q_2 \cdot Q_1 \cdot Q_0$

(Similar pattern extends for higher bits)

FSM (Conceptual)

Circular FSM with 256 states. Each state i transitions to state $(i + 1)$ mod 256.



Key Points

- Synchronous counters → all FFs triggered together by same clock.
- In up counter, LSB toggles every clock. Each higher bit toggles when all lower bits are 1.
- 4-bit → 16 states (MOD-16)
  8-bit → 256 states (MOD-256)

Advantages

- No ripple delay.
- Higher speed operation.
- Predictable timing.
- Easy to expand to n bits.

Applications

- Frequency division.
- Digital clocks / timers.
- Event counting.
- Sequence generation.
- Address generation.
- Data acquisition systems.

Notes

- X in excitation / characteristic table → don't care.
- K-maps used to minimize logic for JK inputs.
- FSM shows complete counting sequence.

Summary

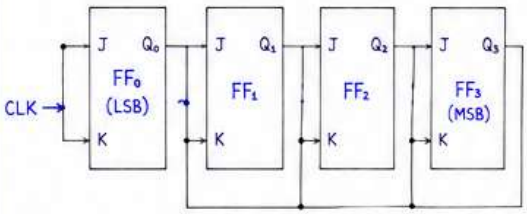
Synchronous counter uses n JK FFs with common clock. For i^{th} bit, $J_i = K_i$ = product of all lower order Q's. It counts 0 to $2^n - 1$ and repeats.

COUNTERS - ASYNCHRONOUS (RIPPLE) COUNTERS

1. ASYNCHRONOUS 4 BIT UP COUNTER (MOD-16)

Design :

Using 4 JK FFs in toggle mode ($J = K = 1$).



Each FF toggles on falling edge of previous stage output.

Characteristic Table (JK FF)

J	K	Q(t+1)	Operation
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}(t)$	Toggle

State Table (MOD-16)

PS				NS				Decimal
Q_3	Q_2	Q_1	Q_0	Q_{3+}	Q_{2+}	Q_{1+}	Q_{0+}	
0	0	0	0	→	0	0	1	0
0	0	0	1	→	0	1	0	1
0	0	1	0	→	0	1	1	2
0	0	1	1	→	0	1	0	3
0	1	0	0	→	0	0	1	4
0	1	0	1	→	0	1	0	5
0	1	1	0	→	1	1	1	6
0	1	1	1	→	1	0	0	7
1	0	0	0	→	1	0	1	8
1	0	0	1	→	1	0	0	9
1	0	1	0	→	1	0	1	10
1	0	1	1	→	1	1	0	11
1	1	0	0	→	1	1	1	12
1	1	0	1	→	1	1	1	13
1	1	0	1	→	1	1	0	13
1	1	1	0	→	1	1	1	14
1	1	1	1	→	0	0	0	15

Transition Table

Present State	Next State	Transition
0000	0001	0 → 1
0001	0010	1 → 2
0010	0011	2 → 3
0011	0100	3 → 4
0100	0101	4 → 5
0101	0110	5 → 6
0110	0111	6 → 7
0111	1000	7 → 8
1000	1001	8 → 9
1001	1010	9 → 10
1010	1011	10 → 11
1011	1100	11 → 12
1100	1101	12 → 13
1101	1110	13 → 14
1110	1111	14 → 15
1111	0000	15 → 0

K-Maps for J Inputs (all = K inputs)

J_0 (K_0)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$J_0 = 1$, $K_0 = 1$

J_1 (K_1)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00		1	1	0
01		1	1	0
11		1	1	0
10		1	1	0

$J_1 = Q_0$, $K_1 = Q_0$

J_2 (K_2)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00				1
01				1
11				1
10				1

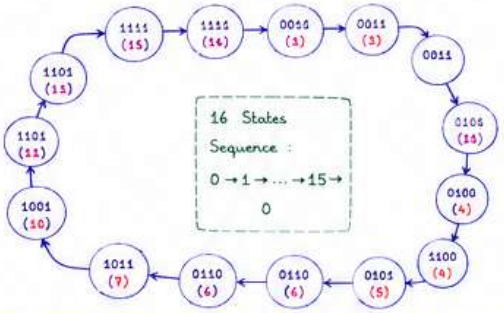
$J_2 = Q_1 \cdot Q_0$, $K_2 = Q_1 \cdot Q_0$

J_3 (K_3)

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00				
01				
11			1	
10			1	1

$J_3 = Q_2 \cdot Q_1 \cdot Q_0$, $K_3 = Q_2 \cdot Q_1 \cdot Q_0$

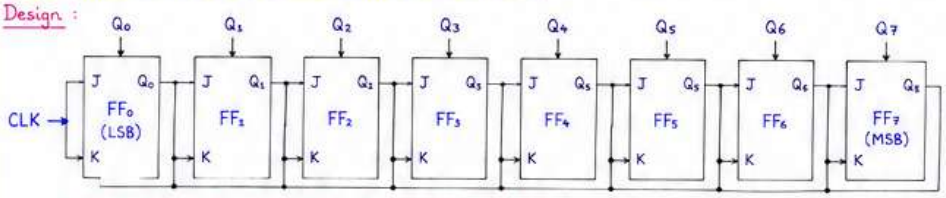
FSM (State Diagram)



---

2. ASYNCHRONOUS 8 BIT UP COUNTER (MOD-256)

Design :



Each FF toggles on falling edge of previous stage output.

Transition Table (8 bit)

Present State	Next State	Transition
00000000	00000001	0 → 1
00000001	00000010	1 → 2
...	...	...
01111111	10000000	127 → 128
...	...	...
11111110	11111111	254 → 255
11111111	00000000	255 → 0

Characteristic Table (JK FF)

J	K	Q(t+1)	Operation
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	$\bar{Q}(t)$	Toggle

Excitation Table (JK FF)

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

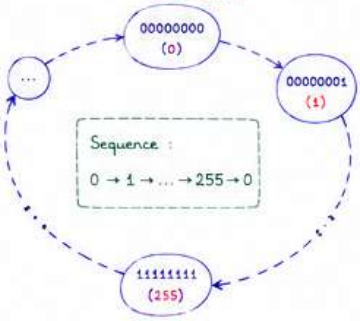
X → Don't care

State Table (MOD-256)

PS		NS		Decimal
Q_7 .... Q_0		Q_{7+} .... Q_{0+}		
0000	0000	→	0000 0001	0
0000	0001	→	0000 0010	1
...				
0111	1111	→	1000 0000	127
1000	0000	→	1000 0001	128
...			...	
1111	1110	→	1111 1111	254
1111	1111	→	0000 0000	255

FSM (Conceptual)

256 states



K-Maps for J (and K) Inputs

$J_0 = K_0$

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$J = 1$

$J_1 = K_1$

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	1	1	0
01	0	1	1	1
11	0	1	1	1
10	1	1	1	1

$= Q_0$

$J_2 = K_2$

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00	0	0	0	1
01	0			1
11	0	0		1
10	0	0	0	1

$= Q_1 \cdot Q_0$

$J_3 = K_3$

Q_3Q_2 \ Q_1Q_0	00	01	11	10
00				1
01				
11			1	1
10			1	1

$= Q_2 \cdot Q_1 \cdot Q_0$

...

$J_7 = K_7$

Q_6	00	01	11	10
00				
01				
11				
10				1

$J_7 = K_7 = Q_6 \cdot Q_5 \cdot Q_4 \cdot Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0$

TYPES (Asynchronous)

- Ripple counters.
- Clock applied to only first FF.
- Each FF triggered by previous FF output.
- Simple but has ripple delay.

DESIGN STEPS

1. Define MOD number (N states).
2. No. of FFs = n such that $2^n \geq N$.
3. Use JK FFs with $J = K = 1$.
4. Connect FFs in ripple ($Q \rightarrow CLK$).
5. Draw state & transition tables.
6. Obtain excitation table.
7. Minimize using K-maps (if needed).
8. Draw circuit & FSM.

ADVANTAGES

- Very simple & easy to design.
- Requires minimum gates.
- Good for low speed / general purpose.

APPLICATIONS

- Frequency division.
- Time delay generation.
- Event counting.
- Sequence generation.
- Digital clocks.
- Traffic light controllers.

NOTES

- Asynchronous counters are also called Ripple counters.
- More stages → more delay.
- Not suitable for high speed systems.

KEY POINTS

- 4-bit ripple counter → 16 states (MOD-16).
- 8-bit ripple counter → 256 states (MOD-256).
- Each bit toggles when all lower bits are 1.

SHIFT REGISTERS (GENERAL)

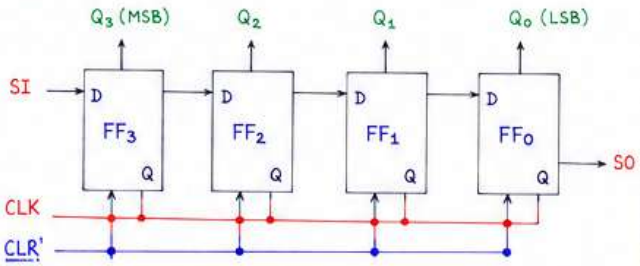
1. INTRODUCTION

- ★ A shift register is a group of flip-flops connected in cascade so that data is shifted from one stage to the next on each clock pulse.
- ★ All flip-flops share a common clock.
- ★ Data moves either Left → or Right ← depending on connection.
- ★ Operations :
  1) Serial In (SI)
  2) Serial Out (SO)
  3) Hold (no shift)
  4) Clear / Reset

Symbols :
D_i – Data input at stage i
Q_i – Output of stage i
CLK – Clock
CLR – Asynchronous clear (active low)

2. DESIGN OF 4-BIT SHIFT REGISTER (SISO)

Right Shift (Data moves → Right)



On each CLK ↑ : $Q_3 \leftarrow Q_2$, $Q_2 \leftarrow Q_1$, $Q_1 \leftarrow Q_0$, $SO = Q_0$

(Left shift can be obtained by reversing connections)

3. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

Characteristic Equation : $Q(t+1) = D$

5. STATE TABLE OF 4-BIT REGISTER (SISO)

PS (Q_3 Q_2 Q_1 Q_0)	NS (After one clock) (Q_{3+} Q_{2+} Q_{1+} Q_{0+})	SO (Q_0)
0 0 0 0	0 0 0 0	0
0 0 0 1	0 0 0 0	1
0 0 1 0	0 0 0 1	0
0 0 1 1	0 0 1 0	1
⋮ ⋮	⋮ ⋮	⋮
1 1 1 0	0 1 1 1	0
1 1 1 1	1 1 1 1	1

If SI = 0 : zeros shift in from left
If SI = 1 : ones shift in from left

6. TRANSITION TABLE (Right Shift, SI input)

PS (Q_3 Q_2 Q_1 Q_0)	SI	NS (Q_{3+} Q_{2+} Q_{1+} Q_{0+})	SO (Q_0)
abcd	0	0 a b c	d
abcd	1	1 a b c	d
abcd	X	a b c d	d

a, b, c, d = 0 or 1 X = Don't care

7. K-MAPS FOR D INPUTS (4-BIT)

For D_3 (since $Q_{3+} = SI$)

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$D_3 = SI$

For D_2 ($Q_{2+} = Q_3$)

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

$D_2 = Q_3$

For D_1 ($Q_{1+} = Q_2$)

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

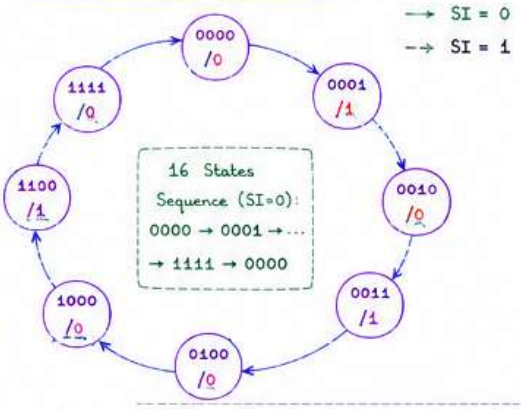
$D_1 = Q_2$

For D_0 ($Q_{0+} = Q_1$)

Q_1Q_0 \ Q_3Q_2	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

$D_0 = Q_1$

8. FSM (STATE DIAGRAM)



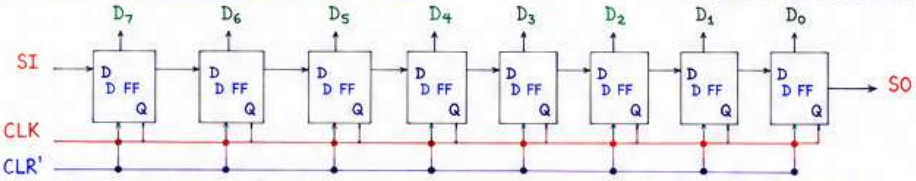
9. DESIGN SUMMARY (4-BIT)

- ★ 4 D flip-flops in cascade.
- ★ Common CLK to all FFs.
- ★ SI applied to MSB (leftmost).
- ★ For right shift :

$D_3 = SI$, $D_2 = Q_3$, $D_1 = Q_2$, $D_0 = Q_1$

- ★ SO = Q_0 (LSB output).
- ★ CLR' clears all FFs to 0.

10. EXTENSION TO 8-BIT REGISTER



State Table (8-bit, SI=0)

PS ($Q_7 \cdots Q_0$)	NS ($Q_{7+} \cdots Q_{0+}$)	SO (Q_0)
0000 0000	0000 0000	0
0000 0001	0000 0000	1
⋮ ⋮	⋮ ⋮	⋮
1111 1110	0111 1111	0
1111 1111	1111 1111	1

Transition Table (8-bit)

PS ($Q_7 \cdots Q_0$)	SI	NS ($Q_{7+} \cdots Q_{0+}$)	SO (Q_0)
$A_7 \ldots A_0$	0	0 $A_7 \cdots A_1$	A_0
$A_7 \ldots A_0$	1	1 $A_7 \cdots A_1$	A_0
$A_7 \ldots A_0$	X	$A_7 \cdots A_0$	A_0

A_i = 0 or 1 , X = Don't care

11. GENERAL EQUATIONS (n-bit SISO)

Right shift :
$D_{n-1} = SI$
$D_i = Q_{i+1}$ for $0 \le i < n-1$
$SO = Q_0$

Left shift :
$D_0 = SI$
$D_i = Q_{i-1}$ for $1 \le i < n$
$SO = Q_{n-1}$

KEY POINTS

- ★ Registers store n-bit data.
- ★ Data moves on clock edge.
- ★ D FF excitation : D = Q(t+1).
- ★ K-maps give direct minimal Boolean inputs.

ADVANTAGES

- ★ Simple and regular design.
- ★ High speed (parallel clock).
- ★ Easy to expand to n-bits.
- ★ Used in buffers, FIFOs, comms.

APPLICATIONS

- ★ Temporary data storage.
- ★ Serial/parallel conversion.
- ★ Data transfer between units.
- ★ Timing delay lines.
- ★ Digital signal processing.

INTERVIEW / VIVA POINT

Q: What is the purpose of shift register ?
A: To store and move data serially/parallel on clock.
Q: Characteristic of D FF ?
A: Q(t+1) = D.

REGISTERS (GENERAL)

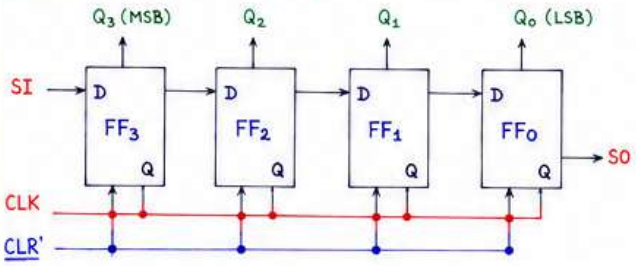
1. INTRODUCTION

- A register is a group of flip-flops connected in cascade so that data is stored and moved on clock pulses.
- All flip-flops in a register share a common clock.
- Data can move either Left → or Right ← depending on connections.
- Operations :
  1) Serial In (SI)
  2) Serial Out (SO)
  3) Hold (no shift)
  4) Clear / Reset

Symbols :
- D_i - Data input at stage i
- Q_i - Output of stage i
- CLK - Clock
- CLR' - Asynchronous clear (active low)

2. DESIGN OF 4 - BIT REGISTER (SISO)

Right Shift (Data moves → Right)



On each CLK ↑ : $Q_3 \leftarrow Q_2$, $Q_2 \leftarrow Q_1$, $Q_1 \leftarrow Q_0$, $SO = Q_0$

(Left shift can be obtained by reversing connections)

3. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

Characteristic Equation : $Q(t+1) = D$

5. STATE TABLE OF 4-BIT REGISTER (SISO)

PS ($Q_3\ Q_2\ Q_1\ Q_0$)	NS (After one clock) ($Q_{3+}\ Q_{2+}\ Q_{1+}\ Q_{0+}$)	SO (Q_0)
0 0 0 0	0 0 0 0	0
0 0 0 1	0 0 0 0	1
0 0 1 0	0 0 0 1	0
0 0 1 1	0 0 1 0	1
: :	: :	:
1 1 1 0	0 1 1 1	0
1 1 1 1	1 1 1 1	1

If SI = 0 : zeros shift in from left

If SI = 1 : ones shift in from left

6. TRANSITION TABLE (Right Shift, SI input)

PS ($Q_3\ Q_2\ Q_1\ Q_0$)	SI	NS ($Q_{3+}\ Q_{2+}\ Q_{1+}\ Q_{0+}$)	SO (Q_0)
a b c d	0	0 a b c	d
a b c d	1	1 a b c	d
a b c d	X	a b c d	d

a, b, c, d = 0 or 1 X = Don't care

7. K-MAPS FOR D INPUTS (4-BIT)

For D_3 (since $Q_{3+} = SI$)

Q_3Q_0 \ Q_2Q_1	00	01	11	10
00	1	1	1	
01	1	1	1	
11	1	1	1	
10	1	1	1	

$D_3 = SI$

For D_2 ($Q_{2+} = Q_3$)

Q_3Q_0 \ Q_2Q_1	00	01	11	10
00	0	0	1	1
01	0	0	1	1
11	0	0	1	1
10	0	0	1	1

$D_2 = Q_3$

For D_1 ($Q_{1+} = Q_2$)

Q_3Q_0 \ Q_2Q_1	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	1
10	0	0	1	0

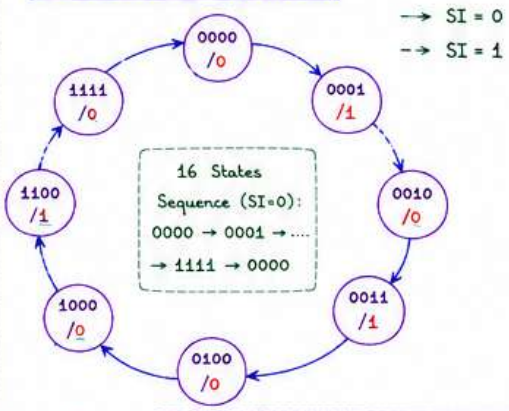
$D_1 = Q_2$

For D_0 ($Q_{0+} = Q_2$)

Q_3Q_0 \ Q_2Q_1	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	0
10	0	0	1	0

$D_0 = Q_1$

8. FSM (STATE DIAGRAM)



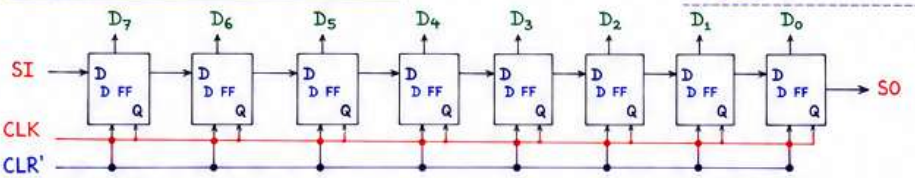
9. DESIGN SUMMARY (4-BIT)

- 4 D flip-flops in cascade.
- Common CLK to all FFs.
- SI applied to MSB (leftmost).
- For right shift :

$D_3 = SI$, $D_2 = Q_3$, $D_1 = Q_2$, $D_0 = Q_1$

- SO = Q_0 (LSB output).
- CLR' clears all FFs to 0.

10. EXTENSION TO 8-BIT REGISTER



State Table (8-bit, SI=0)

PS ($Q_7 \cdots Q_0$)	NS ($Q_{7+} \cdots Q_{0+}$)	SO (Q_0)
0000 0000	0000 0000	0
0000 0001	0000 0000	1
: :	: :	:
1111 1110	0111 1111	0
1111 1111	1111 1111	1

Transition Table (8-bit)

PS ($Q_7 \cdots Q_0$)	SI	NS ($Q_{7+} \cdots Q_{0+}$)	SO (Q_0)
$A_7 \cdots A_0$	0	0 $A_7 \cdots A_1$	A_0
$A_7 \cdots A_0$	1	1 $A_7 \cdots A_1$	A_0
$A_7 \cdots A_0$	X	$A_7 \cdots A_0$	A_0

A_i = 0 or 1 , X = Don't care

11. GENERAL EQUATIONS (n-bit SISO)

Right shift :

$D_{n-1} = SI$

$D_i = Q_{i+1}$ for $0 \le i < n-1$

$SO = Q_0$

Left shift :

$D_0 = SI$

$D_i = Q_{i-1}$ for $1 \le i < n$

$SO = Q_{n-1}$

KEY POINTS
- Registers store n-bit data.
- Data moves on clock edge.
- D FF excitation : D = Q(t+1).
- K-maps give minimal Boolean inputs.

ADVANTAGES
- Simple & regular design.
- High speed (parallel clock).
- Easy to expand to n-bits.
- Used in buffers, FIFOs, communication.

APPLICATIONS
- Temporary data storage
- Serial/parallel conversion
- Data transfer between units
- Timing delay lines
- Digital signal processing

INTERVIEW / VIVA POINT
Q: Purpose of a shift register ?
A: To store and move data.
Q: Characteristic of D FF ?
A: SO = Q_0 = D
Q: Output in right shift ?
A: Q(t+1) = D

Author : Konda Tharun Vivekananda

SHIFT REGISTERS - TYPES (SISO, SIPO)

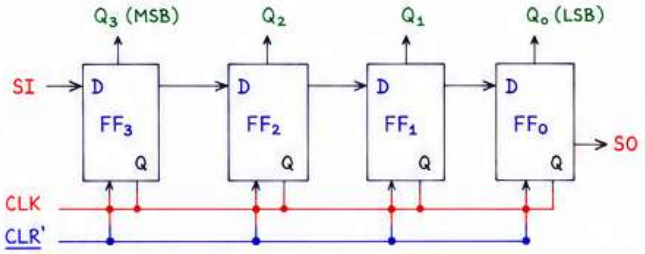
1. INTRODUCTION

- ★ Shift register is a group of D flip-flops connected in cascade.
- ★ Data is shifted from one stage to the next on each clock pulse.
- ★ Two basic types (based on I/O): 1) SISO 2) SIPO
- ★ Here we design 4-bit registers using D flip-flops only.

Symbols :
SI - Serial In
SO - Serial Out
Q_i - Output of stage i
CLK - Clock
CLR' - Asynchronous clear (active low)

A. SISO (Serial In - Serial Out)

Right shift (data moves → right)



On each CLK ↑ : $Q_3 \leftarrow Q_2$, $Q_2 \leftarrow Q_1$, $Q_1 \leftarrow Q_0$, $Q_0 \leftarrow SI$

$SO = Q_0$

2. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

Characteristic Equation : $Q(t+1) = D$

3. STATE TABLE OF 4-BIT REGISTER (SISO)

PS (Q_3 Q_2 Q_1 Q_0)	NS (After one clock) (Q_{3+} Q_{2+} Q_{1+} Q_{0+})	SO (Q_0)
0 0 0 0	0 0 0 0	0
0 0 0 1	0 0 0 1	1
0 0 1 1	0 0 0 1	0
0 0 1 1	1 0 0 1	1
:	:	:
1 1 0 0	0 1 1 0	0
1 1 1 1	1 1 1 0	1

If SI = 0 → 0 shifts in from left

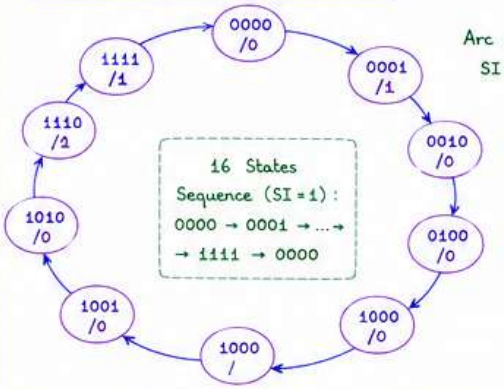
If SI = 1 → 1 shifts in from left

4. TRANSITION TABLE (SISO) - SISO

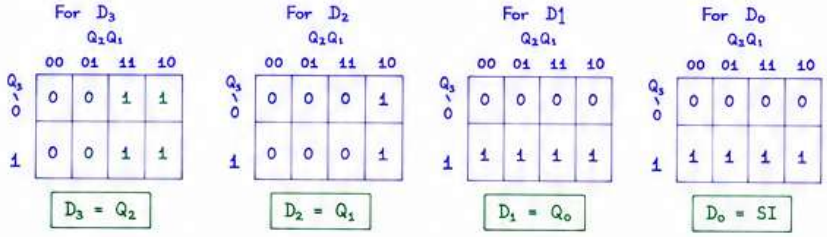
PS (Q_3 Q_2 Q_1 Q_0)	SI	NS (Q_{3+} Q_{2+} Q_{1+} Q_{0+})	SO (Q_0)
a b c d	0	0 a b c	d
a b c d	1	1 a b c	d
a b c d	X	X X X X	d

a, b, c, d = 0 or 1 X = Don't care

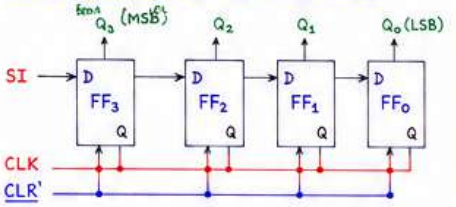
3. FSM (STATE DIAGRAM - SISO)



7. K-MAPS FOR D INPUTS (SISO, Right shift)



B. SIPO (Serial In - Parallel Out)



On each CLK ↑ : $Q_3 \leftarrow Q_2$, $Q_2 \leftarrow Q_1$, $Q_1 \leftarrow Q_0$, $Q_0 \leftarrow SI$

Parallel outputs are available at Q_3 Q_2 Q_1 Q_0.

(After 4 clock pulses, 4 serial bits appear in parallel)

8. STATE TABLE (SIPO)

PS (Q_3 Q_2 Q_1 Q_0)	SI	NS (Q_{3+} Q_{2+} Q_{1+} Q_{0+})
0 0 0 0	0	0 0 0 0
0 0 0 0	1	0 0 0 1
0 0 1 1	0	0 0 0 1
0 0 1 1	1	1 0 0 1
:		:
1 1 0 1	0	0 1 1 0
1 1 0 1	1	1 1 1 0

9. EXCITATION TABLE (D FF)

PS (Q)	NS (Q_+)	D
0	0	0
0	1	1
1	0	0
1	1	1

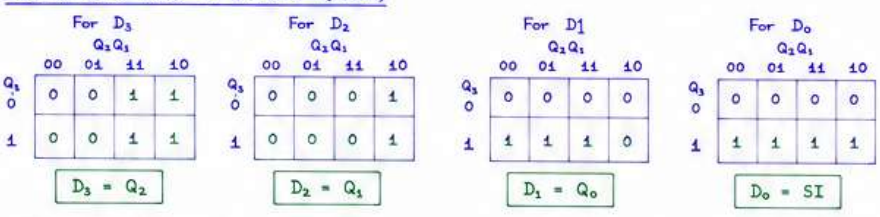
$D = Q_+$

10. TRANSITION TABLE (SIPO)

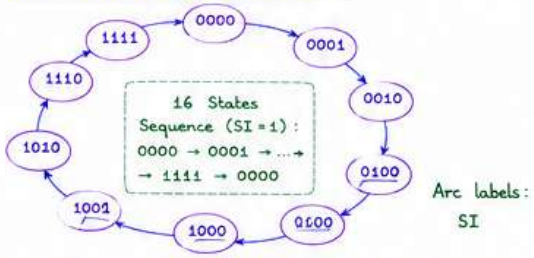
PS (Q_3 Q_2 Q_1 Q_0)	SI	NS (Q_{3+} Q_{2+} Q_{1+} Q_{0+})
a b c d	0	0 a b c
a b c d	1	1 a b c
a b c d	X	X X X X

a, b, c, d = 0 or 1 X = Don't care

11. K-MAPS FOR D INPUTS (SIPO)



12. FSM (STATE DIAGRAM - SIPO)



KEY POINTS

- ★ Data shifts on each clock ↑.
- ★ In SISO → only serial O/P (SO).
- ★ In SIPO → parallel O/P (Q_3..Q_0).
- ★ D FF excitation : $D = Q_+$.

ADVANTAGES

- ★ Simple and regular structure.
- ★ Easy to implement.
- ★ Useful for data transfer and storage applications.

APPLICATIONS

- ★ Serial/parallel data conversion.
- ★ Data buffering.
- ★ Keyboards, communication links.
- ★ Temporary data storage.

INTERVIEW / VIVA POINT

Q: What is the difference between SISO and SIPO shift registers ?

A: SISO has one serial output, SIPO has parallel outputs.

Author : Konda Tharun Vivekananda

Shift Registers - Types (PISO, PIPO)

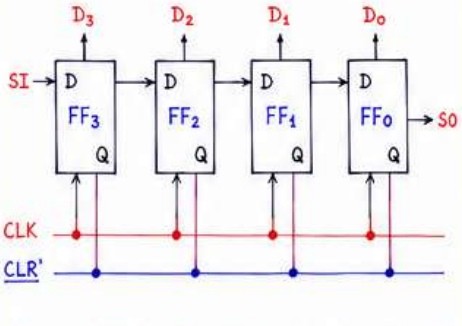
1. INTRODUCTION

- ★ Shift register is a sequential circuit made of cascade of flip-flops.
- ★ Data is shifted or stored and available in serial or parallel form.
- ★ Four basic types based on I/O: (i) SISO (ii) SIPO (iii) PISO (iv) PIPO.
- ★ Here we study PISO and PIPO using D flip-flops.

Symbols :
PI - Parallel In
SO - Serial Out
PO - Parallel Out
Qi - Output of stage i
CLK - Clock
CLR' - Asynchronous clear (active low)

A. PISO (Parallel In - Serial Out)

Right shift (data moves → right)



On each CLK ↑ : $Q_3 \to Q_2$, $Q_2 \to Q_1$, $Q_1 \to Q_0$, $Q_0 \to SO$, D_3, D_2, D_1, D_0 are loaded in parallel.

2. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

3. EXCITATION TABLE (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

Characteristic Equation : $Q(t+1) = D$

4. K-MAP FOR D INPUT

Q(t) \ Q(t+1)	0	1
0	0	1
1	0	1

Minimized : $D = Q(t+1)$

(Same as excitation equation)

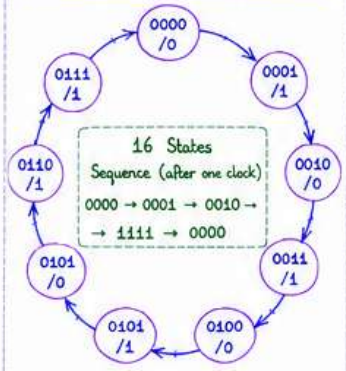
5. STATE TABLE (PISO)

PS (Q_3, Q_2, Q_1, Q_0)	Inputs (D_3, D_2, D_1, D_0)	NS $(Q_{3+}, Q_{2+}, Q_{1+}, Q_{0+})$	SO (Q_{0+})
0000	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	d_0
0001	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	d_0
0010	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	d_0
0011	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	d_0
⋮	⋮	⋮	⋮
1110	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_0\ d_0$	d_0
1111	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_0\ d_2\ d_0$	d_0

6. TRANSITION TABLE (PISO)

PS (Q_3, Q_2, Q_0)	Inputs (D_3, D_2, D_1, D_0)	NS $(Q_{3+}, Q_{2+}, Q_{1+}, Q_{0+})$	SO (Q_{0+})
abcd	w x y z	w x y z	z
abcd	w x y z	w x z z	z
abcd	w x y z	w z z z	z
abcd	w x y z	w z z z	z
⋮	⋮	⋮	⋮
1110	w x y z	z z z z	z
1111	w x y z	z z z z	z

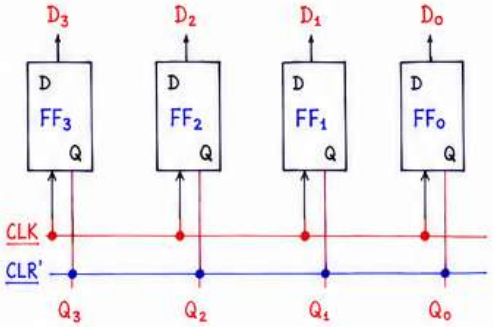
7. FSM (STATE DIAGRAM - PISO)



Arc labels : $D_3\ D_2\ D_1\ D_0$ / SO

B. PIPO (Parallel In - Parallel Out)

No shift - all bits loaded together



On each CLK ↑ : $Q_3 = D_3$, $Q_2 = D_2$, $Q_1 = D_1$, $Q_0 = D_0$

(All bits loaded in parallel and appear in parallel)

2. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

3. EXCITATION TABLE (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

$Q(t+1) = D$

$D = Q(t+1)$

4. K-MAP FOR D INPUT

Q(t) \ Q(t+1)	0	1
0	0	1
1	0	1

Minimized : $D = Q(t+1)$

(Same as excitation.)

5. STATE TABLE (PIPO)

PS (Q_3, Q_2, Q_1, Q_0)	Inputs (D_3, D_2, D_1, D_0)	NS $(Q_{3+}, Q_{2+}, Q_{1+}, Q_{0+})$	PO $(Q_{3+}, Q_{2+}, Q_{1+}, Q_{0+})$
0000	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$
0001	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$
0010	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$
0011	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$
⋮	⋮	⋮	⋮
1111	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$	$d_3\ d_2\ d_1\ d_0$

6. TRANSITION TABLE (PIPO)

PS (Q_3, Q_2, Q_1, Q_0)	Inputs (D_3, D_2, D_1, D_0)	NS $(Q_{3+}\ Q_{2+}\ Q_{1+}\ Q_{0+})$	PO $(Q_{3+}\ Q_{2+}\ Q_{1+}\ Q_{0+})$
abcd	w x y z	w x y z	w x y z
abcd	w x y z	w x y z	w x y z
abcd	w x y z	w x y z	w x y z
⋮	⋮	⋮	⋮
abcd	w x y z	w x y z	w x y z

a, b, c, d , w, x, y, z = 0 or 1

7. FSM (STATE DIAGRAM - PIPO)

ALL 16 states exist but there is NO fixed sequence between states.
Next state equals the input pattern.
PS $(Q_3, Q_2, Q_1, Q_0) \to (D_3, D_2, D_1, D_0) \to$ NS (Same as input)

Arc labels : $D_3\ D_2\ D_1\ D_0$
(Output at any state = current PS)

SUMMARY / KEY POINTS

- ★ PISO : Loads 4 bits in parallel and shifts right, gives serial output from Q_0.
- ★ PIPO : Loads 4 bits in parallel and gives all outputs in parallel.
- ★ D flip-flop : $Q(t+1) = D$, $D = Q(t+1)$
- ★ In PISO, after 4 clocks all bits are shifted out.
- ★ In PIPO, outputs are available immediately after clock.

ADVANTAGES

- ★ Simple hardware (D FF cascade).
- ★ Synchronous operation.
- ★ Flexibility of serial / parallel data conversion.
- ★ Useful in data buffering and interface circuits.

APPLICATIONS

- ★ Serial data transmission (PISO).
- ★ Parallel data transfer (PIPO).
- ★ Data buffering / temporary storage.
- ★ Interface in microprocessor systems.
- ★ Communication links.

INTERVIEW / VIVA POINT

Q: What is difference between PISO and PIPO shift registers ?
A: PISO gives serial output after shifting, whereas PIPO gives all outputs in parallel after single clock.

SHIFT REGISTERS COUNTERS - TYPES (RING, JOHNSON)

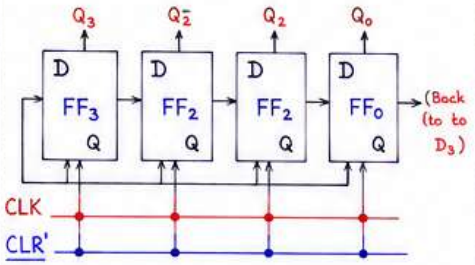
1. INTRODUCTION

- ★ Shift register counterg is a multi-bit sequential circuit obtained by closing the shift register into a loop.
- ★ Two important types :
  - (i) Ring counter
  - (ii) Johnson (Twisted ring) counter

Symbols :
- D - D flip-flop
- Q_i - Output of stage i
- CLK - Clock
- CLR' - Asynchronous clear (active low)

A. RING COUNTER

One '1' circulates in a ring.



On each CLK ↑ : $Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0 \rightarrow Q_3$

2. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

Characteristic Eqn : $Q(t+1) = D$

3. EXCITATION TABLE (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Eqn : $D = Q(t+1)$

4. STATE TABLE (RING COUNTER)

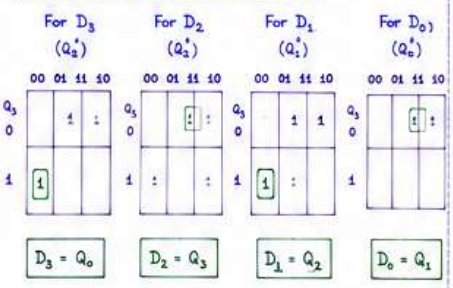
PS ($Q_3\ Q_2\ Q_1\ Q_0$)	NS (After one clock)	Comments
1 0 0 0	0 1 0 0	'1' moves right
0 1 0 0	0 0 1 0	-
0 0 1 0	0 0 0 1	-
0 0 0 1	1 0 0 0	-

Total = 4 states

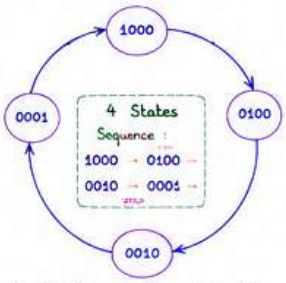
5. TRANSITION TABLE (RING)

PS ($Q_3\ Q_2\ Q_1\ Q_0$)	Inputs	NS ($Q_3^+\ Q_2^+\ Q_1^+\ Q_0^+$)
1 0 0 0	-	0 1 0 0
0 1 0 0	-	0 0 1 0
0 0 1 0	-	0 0 0 1
0 0 0 1	-	1 0 0 0

6. K-MAPS FOR D INPUTS (RING)

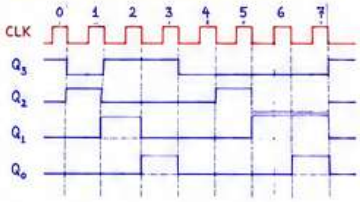


7. FSM (STATE DIAGRAM - RING)



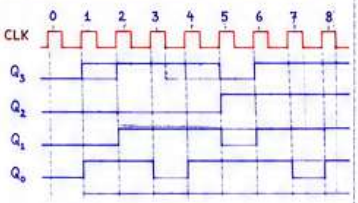
Arc labels : - (unconditional)

8. EXAMPLE WAVEFORM (RING COUNTER)



Only one output is '1' at a time.

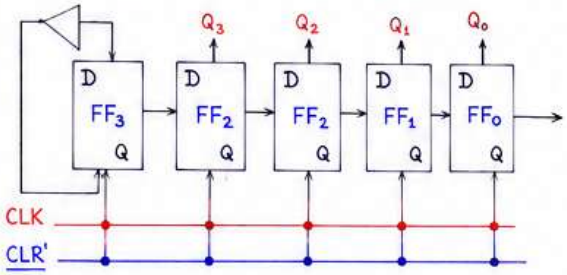
9. EXAMPLE WAVEFORM (JOHNSON COUNTER)



Number of states = 2^n (here n=4 → 8)

B. JOHNSON (TWISTED RING) COUNTER

Complement of last stage is fed back.



On each CLK ↑ : $Q_3 \rightarrow Q_2 \rightarrow Q_1 \rightarrow Q_0 \rightarrow \overline{Q}_0 \rightarrow D_3$

2. CHARACTERISTIC TABLE (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

Characteristic Eqn : $Q(t+1) = D$

3. EXCITATION TABLE (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Eqn : $D = Q(t+1)$

4. STATE TABLE (JOHNSON)

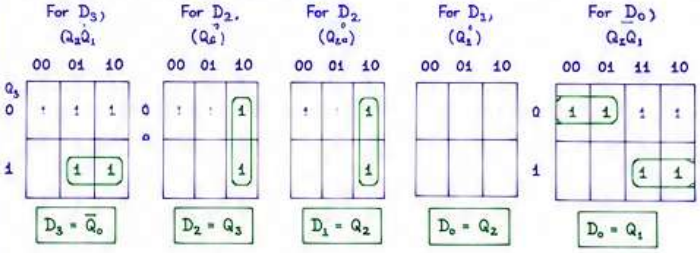
PS ($Q_3\ Q_2\ Q_1\ Q_0$)	NS (After one clock)
0 0 0 0	0 0 0 1
0 0 0 1	0 0 1 1
0 0 1 1	0 1 1 1
0 1 1 1	1 1 1 1
1 1 1 1	1 1 1 0
1 1 1 0	1 1 0 0
1 1 0 0	1 0 0 0
1 0 0 0	0 0 0 0

Total = 8 states

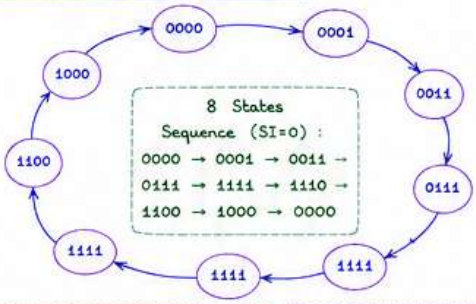
5. TRANSITION TABLE (JOHNSON)

PS ($Q_3\ Q_2\ Q_1\ Q_0$)	Inputs	NS ($Q_3^+\ Q_2^+\ Q_1^+\ Q_0^+$)
0 0 0 0	-	0 0 0 1
0 0 0 1	-	0 0 1 1
0 0 1 1	-	0 1 1 1
0 1 1 1	-	1 1 1 1
1 1 1 1	-	1 1 1 0
1 1 1 0	-	1 1 0 0
1 1 0 0	-	1 0 0 0
1 0 0 0	-	0 0 0 0

6. K-MAPS FOR D INPUTS (JOHNSON)



7. FSM (STATE DIAGRAM - JOHNSON)



Arc labels : - (unconditional)

SUMMARY / KEY POINTS

- ★ Ring counter has n states (here 4). One '1' circulates.
- ★ Johnson counter has 2n states (here 8).
- ★ Both use D flip-flops with common CLK.
- ★ D FF excitation : $D = Q(t+1)$
- ★ Ring : $D_3 = Q_0$, $D_2 = Q_3$, $D_1 = Q_2$, $D_0 = Q_1$
- ★ Johnson : $D_3 = \overline{Q}_0$, $D_2 = Q_3$, $D_1 = Q_2$, $D_0 = Q_1$

ADVANTAGES

- ★ Simple and regular structure.
- ★ No external gating required.
- ★ Ring counter is economical in hardware (n states).
- ★ Johnson counter gives more states (2n) for same hardware.

APPLICATIONS

- ★ Sequence generation.
- ★ Timing / control circuits.
- ★ Ring counter : one-hot signaling.
- ★ Johnson counter : counters, state sequencing.
- ★ Used in embedded controllers, communication, testing.

INTERVIEW / VIVA POINT

Q : What is the difference between Ring and Johnson counters ?

A : Ring counter has n states with one '1' circulating. Johnson counter has 2n states with complement feedback.

Frequency Divider and Multiplier

1. Introduction

- Frequency divider gives an output frequency $f_{out} < f_{in}$
- Frequency multiplier gives an output frequency $f_{out} > f_{in}$
- Implemented using counters (dividers) and flip-flops / logic (multipliers)

Useful Relations

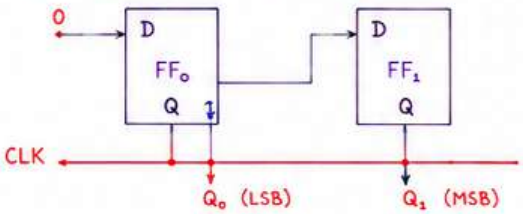
Divider : $f_{out} = \frac{f_{in}}{N}$

Multiplier : $f_{out} = N \times f_{in}$

where N = divide / multiply factor

2. Frequency Divider

- A counter with N states generates an output that repeats once in N clock cycles
- Example design : Divide by-4 using 2-bit counter



1. Characteristic Table (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

2. Excitation Table (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

3. State Table (Divide-by-4)

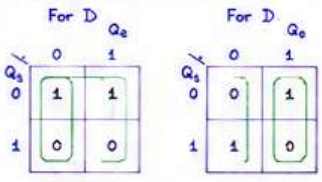
PS (Q_1 Q_0)	NS (Q_1' Q_0')	f_{out} (f_{out} = Q_0)	Comments
0 0	0 1	0	Start
0 1	1 0	0	High for 2 cycles
1 0	1 1	1	
1 1	0 0	1	

Sequence : 00 → 01 → 10 → 11 → 00 → ...

4. Transition Table

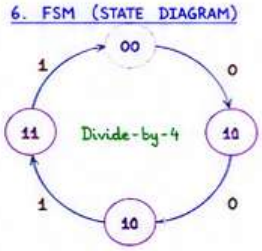
PS (Q_1 Q_0)	NS (Q_1' Q_0')
0 0	0 1
0 1	1 0
1 0	1 1
1 1	0 0

5. K-Maps (D Inputs)



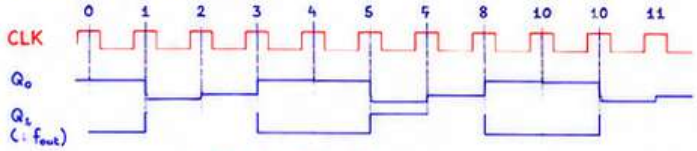
$D_0 = \overline{Q_1}$ $D_1 = Q_0$

6. FSM (State Diagram)



(Output $f_{out} = f_{in}/4$)

7. Waveform (Divide-by-4)



Here $f_{out} = f_{in}/4$

4. Design Steps - Summary

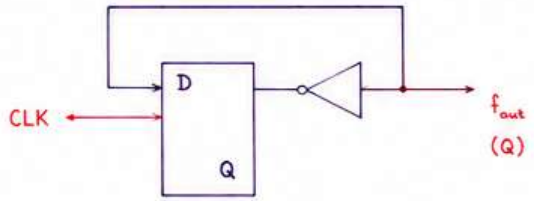
(1) Determine required factor N
(2) Decide divide or multiply
(3) Choose suitable counter / FF configuration
(4) Derive sequence / states
(5) Draw state diagram
(6) Obtain logic or FF design
(7) Verify with waveform

Key Point

Any counter that has N states acts as a divide-by-N circuit.

3. Frequency Multiplier

- Output frequency is an integer multiple (N times) of input frequency
- Example design : 2× Multiplier using D-FF and combinational logic



1. Characteristic Table (D FF)

D	Q(t)	Q(t+1)	Operation
0	0	0	No change
0	1	0	Reset
1	0	1	Set
1	1	1	No change

2. Excitation Table (D FF)

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

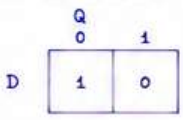
3. State Table (2× Multiplier)

PS fS (Q)	NS NS (Q')	f_{out} (f_{out} = Q)	Comments
0	1	0	Toggles every clock ↑
1	0	1	

4. Transition Table

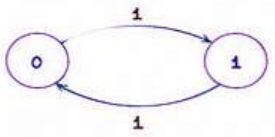
PS fS (Q)	NS NS (Q')
0	1
1	0

5. K-Map for D



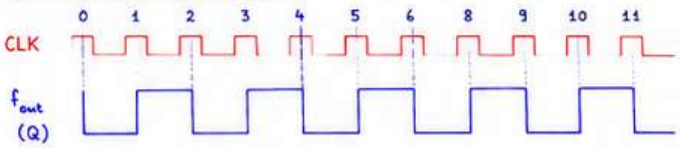
$D = \overline{Q}$

6. FSM (State Diagram)



(Output toggles every cycle)

7. Waveform (2× Multiplier)



Here $f_{out} = 2 \times f_{in}$

Advantages

- Simple and low cost
- Wide range of frequency control
- Easy to implement using counters / flip-flops

Limitations

- Divider output limited to integer division
- Multiplier gives integer multiples only

Applications

- Frequency dividers : clock division, timers, baud rate generators
- Multipliers : clock synthesis, PLL circuits, communication

Interview / Viva Point

Q : What is the relation between input and output frequency of a divide-by-N circuit ?

A : $f_{out} = \frac{f_{in}}{N}$

EDGE DETECTOR CIRCUIT

1. INTRODUCTION

* Edge detector produces short pulses of one clock period when a transition (edge) occurs in the input signal.
* Used in counters, timers, debouncers, interrupts, event detection etc.
* Two types of edges :
  (i) Rising edge (0 → 1)
  (ii) Falling edge (1 → 0)

Key Point

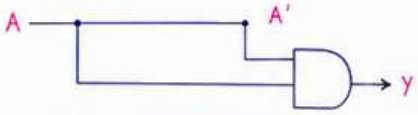
Edge detector converts a level change into a one-clock pulse.

2. EDGE TYPES

Edge Type	Condition	Detection Pulse
Rising Edge	0 → 1	Pulse at the rising instant
Falling Edge	1 → 0	Pulse at the falling instant

3. COMBINATIONAL EDGE DETECTOR

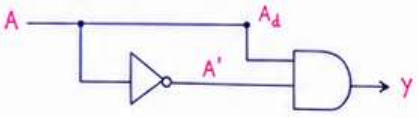
(i) Rising Edge Detector (Non-clocked)



$Y_{(rising)} = A \cdot \overline{A_d}$

Steps
(1) Invert A
(2) Delay A by small amount
(3) AND A with delayed complement

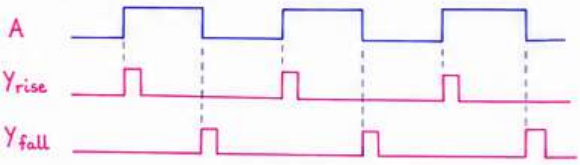
(ii) Falling Edge Detector (Non-clocked)



$Y_{(falling)} = A' \cdot A_d$

Steps
(1) Invert A
(2) Delay A
(3) AND delayed A with A'

Waveforms (Combinational)

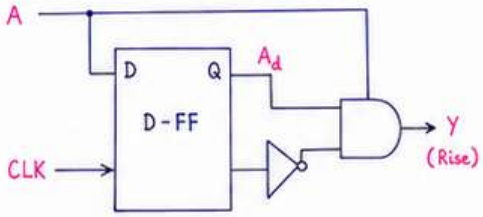


Advantages (Comb.)
* Simple and fast.
* No clock required.

Limitations (Comb.)
* Glitches due to gate delays.
* Pulse width depends on delay and not constant.
* Sensitive to noise.

4. SEQUENTIAL EDGE DETECTOR (CLOCKED)

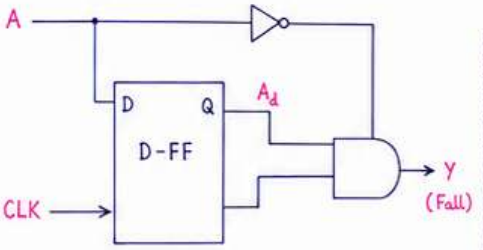
(i) Rising Edge Detector (Using D-FF)



$Y_{rise} = A \cdot \overline{A_d}$

Steps
(1) Store previous value of A in D-FF (A_d).
(2) Invert A_d.
(3) AND A with $\overline{A_d}$.

(ii) Falling Edge Detector (Using D-FF)



$Y_{fall} = \overline{A} \cdot A_d$

Steps
(1) Store A in D-FF.
(2) Invert present A.
(3) AND $\overline{A}$ with A_d.

5. TRUTH TABLES (Sequential)

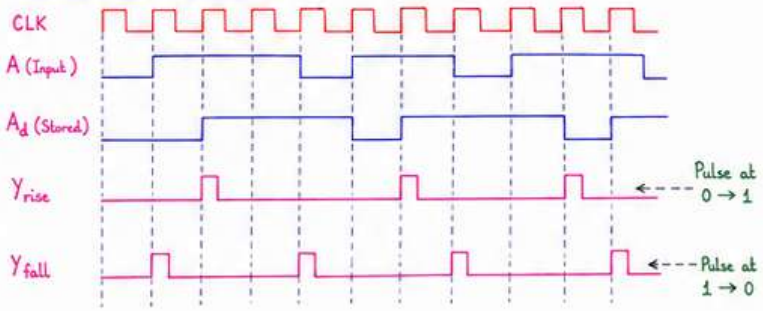
(i) Rising Edge Detector

A	A_d (prev)	Y_{rise}
0	0	0
0	1	0
1	0	1
1	1	0

(ii) Falling Edge Detector

A	A_d (prev)	Y_{fall}
0	0	0
0	1	1
1	0	0
1	1	0

6. WAVEFORMS (Sequential)



Design Steps (General)
1. Identify edge type (rising / falling).
2. Decide implementation (combinational / sequential).
3. Write output equation.
4. Draw logic diagram.
5. Verify truth table.
6. Check waveform.

Applications
* Interrupt generation
* Event / pulse detection
* Switch debouncing
* Counters & timers
* Frequency measurement

Summary / Key Points
* Edge detector → converts level transition into one-clock pulse.
* Sequential design gives glitch-free pulses.
* Two edges : Rising (0→1) and Falling (1→0).

Interview / Viva Point

Q : Why sequential edge detector is preferred over combinational?

A : Because a Sequential detector produces a clean one-clock pulse independent of gate delays and avoids glitches.

STATE MACHINE

1. INTRODUCTION

- A state machine (or finite state machine) is a sequential digital system which can exist in a finite number of states.
- The next state depends on present state and inputs. Outputs may depend on state and/or inputs.
- Represented by State diagram, State table, State transition table etc.

Key Point

FSM remembers past history (through state) and decides future actions.

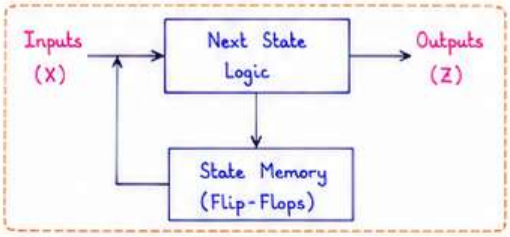
2. TYPES OF STATE MACHINE

(i) Finite State Machine (FSM)	(ii) Infinite State Machine
★ Finite number of states	★ Infinite number of states.
★ Next state determined by present state and inputs	★ Depends on continuous variables
★ Can be designed and implemented using FFs.	★ Cannot be implemented directly with FFs.
★ Examples: Controller, sequence detector	★ Examples: Analog integrator, continuous signal system.

3. NEEDS / IMPORTANCE OF FSM

1. Models systems whose behavior depends on order of events.
2. Used in control and sequencing operations.
3. Reduces complex problems into states and transitions.
4. Easy to analyze, design and implement.
5. Provides predictable and reliable operation

4. BASIC COMPONENTS OF FSM



Where:

X → Input variables
Z → Output variables
State Memory → stores present state
Next State Logic → determines next state
Outputs → depend on state and/or inputs

Types of Outputs

- ★ Moore Machine (Output depends only on state)
- ★ Mealy Machine (Output depends on state & inputs)

Moore Machine

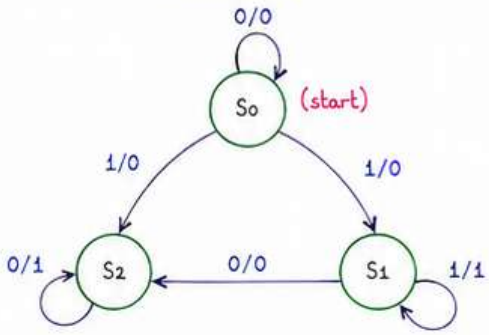
- ★ Output = f(state)
- ★ Output changes only when state changes.
- ★ More stable outputs.
- ★ More no. of states sometimes required.

Mealy Machine

- ★ Output = f(state, input)
- ★ Output can change immediately with input.
- ★ Fewer states usually.
- ★ Susceptible to glitches.

5. STATE DIAGRAM

- ★ Graphical representation of states and transitions.
- ★ Circles represent states.
- ★ Arcs represent transitions labeled with input/output.
- ★ '→' indicates initial (start) state.



Label Format

input / output
↓ (X) ↓ (Z)

6. STATE TABLE (EXAMPLE) (Mealy Machine)

Present State (PS)	Input X		Next State / (ONS/Z)
	0	1	
S0	S0/0	S1/0	S1/0
S1	S2/0	S2/0	S1/1
S2	S2/1	S2/1	S0/0

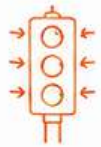
7. STATE TRANSITION TABLE

PS	X = 0	X = 1
S0	S0	S1
S1	S2	S1
S2	S2	S0

(Only next state information, without outputs)

8. APPLICATIONS OF FSM

- ★ Sequence detectors (1011 detector)
- ★ Traffic light controller
- ★ Elevator control system
- ★ Vending machine controller
- ★ Communication protocol control
- ★ CPU control unit
- ★ Digital clock, Timers, Counters



Summary / Key Points

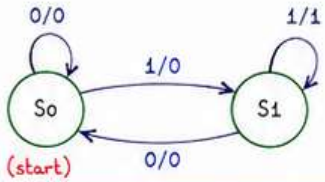
- ★ State machine is a sequential system with finite states.
- ★ Two types: Finite state machine (realizable) and Infinite state machine (not realizable with FFs).
- ★ Next state depends on present state and inputs
- ★ Two output models: Moore (Z = f(state)), Mealy (Z = f(state, input)).
- ★ Widely used in control and sequencing applications

Design Steps (General)

1. Define inputs, outputs and system requirements.
2. Determine number of states
3. Draw state diagram
4. Create state table / transition table
5. Derive state equations and output equations.
6. Implement using Flip-Flops and logic gates
7. Verify / test the design

Digital Example (Very Simple)

Detect sequence '10' (Mealy):



Interview / Viva Point

Q: Difference between Moore and Mealy machine?

A: Moore output depends only on state and changes with state. Mealy output depends on state and input and can change immediately with input.

Finite State Machine (FSM)

1. Introduction

- FSM is a sequential circuit having a finite number of states.
- Output depends on present state and input (and in some models on past outputs).
- It is used to control the sequence of operations in digital systems.

2. Types of FSM

Finite State Machine

- Has finite number of states (N = finite).
- After some inputs it may return to a previous state (cyclic).

Infinite State Machine

- Has infinite number of states (N = ∞).
- Example: Counter with unbounded counting. (Generally not realizable in digital hardware)

Key Point

All practical digital systems use Finite State Machines.

3. Need of FSM

→ To model systems whose behaviour depends on past history.
→ To control the sequence of events.
→ To design controllers for communication, processors, traffic systems, etc.
→ Provides clear structure (states + transitions).
→ Easy to implement using flip-flops and logic gates.

4. Classifications of Finite State Machine

Finite State Machines are of two main types based on output:

Implicit (Mealy) Model

- Output depends on present state and present input.
- Output is associated with transitions.
- Faster response.
- May have more glitches.

Explicit (Moore) Model

- Output depends only on present state.
- Output is associated with states.
- No glitches (stable within state).
- One clock cycle delay in output.

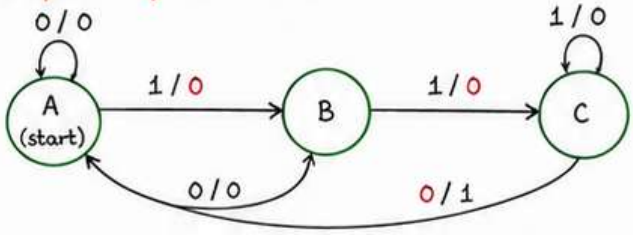
Notation

- For Mealy: next_state/output
- For Moore: output inside the state (edge label shows input)

5. Examples with State Diagrams

(a) Mealy Machine Example

Example: Sequence Detector for '110'



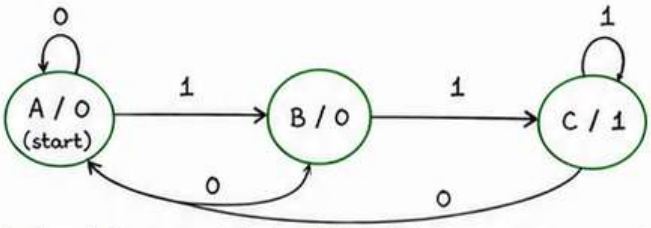
Detects the sequence 1-1-0. Output = 1 when 110 is received.

→ State Table (Mealy)

Present State	Input = 0		Input = 1	
	Next State	Output	Next State	Output
A	A	0	B	0
B	A	0	C	0
C	A	1	C	0

(b) Moore Machine Example

Example: Sequence Detector for '110'



Output = 1 is generated in state C (after 110 detected).

→ State Table (Moore)

Present State	Output	Next State	
		Input = 0	Input = 1
A	0	A	B
B	0	A	C
C	1	A	C

6. Applications of FSM

- Sequence detectors
- Traffic light controllers
- Elevator control
- Vending machines
- CPU control unit
- Communication protocol controllers
- Cryptography
- Robotics and automation

7. Advantages

- Handles complex sequential logic easily.
- Easy to design using state diagrams.
- Modular and scalable.
- Reduces number of flip-flops by state encoding.

Limitations

- Design becomes complex for large number of states.
- Requires careful state assignment.

8. Design Steps (General)

1. Define the problem clearly.
2. Identify I/P and O/P.
3. Specify the states and state diagram,
4. From state diagram → state table.
5. Obtain state assignment (binary codes).
6. Derive FF excitation table.
7. Minimize using K-map.
8. Draw the circuit diagram.

Summary / Key Points

1. FSM has finite number of states.
2. Mealy: Output depends on present state and input.
3. Moore: Output depends only on present state.
4. Mealy → faster, Moore → stable output.
5. FSM used in control & sequencing applications.
6. Design uses state diagram → state table → circuit.
7. State encoding reduces hardware complexity.

Interview / Viva Point

★ Why Moore output is more stable?
Ans: Because output depends only on state, not on input, hence no glitches.

✦ FINITE STATE MACHINE EXAMPLES ✦

1) Sequence Detector (Overlapping & Non-Overlapping), 2) Traffic Light

1. SEQUENCE DETECTOR

A sequence detector is an FSM that monitors an input bit stream and asserts an output = 1 when a specified sequence occurs.

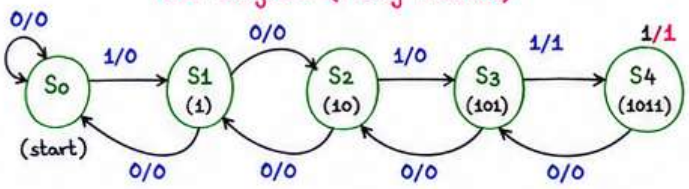
Example : Detect sequence "1011"

Mealy Machine is used (output depends on state & input).

1.1 OVERLAPPING SEQUENCE DETECTOR

Overlapping : After detection, last bits are reused to detect next sequence.

FSM Diagram (Mealy Machine)



State Assignment

S0	S1	S2	S3	S4
00	01	10	11	100

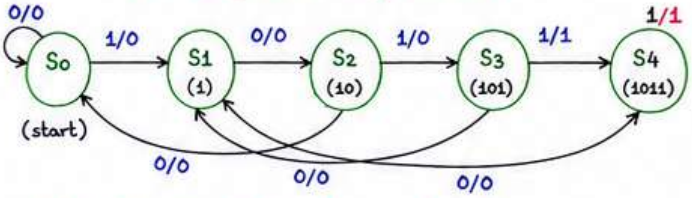
State Transition Table (Overlapping)

Present State	Input X	Next State (Q2 Q1 Q0)		Output Z
		X = 0	X = 1	
S0 (00)		00	01	0
S1 (01)		00	10	0
S2 (10)		01	11	0
S3 (11)		10	100	0
S4 (100)		10	01	1

1.2 NON-OVERLAPPING SEQUENCE DETECTOR

Non-overlapping : After detection, machine returns to start state (no reuse of last bits).

FSM Diagram (Mealy Machine)



State Transition Table (Non-overlapping)

Present State	Input X	Next State (Q2 Q1 Q0)		Output Z
		X = 0	X = 1	
S0 (00)		00	01	0
S1 (01)		00	10	0
S2 (10)		00	11	0
S3 (11)		00	100	0
S4 (100)		00	00	1

Key Point

- Overlapping detector reuses last bits → faster detection.
- Non-overlapping detector resets → simpler but slower.

2. TRAFFIC LIGHT CONTROLLER

A traffic light controller controls 3-way intersection signals in a periodic manner.

Signals : NS (North-South), EW (East-West)

G = Green (go)

Y = Yellow (caution)

R = Red (stop)

2.1 DESIGN STEPS

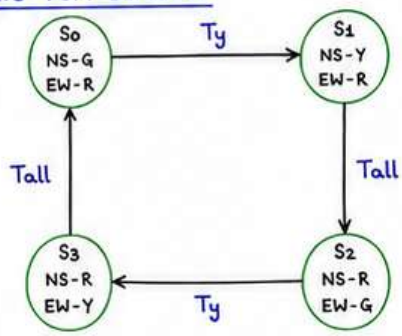
① Define the problem and outputs.
② List the states based on light sequence.
③ Draw state diagram (cyclic).
④ Assign state codes.
⑤ Prepare state table.
⑥ Implement using flip-flops and logic.

2.2 STATE SEQUENCE (Typical)

S0 (NS-Green, EW-Red) →
S1 (NS-Yellow, EW-Red) →
S2 (NS-Red, EW-Green) →
S3 (NS-Red, EW-Yellow) →
back to S0

One cycle: S0 → S1 → S2 → S3 → S0

2.3 FSM DIAGRAM



NS → North-South
EW → East-West
Ty → Yellow timer
Tall → All-Red timer

2.4 STATE TABLE (Outputs)

Present State	Outputs						Next State
	NS Lights			EW Lights			
	G	Y	R	G	Y	R	
S0 (NS-G/EW-R)	1	0	0	0	0	1	S1
S1 (NS-Y/EW-R)	0	1	0	0	0	1	S2
S2 (NS-R/EW-G)	0	0	1	1	0	0	S3
S3 (NS-R/EW-Y)	0	0	1	0	1	0	S0

Summary / Key Points

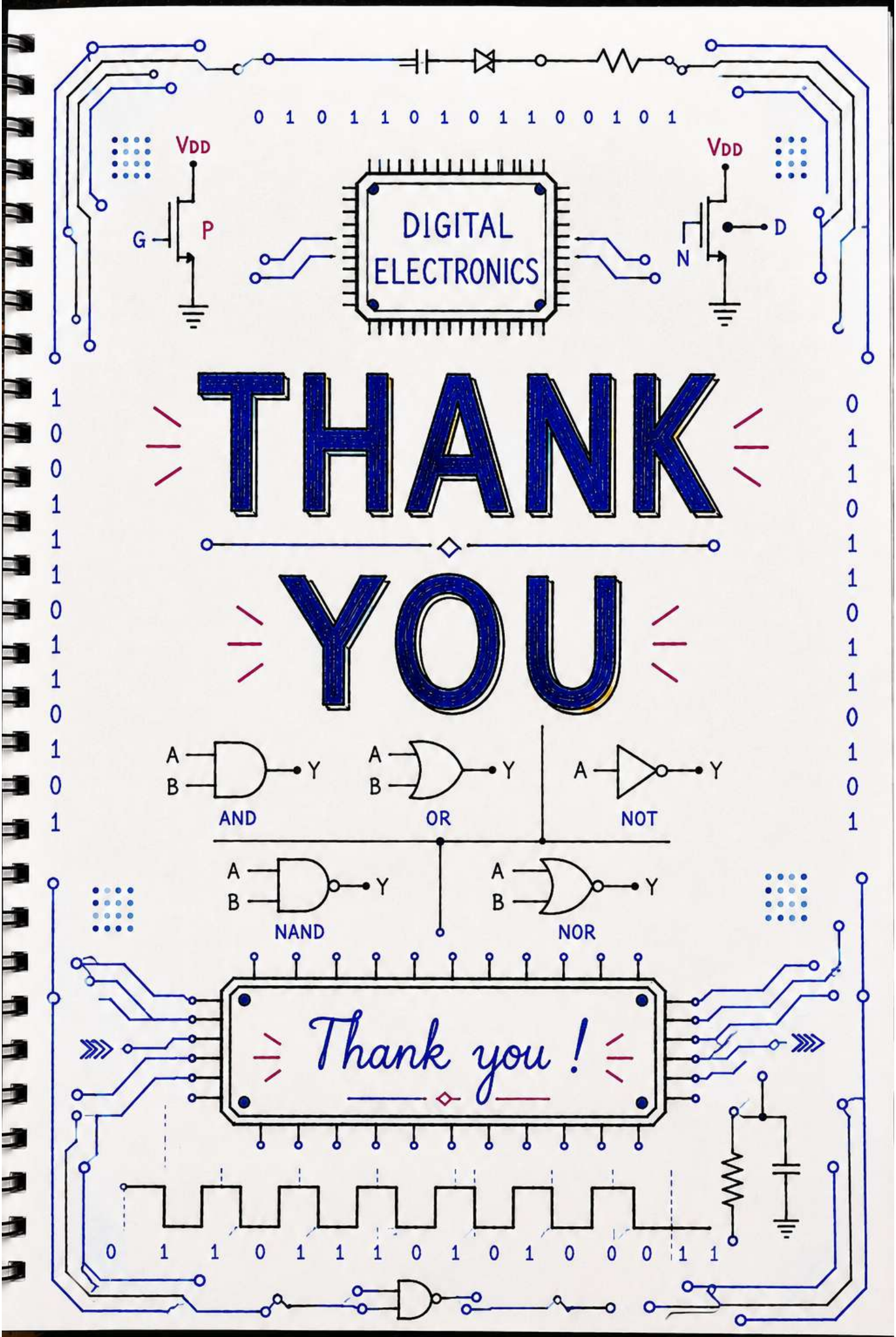
- FSM is specified by states, inputs, outputs & transitions.
- Sequence detector checks input streams for a pattern.
- Overlapping detector reuses last bits for next detection.
- Non-overlapping detector resets after detection.
- Traffic light controller is a real-world cyclic FSM.
- States are encoded and implemented using Flip-flops.
- Timers (Ty, Tall) control the duration of each state.

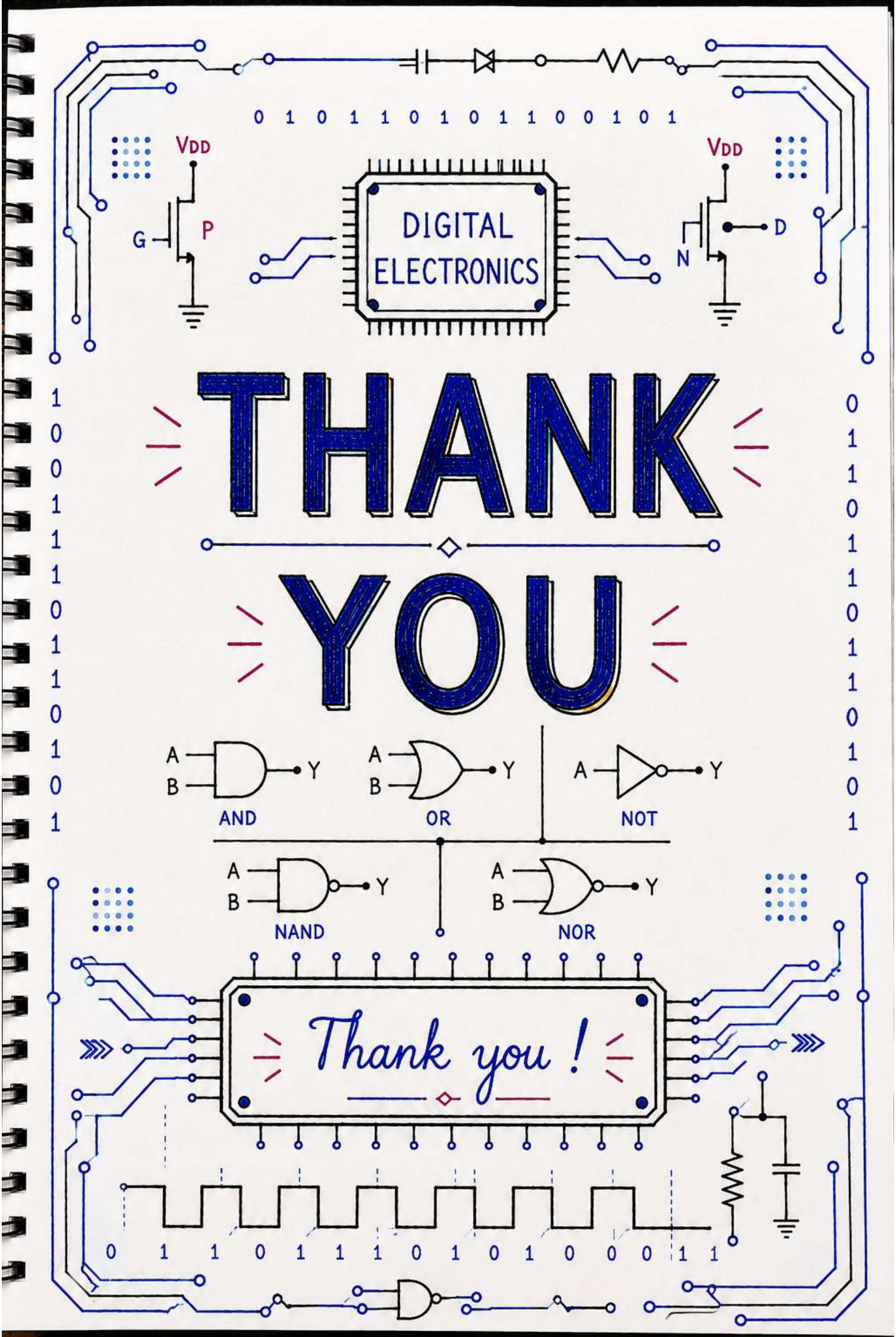
Interview / Viva Point

→ Why Mealy machine is preferred in sequence detectors?
(Answer: Immediate output when sequence occurs.)

→ Why is All-Red time needed in traffic light controllers?
(Answer: For safe transition, avoids collision.)

Author : Konda Tharun Vivekananda


0 1 0 1 1 0 1 0 1 1 0 0 1 0 1
VDD
G
P
DIGITAL ELECTRONICS
VDD
N
D
THANK YOU
1 0 0 1 1 1 0 1 0 1 1 0 0 1 0 1
0 1 1 0 1 1 0 1 0 1 1 0 1 0 1
A
B
Y
AND
OR
NOT
NAND
NOR
Thank you !
0 1 1 0 1 1 1 0 1 0 1 0 0 0 1 1


0 1 0 1 1 0 1 0 1 1 0 0 1 0 1
VDD
G
P
DIGITAL ELECTRONICS
VDD
N
D
THANK YOU
1 0 0 1 1 1 0 1 0 1 1 0 0 1 0 1
0 1 1 0 1 1 0 1 0 1 1 0 1 0 1
A
B
Y
AND
OR
NOT
NAND
NOR
Thank you !
0 1 1 0 1 1 1 0 1 0 1 0 0 0 1 1